

Verifiable Financial and Security Primitives for the Agentic Economy

*A pay-per-call Agentic Service Provider on the x402 / ERC-8004 stack:
the verifiable risk brain for autonomous agents — liquidation, cross-venue portfolio, options
greeks & margin, sizing and execution-quality — with options, transaction-safety, and micro-
structure intelligence.*

TECHNICAL DOCUMENTATION · Version 1.0 · July 2026

Quiver | Agentic Service Provider #5152 (ERC-8004, X Layer)

Live endpoint: <https://quiver-production-c3a8.up.railway.app>

Source & research artifacts: <https://github.com/Tristan-tech-ai/Quiver>

Contact: datascntstrader@gmail.com · Submitted to the OKX AI Genesis Hackathon

Abstract

Registered on X Layer as ERC-8004 agent #5152 under the name *Veritape* and renamed *Quiver* on 19 July 2026, so the on-chain record and the repository (github.com/Tristan-tech-ai/Quiver) carry different names for the same agent. Engine build behind every proof here: `q1-e1fa99d08887d6cc`.

Autonomous trading and wallet agents now move real value on a loop — on the order of 69,000 agents and 169M+ x402 transactions in 2026^{[40],[41]} — yet they cannot perform the deterministic financial mathematics that keeps them solvent: in a benchmark of numerical reasoning over real financial tables, general-purpose models fail 10–20% of the most complex multi-step calculations even at the top of the field, and the smaller open-weight models fail almost all of them^[42], and an autonomous agent lost 62% of its capital trading perpetuals with no human oversight^[43]. The agent risk layer now shipping from exchanges and startups enforces user-set *limits* but does not *compute* the numbers those limits are checked against, and most agent-facing services return figures whose derivation cannot be verified. Quiver is an Agentic Service Provider (ASP) that supplies the missing layer: twenty-two priced quantitative computations, the deterministic subset marked row by row in Table 1, quantitative computations over the x402 payment protocol^[27] and the ERC-8004 trustless-agent registry^[26], spanning options analytics, liquidity-provision and market microstructure, transaction-safety analysis, prediction-market execution, and a **Risk Brain** — perpetual-futures liquidation and funding risk, fractional-Kelly position sizing with risk-of-ruin, and execution-quality (fair-fill) verification. Its defining property is **verifiability**, and it comes in two grades that this paper is careful not to conflate. Where a computation is deterministic, the answer ships a *proof envelope* — the echoed inputs, the exact code identity, a content hash, and a self-check that tests the output against the condition it was solved from — which catches an implementation error but shares its assumptions with the model, so it cannot catch a wrong input or a wrong model, as Section 5.19 states plainly — (the liquidation condition, the Kelly optimality FOC, the constant-product k , the martingale $E[S_T]=F$, the arbitrage-free density). Where an answer depends on a live market read, no re-run promise is possible and the service says so: it ships a signed, timestamped *observation envelope* instead, and the distinction is enforced in code rather than left to the reader. The caller re-runs the open engine and reproduces the number rather than trusting the provider — a guarantee LLM-based tools cannot cheaply make. The options service fits an arbitrage-free SVI volatility surface^{[5],[6]} per expiry and recovers the risk-neutral density by the Breeden–Litzenberger identity^[4]; on live Deribit chains it produces a full terminal-price distribution whose mean reproduces the forward to within 0.001% (the martingale condition), and a probability ladder that corrects the standard $N(d2)$ figure — a correction whose size is a function of the skew it is fed, and which was measured at up to five percentage points on a smile we *constructed* with realistic skew rather than captured from a venue, so that figure is not a live-market measurement and Section 5.3 says which it is. The transaction-safety service simulates an unsigned transaction and also decodes EIP-712 permit signatures^{[20],[21]}, closing the signature-drainer gap that transaction simulators structurally cannot see. The tape service computes Kyle's lambda^[10], Amihud illiquidity^[11], and VPIN^[12] on the live DEX tape. Every quantitative claim is locked by an automated suite of 386 model-free tests — many of which provably fail on the pre-fix code — and, beyond the suite, the risk engine was validated against the real market at population scale: replayed over every open position on a live derivatives venue before the October 10, 2025 crash (79,386 accounts, \$14.6B), and then held to a **pre-registered, out-of-sample** test on two 2026 crashes it had never seen, where distance-to-liquidation — read from the venue's own published liquidation price rather than computed by this engine, a distinction Section 6 draws explicitly because it bounds what the study proves — predicted liquidation out-of-sample: flagged accounts were liquidated at **2.1× the base rate**, catching about nine in ten of the accounts it could see beforehand — about two in three of every liquidation once accounts that opened after the snapshot are counted — while flagging four in ten of all accounts, which expressed as a relative risk against the cleared cohort is 14.3× and 13.3× (Section 6). Two qualifications belong with that number rather than after it: the flag fires on 41.6% of accounts in February and 43.8% in June, and an ablation reported in the same section shows the result reduces to raw distance-to-liquidation, with the beta scaling contributing nothing measurable, which also means the naive baseline a sceptic would ask for was present in the study from the start rather than absent from it. Every one of the twenty-two services has also been purchased end-to-end with real money on both payment rails and its response verified client-side. The guiding principle is stated plainly and enforced throughout: where a signal cannot be grounded in observable data, or verified against an invariant, the service refuses to emit it rather than fabricate precision.

At a Glance

One page, for a reader with thirty documents to get through. Everything here is expanded, sourced, or qualified later; nothing here is stated more strongly than the section it points to.

What it is	An Agentic Service Provider: twenty-two priced computations an autonomous agent calls over HTTP, pays for in-band with x402, and discovers through ERC-8004. No chat, and no human workflow: the one service that renders an image does so for an agent to attach to a message.
Who calls it	Trading and wallet agents, inside a decision loop. Prices run 0.005 to 0.05 USD per call, cheap enough to poll before acting.
The one idea	An agent should not have to trust a number it just bought. Deterministic answers ship a <i>proof envelope</i> (echoed inputs, code identity, content hash, and a self-check against the condition the answer was solved from) that lets the caller re-derive the result. That check catches an implementation error; it cannot catch a wrong input or a wrong model, and Section 5.19 says so. Live-market answers cannot make that promise, so they ship a signed, timestamped <i>observation envelope</i> instead, and say which they are.
What it computes	Liquidation distance and funding drag, correlated-crash portfolio stress, Kelly sizing and risk of ruin, arbitrage-free options analytics and greeks, execution-quality checks, LP and treasury risk, event volatility, transaction and EIP-712 signature safety, DEX microstructure, prediction-market fills, protocol health, and on-chain attestation of a day's answers.
The build behind every proof	Every proof in this document comes from engine build q1-e1fa99d08887d6cc, served live at /build. It replaced q1-bce7e7bccb16ea1b on 26 July 2026 to close four correctness defects an adversarial reviewer with live access found; the worked proof in Appendix C was regenerated on the new build and re-verified end to end, and Section 11.5 records what changed and why the earlier freeze was broken. Two unfinished <i>improvements</i> remain held back (Section 11.4) — a defect and an enhancement do not meet the same bar.
Checkable in thirty seconds	Agent listing on X Layer, one x402 settlement on each rail, an EAS schema on Base, the build hash served at /build, and a worked proof whose content hash and signature a reader can reproduce. All in Appendix C, with the explorer links and the checks.
Strongest evidence	A population-scale replay of the October 2025 crash and two out-of-sample 2026 crashes: flagged accounts were liquidated at 14.3× and 13.3× the rate of cleared ones — on a flag that fires on 41.6% and 43.8% of accounts, which is the number that belongs in the same breath as the ratio (Section 6).
Strongest counter-evidence, ours	Our own ablation shows that result reduces to raw distance-to-liquidation; the beta scaling we built a calibration study for contributes nothing measurable. That distance is the <i>venue's own published</i> liquidation price, not one this engine computed, so the study measures the quantity's predictive power and not our computation of it — which also means the naive baseline a sceptic would ask for was in the study all along, unlabelled. The flag also fires on 42–44% of accounts. All three are in Section 6, not buried.
Traction, honestly	Small, and measured on chain rather than from our own counter. Over the eight days to 27 July 2026, six payer addresses that are not ours sent 44 payments totalling 0.575 USD to the advertised payTo, and four of the six paid more than once — one returning across 2.55 days. Half a dollar is not a business. An earlier version of this row said three wallets and zero recurrence; that came from an in-memory instrument that resets on every deploy, in a document whose argument is that a number you cannot re-derive is a number you should not trust. Everything else is our own disclosed quality-assurance traffic, never counted as sales. The buyer audit in Section 6.5 was commissioned by us, so it is arm's length, not independent.
What it does not do	It refuses to output a directional edge, refuses to infer dealer positioning it cannot measure, reports a variance premium as insignificant because it is, and answers DATA_UNAVAILABLE for free rather than guessing. Section 8 labels every limitation as either structural or scheduled.
Where it runs	One container, one region, trial tier. Availability is measured from outside and published including the outages: 99.63% of cycles clean over the measured record in Section 8. Redundancy is roadmap work, not a claim.

Contents

1 Introduction

- 1.1 The gap: agents can pay, but there is little worth paying for
- 1.2 Design stance and contributions

2 System Architecture

- 2.1 The x402 payment loop
- 2.2 Identity and discovery under ERC-8004
- 2.3 Multi-service host and data plane

3 Design Principles

3.1 Two questions this design invites, answered before they are asked

4 Service Catalogue

4.1 options-desk

4.2 calldata-x

4.3 chart-press

4.4 tape-pulse

4.5 poly-fill, poly-desk, updown-pulse

4.6 protocol-pulse, macro-sentry, loop-digest

4.7 token-scan, wallet-audit

4.8 lp-desk

4.9 Risk Brain: perp-gate, size-gate, exec-verify, options-risk, portfolio-gate (lp-risk, treasury-risk, event-vol and risk-attest are specified in Section 5)

5 Methodology

5.1 Black-76 and the option greeks

5.2 The arbitrage-free volatility surface

5.3 Risk-neutral density and the full distribution

5.4 Variance risk premium

5.5 Tape microstructure estimators

5.6 Dealer gamma exposure and barrier probabilities

5.7 Transaction and signature safety

5.8 Model-free calendar bounds by martingale optimal transport

5.9 Concentrated-liquidity value and divergence loss

5.10 Perpetual liquidation and funding

5.11 Position sizing and the probability of ruin

5.12 Execution quality and adverse fills

5.13 Options-book greeks and scenario margin

5.14 Liquidity-provision divergence and its LVR rate

5.15 Stablecoin-treasury concentration and depeg

5.16 Event-implied expected move

5.17 Batched attestation

5.18 Cross-venue portfolio exposure and correlated-crash stress

5.19 The proof envelope: correctness you re-derive

6 Verification and Testing

6.1 Validation against the real market

6.2 The same result at its least flattering

6.3 An ablation the result did not survive intact

6.4 Independent buyer verification, and a defect it found in the money path

7 Worked Walkthrough

8 Limitations and Honest Disclosures

9 Related Work and Positioning

10 The Build: Story, Process, and a User Scenario

10.1 Starting from a gap

10.2 Choosing the set

10.3 Holding a standard, and what it caught

10.4 The record, dated

10.5 A day in an agent's loop

11 Roadmap: Operating Quiver After the Hackathon

11.1 Standing commitments

11.2 The single metric that governs the plan

11.3 Sequence

11.4 Known gaps that are engineering work, not limits

- 11.5 Defects a live adversarial test found after the freeze, and how they were closed
- 11.6 A second review, and the reason its findings are worse than the first's
- 11.7 A third round, and the pattern that predicts what is still there
- 11.8 Research programme
- 11.9 What the proof changes, and what it does not — the year after
- 11.10 What would falsify this plan

12 Conclusion

A Appendix A: API Reference

B Appendix B: Reproducibility

C Appendix C: Checkable Artifacts

References

1. Introduction

A new class of software buyer has appeared on-chain. Autonomous agents now hold wallets, read markets, and sign transactions on a schedule, and with the arrival of the x402 protocol ^[27] they can pay for a service in-band over HTTP, settling in stablecoin without an account or a human in the loop. The supply side has not caught up. An agent that wants the risk-neutral probability that Bitcoin closes above a strike, or a verdict on whether a pending signature will drain its wallet, has almost nowhere to buy that computation in a form it can call and trust.

1.1 The gap: agents can pay, but there is little worth paying for

Two failures are common in the services that do exist. The first is reachability. The quantitative tooling that desks rely on, from volatility-surface fitters to market-impact models, lives in notebooks and internal libraries that expose no priced, machine-callable interface. An agent cannot invoke it. The second failure is trust. Many agent-facing endpoints return a number, a grade, or a score with no way to check how it was produced. A "health score of 73" or a "buy signal" is worthless to a counterparty that cannot audit the derivation, and worse than worthless if the number was fitted to look good. For financial and security computations, where a wrong answer moves real money, an unverifiable output is a liability.

Quiver is built for that gap. It is an Agentic Service Provider: a single host that publishes twenty-two independently priced computations, each callable by any agent that speaks x402, each registered on-chain under ERC-8004 ^[26] so it can be discovered and its reputation accrued. The services are not thin wrappers over a data feed. They implement the methods a quantitative desk or a security team would actually run, and they are written to a standard this document makes explicit: correctness is proven by test, claims are grounded in observable data, and when the data does not support a claim the service says so.

1.2 Design stance and contributions

The stance is adversarial toward its own output. Before a number is served, we ask what would have to be true for it to be wrong, and we check that against independent data. This produced several corrections during development that a looser process would have shipped. The plain $N(d2)$ risk-neutral probability, standard in textbooks, carries a bias of up to five percentage points on a skewed crypto smile because it omits the volatility-slope term; we measured that bias on a smile constructed for the purpose, since the correction's size is a function of the skew it is given, and replaced the formula. A per-trade Amihud illiquidity ratio returned a value of order 10^9 on a live memecoin tape because sub-cent dust trades drive the denominator to near zero; ground-truthing caught it and the estimator was rebuilt on volume blocks. This document reports those corrections rather than hiding them, because the discipline that produced them is the product.

The contributions are the following.

1. A production ASP of twenty-two priced services on x402 and ERC-8004, live and paid end-to-end, covering options, transaction and token safety, microstructure, prediction markets, concentrated-liquidity economics, protocol risk, and a verifiable risk brain — perpetual-futures liquidation and funding, cross-venue portfolio risk (net exposure, nearest-liquidation, and a correlated-crash stress), fractional-Kelly sizing with risk-of-ruin, and execution-quality (fair-fill) verification.
2. A **proof envelope** on every risk computation: because the engines are deterministic, each answer carries its inputs, code identity, a content hash, and a self-check against the condition the answer was solved from, so a caller re-runs and reproduces the number instead of trusting the provider. What that self-check does and does not establish is stated precisely in Section 5.19: it catches an implementation error, and it shares its assumptions with the model, so it cannot catch a specification error.
3. An arbitrage-free options analytics stack: raw SVI fitted per expiry with the Gatheral–Jacquier butterfly and Roger Lee wing conditions enforced inside the objective ^{[6],[8]}, the calendar condition enforced across expiries on the *shipped* surface rather than on a full-precision object the caller never receives, and the Breeden–Litzenberger density ^[4] policed by a coverage-honest butterfly check that reports the strike range it actually verified and withholds its verdict where the smile could not be evaluated.
4. Model-free no-arbitrage price bounds on calendar and forward-starting option payoffs via martingale optimal transport (Beiglböck, Henry-Labordère and Penkner^[39]): given two fitted expiry marginals, a linear program returns the tightest lower and upper prices consistent with *every* arbitrage-free model, and its feasibility is exactly the Strassen convex-order condition certifying that the two smiles are jointly free of calendar arbitrage — a bound nobody assumes a dynamics to produce.
5. A concentrated-liquidity reality-check that reconstructs a Uniswap-V3 position's realised fees against its divergence loss from the pool's live on-chain swap history, read over a keyless public archive node, so the fee-versus-loss verdict is measured on real flow rather than assumed.

6. A transaction-safety service that combines on-chain simulation with EIP-712 permit-signature analysis ^[20], addressing the dominant modern drainer vector that pure simulators cannot observe.
7. Real market-microstructure estimators on the DEX tape: Kyle's lambda, Amihud illiquidity, and VPIN, with quality gates that return null rather than a false number on thin data.
8. A verification methodology: 386 automated tests of model-free invariants, of which 381 run on every build and five are live-archive integration tests skipped unless an archive RPC is configured, a ground-truthing protocol against live venues, and a pre-registered out-of-sample validation of the risk engine against real market crashes (Section 6) — with every reported figure reproducible by a documented request.

The remainder of the document is organised as follows. Section 2 describes the payment, identity, and hosting architecture. Section 3 states the design principles that constrain every service. Section 4 catalogues the twenty-two services. Section 5 gives the mathematics. Section 6 describes verification. Section 7 walks through live calls end to end. Section 8 is a candid account of what the services do not do. Section 9 positions the work, Section 10 recounts how it was built and how an agent uses it, Section 11 sets out what happens after the hackathon and what would falsify that plan, and Section 12 concludes.

2. System Architecture

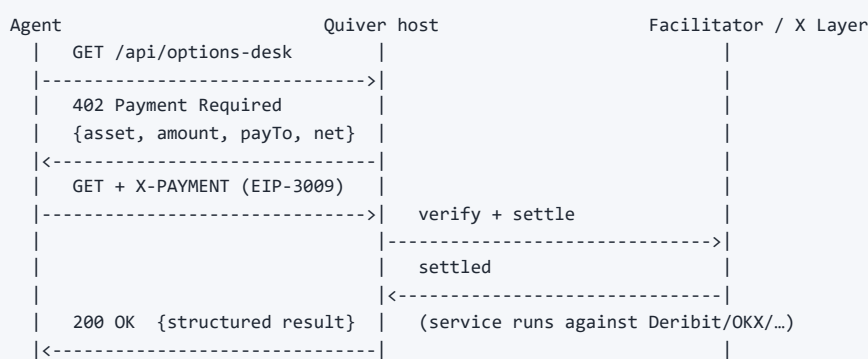
Quiver is one HTTP server that hosts many services. Each service is a pure function from a typed request to a JSON response, gated by a payment middleware and registered as a callable capability. The design keeps the surface small: a request arrives, payment is verified, the service runs against live venue data, and a structured result returns in a single round trip.

2.1 The x402 payment loop

Payment uses x402, the protocol that revives the dormant HTTP 402 status code as a settlement channel ^[27]. The flow has three messages. An agent requests a paid route; the server answers 402 Payment Required with a challenge that names the asset, amount, recipient, and network. The agent constructs a payment authorization and resends the request carrying it. The server verifies the authorization through a facilitator and, once settled, executes the service and returns the result. Quiver implements the v2 *exact* scheme, in which the client authorizes a transfer of a fixed amount to a fixed recipient. That scheme is built on EIP-3009 *transferWithAuthorization* ^[24], so the transfer is gasless and signed off-chain; the token is USD~~F~~0 on X Layer (chain eip155:196) via the OKX facilitator, with a second settlement rail in USDC on Base (chain eip155:8453) through the Coinbase CDP facilitator, offered on the same 402 challenge. Prices are per call and range from 0.005 to 0.05 USDT, low enough that an agent can poll a service inside a decision loop without material cost.

The payment gate is enforced before any business logic. A route that is called without a valid authorization, whether by an honest probe or by the reviewer's conformance tool, receives the mandatory 402 challenge rather than a partial result or an error. This property is checked directly: at the time of writing, every paid route returns 402 to an unauthenticated request.

FIGURE 1 — THE X402 REQUEST LIFECYCLE FOR A PAID QUIVER CALL



2.2 Identity and discovery under ERC-8004

An agent economy needs a way for one agent to find another and to judge whether it is worth transacting with. ERC-8004, the Trustless Agents standard ^[26], provides this with three on-chain registries: Identity, Reputation, and Validation. Identity is a portable ERC-721 token whose metadata points to a registration file describing the agent and its services. Reputation collects client feedback; Validation records independent checks of delivered work. Quiver is registered as ASP #5152 on X Layer under this standard, which makes its twenty-two services discoverable to any agent browsing the registry and lets a track record ac-

crue against a stable identity rather than a URL that can change. ERC-8004 is deliberately silent on payment and execution [26], which is what allows it to compose with x402 for settlement and with the A2MCP calling convention for invocation. The nine deterministic risk-brain engines are additionally exposed over a remote Model Context Protocol [29] (MCP) endpoint that any MCP-speaking agent can list and call directly, at <https://quiver-production-c3a8.up.railway.app/mcp>, registered on the public MCP registry as `quiver-risk-brain`.

2.3 Multi-service host and data plane

Every service is defined by a name, a price, an input schema, a validator, and a runner. The runner is asynchronous and calls out to the venue that owns the ground truth for that service: Deribit for options, the OKX DEX market API for token tape and candles, EVM JSON-RPC nodes for transaction simulation, the Polymarket Gamma and CLOB APIs for prediction markets, and DefiLlama for protocol data. Results are computed on demand; three caches exist, and each is named with its staleness bound rather than described as absent: a per-currency variance-risk-premium estimate held for twelve hours, which survives a transient upstream failure rather than turning it into a refusal; a rendered-chart store keyed by request so an image can be fetched again by URL; and the in-memory cursor store behind `loop-digest`, which expires after twenty-four hours and does not survive a restart, a limitation the service discloses in its own response rather than hiding. Nothing about a token's price, a book, or a market read is cached.

The host is stateless between calls except for the three stores named above that exist for good reasons: a rendered-chart store that lets an image be fetched by URL after the call that produced it, and a cursor store for the `loop-digest` service that lets an agent diff a wallet against its own previous call. Both are ephemeral and per-session. The separation of data plane from compute keeps each service auditable: given the same venue state, the same request produces the same result, and the result names the block or the tape window it was computed on so a caller can reproduce it.

Latency is bounded by the venue round trip rather than by the computation. Measured end to end, the transaction decode and simulation returns in roughly 125 to 160 milliseconds, the fast-tier chart in 220 to 300, and the full options desk, which fetches and normalises an entire Deribit chain and fits a surface across expiries, in 550 to 700. These are within the budget an agent can absorb inside a decision loop, which is the point of pricing calls low enough to poll. Every response carries an `elapsedMs` field so a caller can hold the service to its own timing.

Service	Function	Primary data source	Price (USDT)
options-desk	Options analytics: surface, greeks, RND, GEX, VRP, cross-market	Deribit v2, OKX	0.01
calldata-x	Transaction simulation and EIP-712 signature safety	EVM JSON-RPC	0.005
chart-press	Server-rendered PNG chart with indicators and drawings	OKX DEX candles	0.02
tape-pulse	DEX tape microstructure: Kyle λ , Amihud, VPIN	OKX DEX trades	0.01
poly-fill	Order-book fill simulation and market impact	Polymarket CLOB	0.01
poly-desk	Polymarket wallet book and unrealised PnL	Polymarket	0.01
updown-pulse	Short-window up/down market read (no fabricated edge)	Polymarket, OKX	0.01
protocol-pulse	DeFi protocol risk flags from TVL and hack history	DefiLlama	0.01
macro-sentry	High-impact US macro events ahead + the options-implied expected move to the next one	Curated calendar + ATM IV	0.005
event-vol	Options-implied expected move around an event (1σ + straddle $E \Delta S $ + prob-beyond) & event-isolation from the vol term structure, self-checked	Black-76 (deterministic)	0.01
loop-digest	Cursor-based wallet diff for agent loops	OKX DEX	0.01
lp-desk	Concentrated-liquidity fees vs divergence loss, replayed on real swaps	Uniswap-V3 (keyless RPC)	0.01
token-scan	Wash-trading share of DEX volume, with wallet/tx evidence	OKX DEX trades	0.05
wallet-audit	Authenticity grade of a wallet's PnL and win rate	OKX portfolio	0.05
perp-gate	Perp liquidation price, distance-to-liq & funding drag, with a liquidation-invariant self-check	Hyperliquid / dYdX (live) + inputs	0.01
portfolio-gate	Cross-venue net exposure per underlying, the leg that liquidates first, concentration (HHI), and a correlated-crash stress	Hyperliquid, dYdX (live)	0.05
size-gate	Fractional-Kelly size + risk-of-ruin, with a Kelly-optimality self-check	Edge inputs (deterministic)	0.01
exec-verify	Fair-fill / sandwich check — bps lost to adverse execution, with a constant-product self-check	Swap + pool state (deterministic)	0.01
options-risk	Portfolio greeks + SPAN-style scenario margin for an options book, self-checked against finite differences	Black-76 (deterministic)	0.02
lp-risk	Forward-looking LP <i>impermanent loss</i> — expected divergence versus holding at the horizon, deliberately not labelled LVR (Section 5.14) — plus fee breakeven, self-checked at the token level	Closed-form (deterministic)	0.01
treasury-risk	Stablecoin-treasury concentration (HHI), depeg and correlated-depeg stress, and risk-adjusted yield, self-checked	Deterministic	0.02
risk-attest	Merkle-batch of proof hashes → one root + inclusion proofs + an EIP-712 (EAS-ready) attestation for a single on-chain anchor	Deterministic	0.01

Table 1 — the twenty-two registered services, their data source, and per-call price.

3. Design Principles

3.1 Two questions this design invites, answered before they are asked

If the caller can re-run the engine to check the answer, why pay for the answer? The re-run is a check, not a substitute, and the difference is the whole reason the service exists. An agent that wants a liquidation price has three options: ask a language model to do the algebra, which the published benchmarks put at a 10–20% failure rate on multi-step calculation even for the strongest models, and near-total failure for smaller ones^[42]; implement the venue's margin regime itself, which means tracking notional-tiered maintenance rates, cross-margin account-value definitions, funding accrual and per-venue liquidation conventions that change without notice; or buy a number that arrives with a self-check proving it satisfies the venue's own liquidation condition. Verification is cheap because the artifact is small; getting it right the first time is not, which is precisely why the arithmetic is worth half a cent and the checking is free. The proof envelope does not exist so the caller can avoid paying. It exists so the caller can pay without trusting.

What is an observation envelope actually worth, given it cannot promise a re-run? Less than a proof, and it is important to say what the “less” consists of rather than leave it implied. An observation envelope establishes four things: which engine produced the answer, on which build, from which upstream source, at which instant, signed by a key published in the agent card. It establishes nothing about whether the number is right. What it buys a caller is accountability rather than correctness — the ability to hold a specific answer, from a specific version, at a specific time, against what the market later did, and to do so months later without the provider's cooperation. That is the honest ceiling of a live-data guarantee from any provider, and the reason this document separates the two envelope kinds in code rather than describing both as “verifiable”. A reader who concludes that the strong guarantee attaches to the arithmetic and the weak one to the data has read the design correctly; the response to that is not to overstate the weak one but to be explicit about which services carry which, which Table 1 does row by row.

Five principles constrain every service. They are not aspirations; each is enforced somewhere in the code and testable.

3.2 Ground truth over self-report

A computed signal is validated against an independent source or an analytic identity before it ships. The risk-neutral density is checked against the martingale condition that its mean must equal the forward. Analytic greeks are checked against finite differences of the price. The variance risk premium is checked against a significance test at the honest effective sample size, not the inflated count of overlapping windows. Where ground truth disagreed with a first implementation, the implementation changed.

3.3 Refuse to fabricate

When a quantity cannot be estimated from observable data, the service returns null with a reason rather than a plausible-looking number. The tape microstructure estimators return null on a tape too thin to support them. The options surface excludes near-expiry slices where a diffusive model does not apply, rather than force-fitting them. The short-window up/down service outputs no directional edge at all, because short-horizon direction is empirically indistinguishable from a coin flip and a fabricated fair value would contradict the product's own thesis.

3.4 Arbitrage-free by construction

The volatility surface is fitted with the no-arbitrage inequalities enforced inside the optimisation objective, so an accepted fit satisfies them by construction rather than by hope^[6]. Three conditions are enforced, not one: the butterfly condition per slice, the Roger Lee wing bound, and — the one most easily overlooked — the calendar condition *between* slices, checked on the rounded parameters the caller actually receives rather than on the full-precision fit, because rounding a slice can reintroduce a crossing the fit had eliminated. A fit that fails its quality gate is rejected, and the caller is told the fallback was used. A negative risk-neutral density, the signature of butterfly arbitrage, is detected and disclosed rather than silently served; the check reports the strike range over which it was able to evaluate the density and withholds a clean verdict where the smile could not be evaluated, so “no arbitrage detected” can never mean “detected none over the sliver I could see.”

3.5 Descriptive, not prescriptive

Services report what the market prices and what the data shows. They do not tell a caller what to do. A composite “grade” that implies a validated predictive model is avoided where no labelled dataset exists to validate it; the protocol-risk service was rebuilt from a fitted 0–100 score into a set of individually defensible risk flags for exactly this reason. Every output that touches a trading decision carries a not-advice disclosure.

3.6 Proven by test

The financial mathematics is locked by an automated suite of model-free invariants: put-call parity, the delta relationship, non-negativity of the risk-neutral density, the martingale property of the distribution, and the absence of look-ahead in every time-series indicator. A change that breaks an identity breaks the build. Section 6 describes the suite; it currently holds 386 tests, of which 381 run on every build and five are live-archive integration tests that are SKIPPED unless an archive RPC is configured. None of the 381 fails; the five are not run in the default environment, and calling that “all pass” — as this sentence did — counted a test that never executed as a passing one.

4. Service Catalogue

The twenty-two services fall into seven groups: options intelligence, transaction and token safety, charting, tape microstructure, prediction-market and protocol tooling, concentrated-liquidity economics, and a verifiable risk brain (perpetual liquidation, cross-venue portfolio risk, position sizing, execution-quality verification, and options-portfolio greeks & SPAN-style margin). Two are treated as the flagship of the set and specified in full; the rest are described at the level of what they compute, from where, and with what guarantees. Section 5 gives the mathematics that several of them share.

4.1 options-desk

/api/options-desk · 0.01 USDt

FLAGSHIP A single call returns a desk-grade read of the live crypto options market for BTC, ETH, or SOL, sourced from Deribit with an OKX cross-check. No comparable service exists in the agent registry; options analytics is the widest moat in the catalogue because the mathematics is unforgiving and the data is awkward to handle correctly. The response is organised into layers, each independently useful.

Implied view and the probability ladder. For the two front liquid expiries the service reports the at-the-money forward implied volatility, the one-sigma expected move in dollars and percent, and a ladder of price levels with the risk-neutral probability that spot finishes above each. The probability is not the textbook $N(d_2)$. On a skewed smile that figure is biased, because the true risk-neutral probability carries a term in the volatility slope; Section 5.3 derives it. The service reports the corrected probability and, alongside it, the exact size of the correction at each level, so the caller can see how far the naive number would have been wrong. An earlier draft asserted here that the correction “reaches several percentage points near the money” on a live chain, which was a claim this document nowhere evidenced — the only figure it had measured came from a constructed smile. So it was measured: on a live BTC chain captured for this document, the correction peaks at **+3.3 points** just below the money (63,000 and 64,000 against a spot near 65,000) and **changes sign in the upper wing**, reaching -0.8 points at 68,000. The sign flip is not incidental — it is what equation (8) predicts where the smile slope changes sign, and seeing it in live data is better evidence for the correction than its magnitude is.

Full risk-neutral distribution. Beyond point probabilities, the service integrates the Breeden–Litzenberger density into a normalised distribution and reports its quantiles from the 5th to the 95th percentile, the tail expected shortfall on each side, and the distribution's own volatility, skew, and excess kurtosis. The block carries a self-check: under the risk-neutral forward measure the terminal price is a martingale, so its expectation must equal the forward. The service reports the residual $E[S_T]/F - 1$; on a live BTC expiry this reads 0.001%, which is the strongest available evidence that the density is proper and correctly integrated, which is a weaker statement than correct. The distribution is emitted only when the density is a proper, arbitrage-free density that integrates to one; otherwise it is withheld with a diagnostic.

Dealer gamma and second-order greeks. The service computes dollar gamma exposure (GEX) by strike, the net across the chain, and the gamma-flip spot level where the sign changes, under the conventional assumption that dealers are long call gamma and short put gamma. That assumption is labelled as an assumption, and the reason it is not replaced by a flow-inferred dealer position is stated in the output: the feed does mark block trades — `block_trade_id` and `block_trade_leg_count`, plus `block_rfq_id` on ETH — so they are identifiable, and an earlier version of this paragraph and of the service's own output wrongly said otherwise. The obstacle is attribution, not tagging: a trade's reported direction names the side it was booked from, not whether a dealer was the buyer or the seller, and nothing in the feed says whether a block maker is a dealer at all. Since block-tagged trades are a large share of volume — 48.9% of BTC and 30.2% of ETH option contract volume in a 200-trade window sampled on 27 July 2026 — a flow-based sign would be invertible on exactly the largest trades. Refusing to infer what cannot be inferred is the honest choice. The greek surface includes the second-order volatility greeks vanna and volga, computed analytically and verified against finite differences, and aggregated into a dealer vanna and volga under the same disclosed convention.

Variance risk premium and its significance. The service estimates the premium of implied over realised variance from overlapping windows ^{[16],[17]} and, critically, tests whether it is statistically distinguishable from zero at the effective independent sample size rather than the inflated count of overlapping pairs. On live BTC the ratio of realised to implied volatility is

0.881 with a p-value of 0.254; the service reports that this is not significant and refuses to present the real-world-calibrated probabilities as anything but indicative. This is the single clearest example of the product holding itself to a real quant standard: it computes the premium, then declines to claim an edge the data does not support.

Cross-market divergence. The options market and Polymarket price the same events by different mechanisms. For each live Polymarket threshold market on the coin, the service computes the options-implied probability of the identical question and reports the divergence. Because the market's resolution date is rarely a listed option expiry, the arbitrage-free smile is interpolated to the exact tenor in total variance, linearly in time between the two bracketing expiries, which preserves the calendar no-arbitrage condition; each row discloses whether the smile was interpolated or extrapolated and across which expiries. The raw divergence is then decomposed into the part explained by the variance risk premium, which is structural and not an edge, and the residual dislocation.

Composite verdict. The layers above are fused into one continuous score per axis — volatility richness and directional tilt — with the entire construction disclosed in the response: the literal weights, each component's raw value and its bounded transform, and the renormalisation applied when a component is unavailable (missing inputs are listed, never guessed). Two details make the fusion defensible rather than decorative. The risk-neutral density's price-space skewness has a positive lognormal baseline of $(e^w + 2)\sqrt{(e^w - 1)}$ with $w = (\sigma\sqrt{T})^2$ — a market symmetric in log-returns still shows positive price skew — so the directional component scores the *excess* over that baseline, not the raw number. And when a variance-risk-premium fit exists, implied-versus-realised is measured against the asset's *own habitual* premium rather than against parity, since implied exceeding realised is the norm, not a signal. An axis with fewer than two live components refuses to emit a verdict; a leave-one-out pass names any single component whose removal would flip the reading, so the caller sees exactly how fragile the verdict is; and identity self-checks re-derive the published score from the published parts. The front slice for these reads skips sub-two-day expiries, whose annualised volatility is microstructure-dominated — a contamination that surfaced on a live morning chain precisely because the component values are disclosed, and was fixed and regression-locked the same day.

Model-free calendar bounds. A single smile prices vanilla options; it does not price a payoff that depends on two dates at once, such as a forward-starting straddle or a calendar spread. Rather than assume a stochastic-volatility dynamics to fill the gap, the service computes the tightest price interval consistent with *every* model that reprices both quoted expiries, by martingale optimal transport (Beiglböck, Henry-Labordère and Penkner^[39]): the two fitted marginals, mapped to the martingale coordinate $x = S/F$, become the constraints of a linear program whose optimum is a hard lower and upper no-arbitrage bound. The program is feasible if and only if the two marginals sit in convex order, which by Strassen's theorem is exactly the statement that the pair is free of calendar arbitrage — so the bound doubles as a certificate. Two exact identities gate the output: a payoff in the later date alone must integrate to one under the later marginal, and the expected increment $E[X_2 - X_1]$ must be zero. When either fails the bounds are withheld rather than shipped. Section 5.8 gives the construction. No other agent in the registry prices a two-date payoff without assuming a model.

Supporting layers include the implied-volatility term structure and rank against the DVOL history, coverage-honest skew, max pain, put/call open interest, and an implied-rate curve backed out per expiry. The service fetches and normalises the full Deribit chain on each call, fits the surface once, and reuses it across the layers so every number is consistent with a single arbitrage-free surface.

4.2 calldata-x

/api/calldata-x · 0.005 USDT

FLAGSHIP

This service answers one question an agent must resolve before it signs anything: what will this actually do to my wallet. It operates in two modes, and the second mode is the reason it matters.

Transaction mode. Given raw calldata, the service decodes the four-byte selector against a signature registry and renders the function and arguments in plain language. Given the full unsigned transaction, it goes further and simulates the transaction against live chain state with `eth_simulateV1`, reading the exact asset transfers and approval changes from the simulated Transfer and Approval logs rather than inferring them from the inputs. It reports the gas the transaction would consume, priced at the live gas price into a native-token fee. The target contract is checked for an upgradeable proxy: a non-zero implementation slot under EIP-1967^[22], or the legacy OpenZeppelin slot used by older proxies such as USD Coin, means the logic behind the address can be replaced by its admin after the caller approves it, so today's simulation is not binding on tomorrow's behaviour.

Signature mode. Modern wallet drainers rarely use a transaction. They use an off-chain signature: a Permit or Permit2 authorization under EIP-712^{[20],[21]} that grants a spender the right to move tokens, signed by the victim and submitted later by the attacker. Such a signature is never calldata and never appears in a transaction simulator, so a service that only simulates transactions is structurally blind to the most common way people are drained today. calldata-x decodes the typed-data pay-

load, identifies the permit type, extracts the spender, amount, and expiry, and flags the dangerous combinations: an unlimited allowance, a permit that never expires, a spender that is an externally owned wallet rather than an audited contract, or a domain that does not match the canonical Permit2 deployment. Closing this gap is the highest real-user value in the catalogue.

The verdict. Both modes feed a one-glance verdict keyed on the spender's reputation combined with the effect, not on the allowance number alone. An unlimited approval to a recognised router is routine and labelled a note; the same unlimited approval to a fresh externally owned wallet is a drainer pattern and labelled danger. Reputation is resolved from a router allowlist, from `eth_getCode` which distinguishes a contract from a wallet and detects EIP-7702 delegated accounts ^[23], and from an on-chain activity signal. This keying is what removes the false positive that a naive "unlimited approval is always dangerous" rule produces on every legitimate DeFi interaction.

4.3 chart-press

/api/chart-press · 0.02 USDT

An agent that reasons over price action needs a chart it can attach to a message, and it needs the numbers on that chart to be trustworthy. `chart-press` renders a PNG from OKX DEX candles in two tiers: a fast browserless tier built on server-side Apache ECharts ^[32] rasterised to PNG, and an opt-in high-detail tier that drives a headless browser and falls back to the fast tier on any failure, so a render never errors. The indicator library is a fork of a maintained package of over one hundred indicators ^[33], replacing four hand-rolled ones. Chart types include candles, Heikin-Ashi, line, area, and Renko, with an optional logarithmic price axis; the drawing primitives include horizontal and vertical lines, rectangles, trend lines, rays, channels, Fibonacci retracement and extension, and measured moves. Every render returns a facts block, the price, change, volume, high, and low, computed from the same candle series that is drawn, so the numbers an agent reads back cannot drift from the picture it shows.

4.4 tape-pulse

/api/tape-pulse · 0.01 USDT

`tape-pulse` reads the recent DEX trade tape for a token and reports its microstructure. The tape is first cleaned: routed multi-leg swaps that share one transaction hash are deduplicated so volume is not double-counted, and exchange-flagged rows are dropped. On the clean tape the service reports buy/sell imbalance, net dollar flow, tape VWAP against spot, whale prints, and trade-rate acceleration, and then computes three real microstructure estimators that a desk would recognise: Kyle's lambda, the price impact per unit of signed order flow ^[10]; Amihud illiquidity, the price move per dollar traded ^[11]; and VPIN, the order-flow toxicity measured as buy/sell imbalance across equal-volume buckets ^[12]. Section 5.5 explains why the first two are computed on volume blocks rather than tick by tick, a choice forced by ground-truthing. Trade direction comes from the exchange's own label, which is superior to a classifier such as Lee-Ready ^[14] because it introduces no classification error. Each estimator carries a quality gate and returns null on a tape too thin or too flat to support it; Kyle's lambda additionally reports its fit R-squared and a confidence label, so a weak read is visible rather than dressed up.

4.5 poly-fill, poly-desk, updown-pulse

Polymarket · 0.01 USDT each

poly-fill simulates a pre-trade fill on a Polymarket order book. It walks the live book for a requested notional and reports the volume-weighted executable price, the slippage against the midpoint, and the largest size that stays within a target slippage. The midpoint on a thin book is a fiction; the walk is the truth. Beyond the walk, the service fits the executable-cost curve to the square-root market-impact law ^{[13],[15]} and reports how closely this particular book follows it, so the fit quality is measured rather than assumed, together with the two-sided depth imbalance and a maker-versus-taker note that surfaces the adverse-selection a resting order carries. On a live Bitcoin-threshold market the fit R-squared is around 0.55, an honest number for a discrete prediction-market book.

poly-desk returns a Polymarket wallet's live book: open positions with current marks, unrealised PnL, cash-out value, and the biggest movers, for copy-trading and portfolio agents that track a specific trader.

updown-pulse reads the live short-window BTC or ETH up-or-down market and reports the market's own implied odds, the true time to resolution, a spot observation, and a driftless estimate of how far price could move before the market resolves, scaled from daily realised volatility. It deliberately outputs no fair value and no edge. Short-horizon directional prediction is empirically indistinguishable from a coin flip, so a predictive number here would be false precision that contradicts the product's own thesis. The service surfaces the market and the risk and leaves the directional call to the caller.

4.6 protocol-pulse, macro-sentry, loop-digest

0.005 - 0.01 USDT

protocol-pulse compiles a DeFi protocol's health from DefiLlama: TVL level and trend, ninety-day maximum drawdown, chain concentration, age, and the hack registry. It does not emit a single composite grade. A 0–100 health score would imply a

calibrated, out-of-sample-validated model, and with no labelled dataset of protocol failures to validate weights against, a hand-tuned composite would be goal-seeking. The service instead reports the factual signals and a set of individually defensible risk flags, each a concrete condition with its threshold and rationale disclosed: a hack in the last twelve months, a deep drawdown, a sustained outflow, a very new protocol, a micro-cap TVL, a single-chain dependency. The risk level is a transparent function of the flags present, not a scored model.

macro-sentry returns the high-impact US macro events, FOMC decisions and CPI, PCE, and payrolls releases, that fall inside a caller's lookahead window (the `hours` input), so a trading agent can de-risk before a print. The calendar is curated from published institutional schedules, which are fixed and durable, and the computation is dependency-free. Because the result is filtered against the call time (only events still ahead are returned), the answer is a signed, timestamped *observation* rather than a re-runnable proof — the same honest distinction the deterministic services draw in Section 5.

loop-digest is built for the top of an agent's loop. Given a cursor from the caller's previous call, it returns a compact diff of a wallet's world since then, new fills and per-token PnL drift, so the agent reads one small object instead of re-fetching and re-paying for the entire state on every iteration.

4.7 token-scan, wallet-audit

authenticity · 0.05 USDT each

token-scan answers a question that a price chart cannot: what share of a token's recent DEX volume is organic rather than manufactured. It aggregates the recent tape per wallet and scores the fingerprints of wash trading — round-trip churn, machine-regular timing cadence, buy/sell ping-pong alternation, and volume concentration in a handful of addresses — cross-referenced against OKX holder-cluster funding priors. Every verdict ships its evidence: the specific wallets and transaction hashes behind the score, and a confidence that is lowered rather than hidden when the tape is thin. The number an agent acts on can be audited back to the trades that produced it.

wallet-audit asks the same authenticity question of a track record. Native OKX endpoints report a wallet's PnL and win rate; this service grades whether those numbers reflect skill or are statistically hollow. A win rate over a handful of trades is not evidence, so the audit reports the Wilson score interval on the win rate at the real sample size, flags a record too short to support the claimed edge, and reuses token-scan on the wallet's own traded tokens to catch a PnL inflated by self-manufactured volume. It returns a grade with its working shown, or an explicit insufficient-data verdict, never a flattering point estimate on its own.

4.8 lp-desk

/api/lp-desk · 0.01 USDT

lp-desk is a concentrated-liquidity reality-check, and it is built to *refuse* a product the rest of the market sells. Every “range optimiser” promises an optimal width for a Uniswap-V3 position. The service reconstructs what a chosen range would actually have earned by replaying the pool's real on-chain swap history — fetched over a keyless public archive node, with pool metadata read from the chain rather than assumed — and nets the realised fees against the divergence loss (loss-versus-rebalancing) the position bleeds as price moves through it. The finding, measured on hundreds of thousands of real swaps across two eighteen-month-separated windows, is that no capturable optimal width exists: narrow ranges are destroyed by rebalance bleed rather than gas, the widely shipped hardcoded $\pm 5\%$ band was net-negative in both windows, and a volatility-fitted width — the entire premise of an optimiser — failed to beat a constant reliably out of sample. So lp-desk sells the measurement, not a false optimum, and says plainly when the honest answer is “don't provide this liquidity.” Its own biases are disclosed and both happen to favour narrow ranges, which still lose, so the conclusion is robust; the limitation that swap logs carry only aggregate active liquidity, making the tool exact for relative strategy ranking at small size but not absolute PnL at scale, is stated in the output rather than buried. Section 5.9 gives the position mathematics.

4.9 Risk Brain — perp-gate, size-gate, exec-verify, options-risk, portfolio-gate

/api/perp-gate · /api/size-gate · /api/exec-verify ·
/api/options-risk · /api/portfolio-gate · 0.01-0.05 USDT

The Risk Brain is the layer beneath the risk gate. An autonomous agent moving money on a loop does not fail for want of alpha; it fails on the arithmetic. In a public contest, six frontier models each traded ten thousand dollars of perpetuals autonomously for seventeen days. Four of the six finished in the red and the worst lost 62%; the other two finished ahead, one of them up 22%^[43]. That is one short contest with six participants and no control arm, so it establishes that unsupervised agents *can* lose most of their capital on perpetuals, not how often they do, and nothing about which specific failure caused it. An earlier draft of this sentence said four of the six lost more than 60%, which is false and flattered the argument: only one did. The measured claim underneath does not depend on a contest, and it is narrower than an earlier draft of this document made it. On a benchmark of numerical reasoning over real S&P 500 filing tables, the strongest models tested reach 91.9–95.6% overall

but still fail 10–20% of the multi-step calculations, which the authors call a critical barrier for accuracy-sensitive financial work; the smaller open-weight models fail essentially all of them^[42]. This document previously described that as top models collapsing “from 95.6% accuracy on lookups to near zero”, which misread the source in three ways: 95.6% is one model's *overall* score rather than its lookup score (its lookup score is higher still, 97.0%), the near-zero results belong to the small open-weight models rather than to the leaders, and the tier in question holds ten items. The corrected version is weaker and is the one the source supports. It is also enough for the argument here, because a one-in-five failure rate on multi-step arithmetic is disqualifying for an agent sizing a position, whether or not the failure rate is total. The agent risk products now shipping from exchanges and startups enforce user-set *limits* (max size, leverage caps, drawdown stops) but assume the correct risk numbers as input; they do not compute them. These services compute them, deterministically, and prove each answer.

perp-gate returns a perpetual-futures position's exact liquidation price, the adverse move to liquidation, effective leverage, and — given a funding rate — the funding drag. The liquidation price is not copied from a venue formula; it is *derived* from Hyperliquid's stated liquidation condition (account value below maintenance margin) and then *verified* numerically against that condition on every call, so a wrong derivation fails loudly rather than shipping a confident wrong number. Section 5.10 gives the derivation. **size-gate** converts an edge — discrete odds or a return/volatility pair — into a fractional-Kelly position and the probability of ever drawing down to 50/75/90% of capital; it defaults to quarter-Kelly because full Kelly rides thin, over-estimated edges to ruin, and it self-checks the returned fraction against the first-order condition that *defines* Kelly optimality. **exec-verify** takes a completed swap and the pre-trade pool state and reports how many basis points the fill lost to *adverse* execution — sandwiching, MEV, or a stale quote — beyond the unavoidable fee and own price impact, proving the uncomfortable fact that a fill “within slippage tolerance” can still have been robbed; its benchmark is the constant-product invariant, asserted on every call that supplies pool reserves. A fourth, **options-risk**, aggregates a whole options book: net greeks (delta, gamma, vega, theta, and the second-order vanna and volga) and a SPAN-style scenario margin — the worst repriced P&L over a price×vol grid — because an options book's true risk is not the sum of per-leg notionals. Its self-check is the strongest in the catalogue: all six analytic portfolio greeks are verified against finite-difference derivatives of the actually-repriced book, a discrepancy that surfaced (and forced the upgrade to a machine-precision normal CDF, since the greeks must be consistent with the prices to the last figure). A fifth, **portfolio-gate**, is the whole-book view the others lack: it aggregates perpetual legs *across venues* — Hyperliquid and dYdX — into the true net exposure per underlying (signed notionals summed, so a long on one venue and a short of the same asset on another net out instead of double-counting), the leg that liquidates *first* across the book, a concentration index that exposes when a nominally diversified book is secretly one directional bet, and a correlated-crash stress that moves every leg together as correlations go to one — the regime in which an October 2025 day on which industry trackers put single-day liquidations between roughly \$9 billion and \$19 billion, depending on the venue coverage each one reaches. Section 5.18 derives the aggregation and the four identities it self-checks. Each of these five ships the proof envelope of Section 3: echoed inputs, code identity, a content hash, and the self-check result, so the caller reproduces the number rather than trusting it.

5. Methodology

This section gives the mathematics behind the options, microstructure, and safety services. It defines notation once and uses it consistently. Deribit crypto options are quoted on the forward, so the pricing model throughout is Black-76^[1] rather than spot Black-Scholes^{[2],[3]}.

5.1 Black-76 and the option greeks

Let F be the forward, κ the strike, τ the time to expiry in years, σ the implied volatility, and r the continuously compounded discount rate. Write ϕ and N for the standard normal density and distribution. The Black-76 call price and the moneyness terms are

$$C = e^{-rT} [F \cdot N(d_1) - K \cdot N(d_2)], \quad d_1 = [\ln(F/K) + \frac{1}{2}\sigma^2\tau] / (\sigma\sqrt{\tau}), \quad d_2 = d_1 - \sigma\sqrt{\tau} \quad (1)$$

The first-order greeks follow by differentiation. Delta is $e^{-rT}N(d_1)$ for a call, gamma is $e^{-rT}\phi(d_1)/(F\sigma\sqrt{\tau})$, and vega, the sensitivity to volatility, is

$$\mathcal{V} = e^{-rT} F \phi(d_1) \sqrt{\tau} \quad (2)$$

Dealer hedging is driven not only by these but by the second-order volatility greeks, so the service computes them in closed form and, because a closed form is only as good as its verification, checks each against a finite difference of the first-order greek. Vanna, the cross-derivative of price in forward and volatility, and volga, the convexity of value in volatility, are

$$vanna = \partial\mathcal{V}/\partial F = -e^{-rT} \phi(d_1) d_2 / \sigma, \quad volga = \partial\mathcal{V}/\partial\sigma = \mathcal{V} \cdot d_1 d_2 / \sigma. \quad (3)$$

Both are identical for calls and puts, as they must be, since they are curvature terms in strike and volatility rather than direction. The test suite asserts this identity and asserts that the analytic vanna equals the numerical derivative of vega with respect to the forward to a tolerance of one part in a thousand.

5.2 The arbitrage-free volatility surface

A real options chain does not have one volatility; it has a smile, a function $\sigma(k)$ that must be estimated from a finite set of quoted strikes. Interpolating linearly in log-strike is easy and wrong: it admits butterfly arbitrage and can produce a negative risk-neutral density in the wings, which makes any probability read from it meaningless. The service instead fits the raw SVI parameterisation of Gatheral ^{[5],[7]} to each expiry's total variance $w(k) = \sigma^2(k)\tau$ as a function of log-moneyness $k = \ln(K/F)$,

$$w(k) = a + b \{ \rho(k - m) + \sqrt{[(k - m)^2 + s^2]} \}. \quad (4)$$

The five parameters are fitted by Nelder–Mead minimisation ^[30] of squared volatility error, but with the no-arbitrage conditions of Gatheral and Jacquier ^[6] enforced as penalties inside the objective, so a converged fit is arbitrage-free by construction rather than by inspection. The butterfly condition is the non-negativity of the Durrleman function,

$$g(k) = (1 - k \cdot w'(k) / 2w(k))^2 - (w'(k)^2 / 4)(1/w(k) + 1/4) + w''(k)/2 \geq 0, \quad (5)$$

checked on a grid of log-moneyness, and the large-strike wing is bounded by the Roger Lee moment condition $b(1+|\rho|) < 2$ ^[8], which is equivalent to the density decaying at infinity. A slice whose fit exceeds a root-mean-square gate of 1.5 volatility points, or whose no-arbitrage checks fail, is rejected, and the caller is told the disclosed fallback was used rather than being served a bad surface silently.

One consequence of taking the model seriously is knowing where it does not apply. Near-expiry smiles are dominated by jumps, not diffusion, and a diffusive parameterisation such as SVI cannot describe them; the residual root-mean-square rises from under a volatility point at two or more days to several volatility points inside a day. The service excludes those slices by design and says so, rather than force-fitting a model outside its regime.

To price an event whose date is not a listed expiry, the surface is interpolated in the maturity direction. The correct interpolation is linear in total variance at fixed moneyness, because the calendar no-arbitrage condition is exactly that total variance be non-decreasing in maturity ^[6]; interpolating volatility, or snapping to the nearest expiry, can violate it. For a target maturity τ bracketed by fitted slices $\tau_a < \tau < \tau_b$,

$$w(k, T) = w_a(k) + (w_b(k) - w_a(k)) \cdot (T - T_a) / (T_b - T_a), \quad \sigma(k, T) = \sqrt{w(k, T) / T}. \quad (6)$$

Outside the fitted range the service holds volatility flat across the gap and discloses the extrapolation per row.

5.3 Risk-neutral density and the full distribution

The risk-neutral distribution of the terminal price is implicit in the option prices. Breeden and Litzenberger ^[4] showed that the risk-neutral cumulative distribution and density are the first and second derivatives of the call price in strike,

$$P(S_T > K) = -e^{rT} \partial C / \partial K, \quad f(K) = e^{rT} \partial^2 C / \partial K^2. \quad (7)$$

With a strike-dependent volatility the strike derivative of the call carries a term in the smile slope. Differentiating (1) with $\sigma = \sigma(k)$ and substituting into (7) gives the exact probability,

$$P(S_T > K) = N(d_2) - e^{rT} \cdot \nu \cdot \partial \sigma / \partial K. \quad (8)$$

The plain $N(d_2)$ is the first term alone. On a flat smile the second term vanishes and the two agree; on a real skewed crypto smile the second term is material, and it changes sign where the smile slope changes sign. We measured the omission on a constructed seven-day BTC smile with realistic skew (a constructed surface, not a venue capture, so that the bias is measured against a known answer): $N(d_2)$ overstates the probability by 3.3 points at one strike and 5.0 at another, and understates it in the opposite wing. Reporting $N(d_2)$ to one-tenth of a point while carrying a five-point error is not a simplification; it is a defect, and the service uses (8).

Integrating the density (7) over a strike grid gives the full distribution. The service reports the quantiles from the 5th to the 95th percentile by inverting the resulting cumulative distribution, the tail expected shortfalls $E[S \mid S \leq q_5]$ and $E[S \mid S \geq q_{95}]$ ^[19], and the distribution's volatility, skew, and excess kurtosis. Correctness is checked by an identity that requires no external data. Under the risk-neutral forward measure the terminal price is a martingale, so its expectation equals the forward:

$$E[S_T] = \int K f(K) dK = F. \quad (9)$$

The service computes the residual $E[S_T]/F - 1$ and reports it as a self-check. On a live BTC expiry it reads 0.001%, and the raw integral of the density reads 1.000, which together certify that the numerical density is a proper density and that the integration is correct. The distribution is withheld when these checks fail.

5.4 Variance risk premium and honest significance

Implied variance tends to exceed subsequently realised variance; the difference is the variance risk premium, the compensation option sellers earn for bearing volatility risk ^{[16],[17],[18]}. The service estimates it as the median ratio of realised to implied volatility over a rolling horizon. Realised volatility over a window is the annualised standard deviation of daily log returns; implied volatility is the DVOL index. The subtlety is significance. Overlapping windows are highly correlated, so the number of windows badly overstates the number of independent observations. Using the raw count would manufacture significance. The service deflates to an effective sample,

$$n_{\text{eff}} = (\text{number of overlapping pairs}) / \text{horizon}, \quad (10)$$

and tests whether realised is below implied more often than chance with a one-sided binomial test at n_{eff} . On live BTC the median ratio is 0.881, which looks like a premium, but at the honest effective sample the p-value is 0.254. Deflating the sample to n_{eff} is a deliberately blunt correction. The standard treatment for overlapping windows is a HAC or Newey–West variance with a bandwidth set by the overlap, and it would be the right thing to run here; a non-parametric block bootstrap over the ratio series would be better still, since the statistic is a median rather than a mean. Neither is implemented. What can be said for the crude version is the direction of its error, and an earlier draft of this passage got that direction backwards. Collapsing correlated observations into a smaller independent count costs power, which *raises* the p-value: 0.254 is therefore a *ceiling*, not a floor, and a properly HAC-corrected test — whose effective sample sits between n_{eff} and the full count — would return something smaller and could well reject at the same level. So nothing follows about what the sharper test would say; the earlier claim that a premium failing this test would also fail a HAC-corrected one is simply invalid, and is withdrawn. What the crude test does support is the refusal itself: we decline to sell a premium we have not established, and a conservative test is the right instrument for that decision even though it is the wrong instrument for the opposite one. The service reports that the premium is not statistically distinguishable from zero and marks every real-world-calibrated probability it derives as indicative only. Computing a plausible number and then refusing to over-claim it is the behaviour a quantitative reviewer looks for, and it is enforced here in code.

5.5 Tape microstructure estimators

Three estimators summarise the order flow in a DEX tape. Let a block be a set of consecutive trades; for block j let r_j be the log return across the block, v_j the dollar volume, and q_j the signed dollar flow with buys positive and sells negative. Kyle's lambda ^[10], the permanent price impact per unit of signed flow, is the slope of the regression of block return on signed flow,

$$r_j = \lambda \cdot q_j + \varepsilon_j \quad (11)$$

reported with its R-squared and sign-agreement so a weak relationship is visible. Amihud illiquidity ^[11] is the average price move per dollar traded,

$$ILLIQ = \text{mean}_j (|r_j| / V_j), \text{ reported as } \text{pctMovePer1kUsd} = ILLIQ \times 10^3 \times 10^2. \quad (12)$$

The reporting scale is written into the equation because without it the figures below cannot be reproduced from it, and an earlier draft omitted it. In its raw form $ILLIQ$ is a fractional return per dollar, so a value of 0.303 in those units would mean a thirty-percent move per dollar traded, which is nonsense; every Amihud figure in this document is in the field's own units of *percentage move per thousand dollars of volume*, which is what the response names it. Both figures quoted below are on that same scale, so the comparison between them is meaningful.

VPIN ^[12] is the mean absolute buy-minus-sell imbalance across equal-volume buckets, a number in $[0,1]$ where zero is perfectly two-sided flow and one is fully one-sided.

The choice of block rather than tick is not cosmetic; it is a correction that ground-truthing forced. A per-trade Amihud ratio divides by the dollar size of a single trade, and a DEX tape contains sub-cent dust trades whose near-zero denominator sends the ratio to enormous values. On a live memecoin tape the per-trade estimator returned an Amihud figure of order 1.6×10^9 , obvious nonsense. Two facts caused it: the dust denominators, and the observation that the exchange returns a sampled large-trade feed for a hot token, so two consecutive trades can be hours apart and their return reflects drift rather than impact. Aggregating into equal-count blocks removes both problems, because a block aggregates real volume and a real time interval, and it is the faithful period form of both Amihud and Kyle. On the same token the block estimator returns an Amihud of 0.303,

a sane price move per thousand dollars. The tape-density diagnostic that surfaces the sampled feed is reported alongside, so the caller can see when the impact read rests on thin ground.

5.6 Dealer gamma exposure and barrier probabilities

Two further constructions support the options service. Gamma exposure summarises how much delta dealers must hedge as spot moves. The dollar gamma of a strike is its Black-76 gamma scaled by open interest and by the square of spot, and the net across the chain applies the dealer-sign convention of Section 4.1,

$$GEX(S) = \sum_i \varepsilon_i \cdot \Gamma_i(S) \cdot OI_i \cdot S^2 \cdot 0.01, \quad \varepsilon_i = +1 \text{ (call)}, -1 \text{ (put)}. \quad (13)$$

Net positive gamma means dealer hedging dampens moves toward the largest strikes; net negative gamma means hedging amplifies them. The gamma-flip is the spot level at which $GEX(S)$ crosses zero, found by scanning a spot band and interpolating the crossing. The convention is an assumption, disclosed as such in Section 8; the construction itself is exact given the convention.

The cross-market service prices reach-by-date questions with the one-touch barrier probability. Under the driftless forward measure, the reflection principle gives the probability that the forward touches a barrier at any time before expiry,

$$P(\text{touch } H > F) = N(d_2) + (F/H) \cdot N(d_1), \quad P(\text{touch } L < F) = N(-d_2) + (F/L) \cdot N(-d_1), \quad (14)$$

with $d_1 = [\ln(F/B) + \frac{1}{2}\sigma^2 T]/(\sigma\sqrt{T})$ and $d_2 = d_1 - \sigma\sqrt{T}$ for the barrier B . A touch probability is always at least the finish-above probability, since a path that ends beyond the barrier must have crossed it; the test suite asserts this inequality. Because a one-touch is path-dependent, no single volatility is smile-consistent, so the service reports the model-uncertainty span between the barrier and at-the-money volatilities rather than implying false precision.

5.7 Transaction and signature safety

The transaction path decodes calldata and, given the full unsigned transaction, executes it against live state with `eth_simulateV1`, extracting asset and approval changes from the emitted Transfer and Approval events. Upgradeability is checked by reading the proxy implementation slot: the EIP-1967 slot ^[22], its beacon variant, and the pre-standard OpenZeppelin slot used by early proxies. A non-zero implementation flags a contract whose logic its admin can replace after approval.

The signature path parses an EIP-712 typed-data payload ^[20]. EIP-712 binds a signature to a domain and a typed struct through a hash of the form `keccak256("\x19\x01" || domainSeparator || hashStruct(message))`; a signature that a wallet displays as harmless can authorise a transfer through a Permit or Permit2 grant. The service recognises the Permit2 ^[25] and EIP-2612 ^[21] permit shapes, extracts the spender, amount, and expiry, checks the domain against the canonical Permit2 deployment, and flags an unlimited allowance, a non-expiring permit, a batch that touches many tokens, or a spender that resolves to an externally owned or EIP-7702-delegated wallet ^[23] rather than an audited contract. Because these grants are signatures and never transactions, no amount of transaction simulation can see them; a service that stops at simulation leaves the dominant drainer vector uncovered.

5.8 Model-free calendar bounds by martingale optimal transport

A single expiry's smile pins down the risk-neutral law of the terminal price at that one date; it says nothing about the joint law across two dates, which is exactly what a forward-starting or calendar payoff needs. The classical response is to assume a dynamics — Heston, local volatility, a jump model — and inherit its misspecification. Martingale optimal transport instead asks for the entire range of prices consistent with the two marginals and no arbitrage, and returns a hard interval rather than a point (Beiglböck, Henry-Labordère and Penkner^[39]). Work in the martingale coordinate $x = S/F(T)$, in which each fitted marginal has mean one. Let μ be the earlier date's law and ν the later's, both recovered from the fitted smiles by Breeden–Litzenberger and discretised to grids $\{x_i\}$ and $\{y_j\}$ with probabilities $\{\mu_i\}$, $\{\nu_j\}$. A joint law is a coupling $\pi_{ij} \geq 0$, and the no-arbitrage price of a payoff c is bounded by the linear program

$$[lower, upper] = \min / \max_{\pi} \sum_{ij} \pi_{ij} c(x_i, y_j) \quad s.t. \quad \sum_j \pi_{ij} = \mu_i, \quad \sum_i \pi_{ij} = \nu_j, \quad \sum_{ij} \pi_{ij} (y_j - x_i) = 0. \quad (15)$$

The first two constraints reproduce the quoted marginals; the third is the martingale condition $E[Y \mid X = x_i] = x_i$ that no-arbitrage imposes between the two dates. The program is feasible if and only if μ and ν lie in convex order, written $\mu \leq \nu$, which by Strassen's theorem is precisely the existence of a martingale coupling — equivalently, that the two smiles carry no calendar arbitrage. The same linear program that prices the payoff therefore certifies the surface: infeasibility is a calendar-arbitrage detector, agreeing with the total-variance monotonicity check of Section 5.2 by a completely independent route. Two identities gate the result before it ships. The later marginal must itself have unit mean, $\int y \, d\nu = 1$, and the transport must preserve it, so

$E_{\pi}[Y - X] = 0$ to numerical tolerance. When either fails — the signature of a mis-discretised or crossed marginal — the bounds are withheld rather than served, in keeping with the refuse-to-fabricate principle.

5.9 Concentrated-liquidity value and divergence loss

A Uniswap-V3 position supplies liquidity L across a price range $[P_a, P_b]$ and earns fees only while the pool price sits inside it. At a price P within the range the position holds

$$amount0 = L (1/\sqrt{P} - 1/\sqrt{P_b}), \quad amount1 = L (\sqrt{P} - \sqrt{P_a}), \quad (16)$$

token0 vanishing at the top of the range and token1 at the bottom. lp-desk fixes L from the caller's capital at entry, then replays the pool's real swap sequence read from chain: at each in-range swap it credits the position's pro-rata fee share $L / (L_{active} + L)$ of the tier applied to that swap's volume, and re-marks the holdings by (16) at the new price. The profit-and-loss against simply holding the entry bundle decomposes as

$$(LP - HODL) = \text{fees earned} - \text{divergence loss}, \quad (17)$$

where the divergence loss is the concentrated-liquidity form of loss-versus-rebalancing — the value the position sheds to arbitrageurs as price crosses it, growing as the range narrows. The replay is exact for *ranking* strategies at small size, where the pro-rata share is faithful; it is a relative measure, not absolute PnL at scale, because public swap logs expose only aggregate active liquidity and not the per-tick order book, and the service discloses this limit in its own output. The empirical result across the replayed windows is that fees never overtake divergence loss for the narrow ranges an optimiser would recommend, which is why lp-desk reports the measurement and declines to name an optimum it cannot honestly defend.

5.10 Perpetual liquidation and funding

The perp-gate liquidation price is not copied from a venue's formula — it is *derived* from the stated liquidation condition and then verified against it on every call. A venue liquidates an isolated position when its account value falls to the maintenance margin. With posted margin M , size q in base units, entry P_0 , side $s \in \{+1, -1\}$, and a maintenance rate mmr (half the initial rate at maximum leverage, so $mmr = 0.5 / \text{maxLeverage}$), the account value at price P is $M + s \cdot q \cdot (P - P_0)$ and the maintenance requirement is $q \cdot P \cdot mmr$. Equating the two and solving gives

$$P_{liq} = (s \cdot q \cdot P_0 - M) / (q (s - mmr)), \quad (18)$$

which for a long is $(P_0 - M/q) / (1 - mmr)$. On every call the engine substitutes P_{liq} back and confirms that account value equals the maintenance requirement there — the same condition it solved — so a wrong derivation fails loudly (residual of order 10^{-11} to 10^{-12} against a tolerance scaled to notional; the worked example in Appendix C shows 2.05×10^{-12} against a tolerance of 0.064) rather than shipping a confident wrong number. This is the discipline in action: deriving from the primary source, rather than a widely-repeated secondary formula that dropped the maintenance term and reported roughly double the true buffer. Funding is projected separately on the notional at the venue's cadence (hourly on Hyperliquid). The maintenance rate is not a single constant: on Hyperliquid it steps up with position notional through the venue's margin tiers, so a large position is measured against the tighter rate its size actually incurs rather than the headline one, and the identical derivation is driven by a second venue's live parameters when a leg sits on dYdX, whose maintenance-margin fraction is published directly. Figure 2 shows how the liquidation buffer collapses with leverage.

The liquidation buffer collapses with leverage

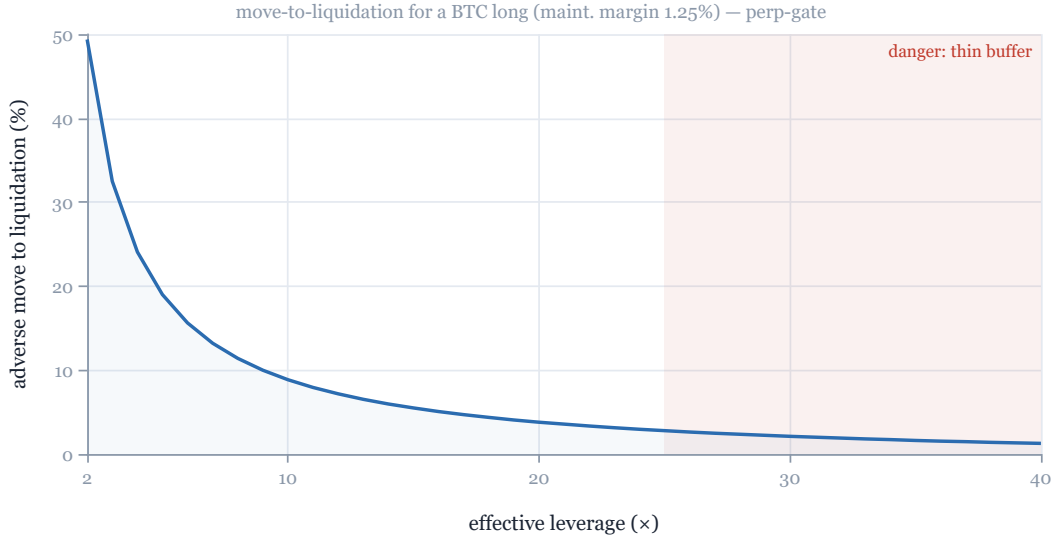


Figure 2 — move-to-liquidation for a BTC long at maintenance margin 1.25%, computed by perp-gate across leverage.

5.11 Position sizing and the probability of ruin

Over-betting, not a bad thesis, is what ruins a leveraged agent, so size-gate converts an edge into a fraction of capital and reports the probability that fraction ever ruins it. Full Kelly maximises expected log-growth: for a discrete edge (win probability p , net odds b) it is $f^* = (p(b+1)-1)/b$, the argmax of $g(f) = p \cdot \ln(1+fb) + (1-p) \ln(1-f)$; for a continuous edge, $f^* = \mu/\sigma^2$. Full Kelly is growth-optimal but its drawdowns are violent, so the engine bets a fraction λ of full Kelly (default a quarter). The probability of *ever* drawing down to a fraction α of capital is derived, not recalled: under fractional-Kelly λ log-wealth is a Brownian motion with drift $g = (\mu^2/\sigma^2)\lambda(1-\lambda/2)$ and variance rate $s^2 = \lambda^2\mu^2/\sigma^2$; the probability that a drifting Brownian motion ever reaches a level $-L$ is $\exp(-2gL/s^2)$, and with $L = \ln(1/\alpha)$ this collapses to

$$RoR(\alpha, \lambda) = \alpha^{(2-\lambda)/\lambda}, \quad (19)$$

which at full Kelly ($\lambda=1$) reduces to the classic result $RoR=\alpha$. The engine self-checks the returned fraction against the first-order condition $g'(f^*)=0$ that *defines* Kelly, the half-Kelly identity (growth $\times 0.75$, volatility $\times 0.5$), and the ruin anchor, and it flags when the recommended bet exceeds the bankroll (Kelly implying leverage, where the ruin figure understates the true risk of a forced liquidation). Figure 3 shows why professionals under-bet: ruin risk falls super-linearly as the Kelly fraction shrinks.

Risk of ruin shrinks super-linearly with fractional Kelly

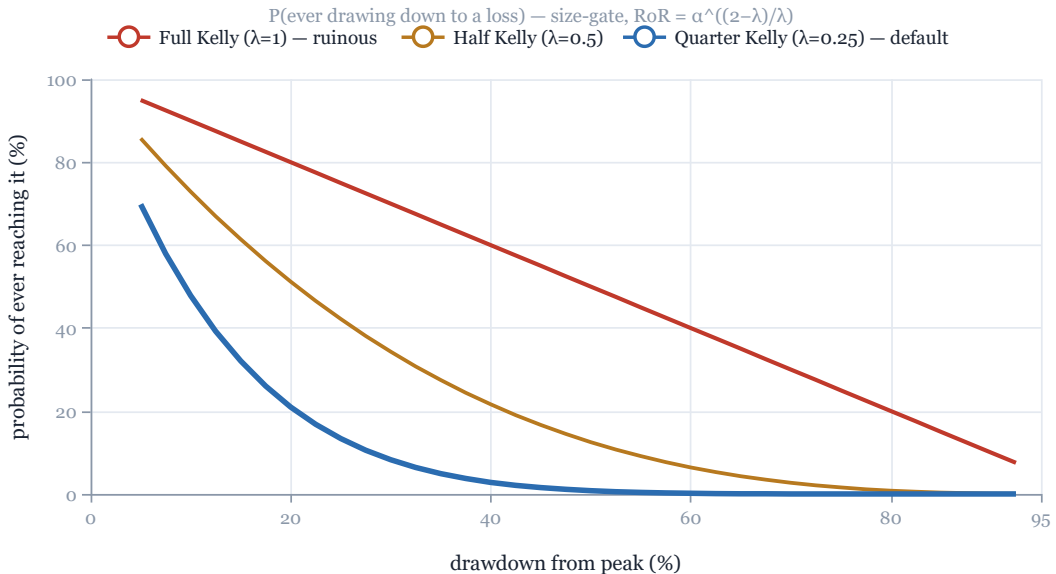


Figure 3 — probability of ever reaching a given drawdown at full, half and quarter Kelly, from size-gate. Full Kelly is linear in the drawdown; quarter Kelly is α^7 .

5.12 Execution quality and adverse fills

“Filled within slippage tolerance” is not protection — a sandwich completes the trade inside the tolerance while extracting value the caller never sees. For a constant-product pool with pre-trade reserves (x, y) and fee f , the honest output for input Δx is

$$out = y \cdot \Delta x (1-f) / (x + \Delta x (1-f)), \quad (20)$$

which exactly preserves the fee-adjusted product, $(x + \Delta x (1-f))(y - out) = xy$ — the invariant `exec-verify` asserts whenever it is given pool reserves — in the alternative mode the benchmark is a price the caller supplies, so there is no pool model to assert against and the response now reports that check as not-run rather than as a pass. The realised fill's shortfall against this honest figure is reported as adverse-execution basis points, separated from the unavoidable fee and the caller's own price impact; a favourable fill is labelled as such rather than mislabelled adverse. The engine discloses that its benchmark is only as good as the reserve snapshot's timing (block-start versus immediately-pre-transaction), because that choice determines whether a front-run is inside or outside the measured baseline.

5.13 Options-book greeks and scenario margin

An options book's risk is not the sum of its legs' notionals. `options-risk` aggregates the position-weighted greeks ($\sum q_i \cdot greek_i$) on Black-76 and computes a SPAN-style scenario margin — the worst repriced profit-and-loss over a grid of price and volatility shocks, which is the loss the book must be held against. Its self-check is the most exacting in the catalogue: the analytic portfolio greeks are verified against finite-difference derivatives of the *actually repriced* book, so $dV/d(shock)$ must equal $\sum q_i \cdot \delta_i \cdot F_i$ and the second difference $\sum q_i \cdot \gamma_i \cdot F_i^2$. Enforcing that to a tight tolerance surfaced a latent flaw: the pricing used an Abramowitz–Stegun^[31] normal CDF whose numerical derivative disagreed with the exact-density greeks at $\sim 5 \times 10^{-4}$, flooring any greek self-check. Rather than widen the tolerance to make the red turn green, we replaced the CDF with a machine-precision (Hart) algorithm, restoring price–greek consistency to $\sim 4 \times 10^{-8}$ and improving every options service that shares the pricer. Figure 4 shows the scenario surface of a short straddle.

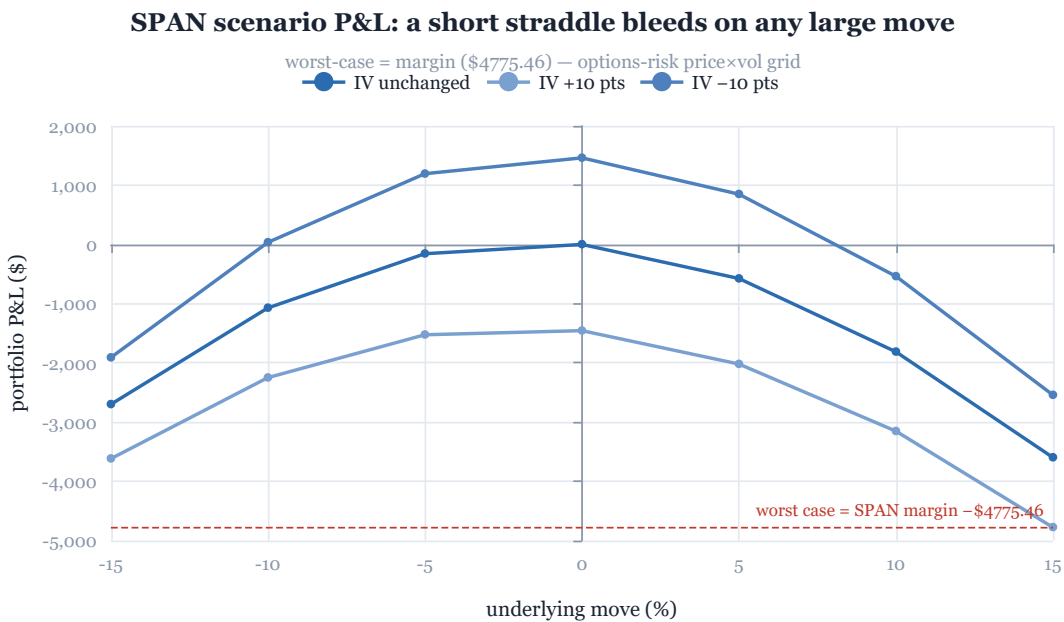


Figure 4 — SPAN-style scenario P&L of a short BTC straddle across the price×volatility grid, from options-risk; the worst cell is the scenario margin.

5.14 Liquidity-provision divergence and its LVR rate

Where `lp-desk` replays history, `lp-risk` forecasts. For a full-range position the impermanent loss at a realised price ratio r is

$$IL(r) = 2\sqrt{r} / (1+r) - 1, \quad (21)$$

verified at the token level against explicit constant-product amounts (the position holds $2\sqrt{kP_1}$ in value against the held bundle). Expanding to leading order, $IL \approx -(1/r)^2/8$, so the expected divergence over a horizon under a lognormal underlying is $E[IL] \approx -\sigma^2 T/8$ — which numerically matches the loss-versus-rebalancing rate of a constant-product pool^[44]. That coincidence has been a source of confusion in this document and the confusion is worth clearing up, because a reviewer was right to press

on it. **Impermanent loss and loss-versus-rebalancing are not the same quantity.** IL is a function of the *terminal* price ratio alone: path-independent, and bounded in $(-100\%, 0]$. LVR is the running loss to arbitrageurs against a rebalancing portfolio: it accrues monotonically along the path, never returns anything, and is therefore *unbounded*. On a path that ends where it began, IL is exactly zero while realised LVR is strictly positive. They agree only in expectation, only to leading order, only under a driftless lognormal — which is precisely the regime in which this expansion was derived, and precisely why the two names get used for one number. An earlier version of this passage then rejected the expansion because “at large total variance the rate returns a loss that cannot happen”. That reasoning was wrong in an instructive way: an unbounded loss is exactly what LVR is, so unboundedness is not evidence that the *rate* is broken — it is evidence that the rate is measuring the other quantity. The boundedness at -100% belongs to impermanent loss, and impermanent loss is what this service reports. The headline is therefore the exact expectation of (21) under $\ln r \sim N(-\sigma^2 T/2, \sigma^2 T)$, evaluated numerically, with the expansion beside it and the gap between the two reported so a caller can see where the approximation left its range. A boundedness check runs on the value actually served. This was a defect found in live testing, not a design choice made in advance; Section 11.5 gives the number it used to return. Given a fee yield the engine returns the net forecast and the break-even volatility above which fees no longer cover the bleed — also solved against the exact expectation, so re-running at that volatility nets zero, which the engine asserts as a self-check. Figure 5 shows the loss is symmetric in log price ratio and always non-positive.

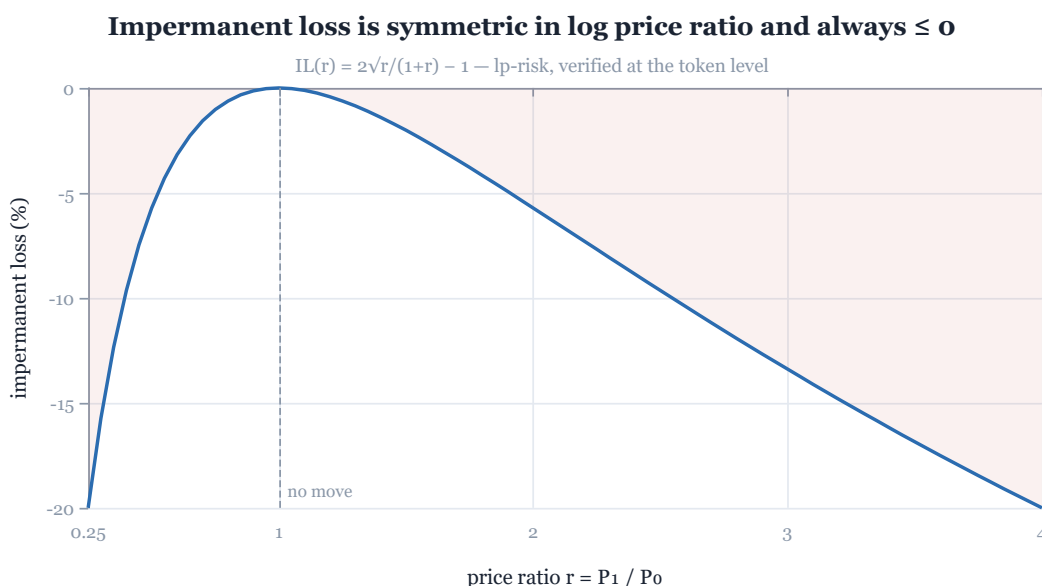


Figure 5 — impermanent loss versus price ratio, from *lp-risk*; zero at no move, symmetric under $r \rightarrow 1/r$.

5.15 Stablecoin-treasury concentration and depeg

A treasury's real risk is concentration and depeg, not its headline yield. *treasury-risk* measures concentration as the Herfindahl index over USD shares, $HHI = \sum w_i^2$, whose reciprocal is the effective number of independent exposures; any single share above a limit is flagged as a breach. Depeg loss for an asset marked at price p against peg q is $\sum \text{amount}_i (q-p)/q$, exact and independently re-checked on the worst-single-depeg scenario, and the risk-adjusted yield subtracts the annualised expected depeg loss wherever probabilities are supplied. Because depegs are not independent — the March 2023 weekend pulled USDC and the DAI it backs down together — the service also stresses a *correlated* depeg in which every stable falls to a common floor at once, the systemic loss a worst-single scenario hides, and reports a correlation-adjusted effective-exposure count $1/(w^T \rho w)$ that drops below the independent Herfindahl figure once a correlation matrix is supplied — identity by default, so no ρ is ever fabricated. The same Herfindahl lens deepens *poly-desk*, turning a flat list of prediction-market positions into a book with a concentration grade and a directional skew.

5.16 Event-implied expected move

A macro calendar gives a date and a “high impact” label; it does not give the market's priced-in magnitude. *event-vol* supplies it. For spot S , at-the-money implied vol σ and horizon τ , the one-standard-deviation move is $\sigma\sqrt{\tau}$, and — since for a driftless forward the ATM straddle price is the risk-neutral expected *absolute* move — the straddle equals $E|S_T - S_0| = 2S(2\Phi(\sigma\sqrt{\tau}/2) - 1)$, which the engine verifies against a numerical integral of $|S_T - S_0|$ over the martingale lognormal. Given the volatility term structure across the event — an expiry just before and just after — the event's own contribution is isolated as

$$\text{event } 1\sigma = S \cdot \sqrt{(\sigma_{\text{after}}^2 \cdot T_{\text{after}} - \sigma_{\text{before}}^2 \cdot T_{\text{before}})} \quad (22)$$

the event-day options technique: the post-event expiry carries the baseline diffusion plus the event, the pre-event expiry carries the baseline alone, and the difference in total variance is the jump the market is pricing. A non-positive event variance means the term structure does not isolate the event, and is disclosed rather than reported as a fabricated move. This is `macro-sentry`'s new depth: with a coin's spot and ATM vol it reports not just that an FOMC is ahead but that the market is pricing, say, a ten-percent one-sigma move into it.

5.17 Batched attestation

To make many computations cheaply and permanently attestable, `risk-attest` combines their content hashes into a single Merkle root (sha256 over packed 32-byte words, sorted pairs, with leaves tagged `0x00` and internal nodes `0x01` so a node cannot be presented as a member). One on-chain anchor of that root — broadcast from the caller's own wallet, never `Quiver`'s — attests every member of the batch at once, and any single computation is later provable with a small inclusion proof. The engine self-checks both directions of a Merkle proof: completeness (every leaf verifies against the root) and soundness twice over — a fabricated non-member must not verify, and neither must a real internal node presented as a member leaf. The second check exists because the first one cannot fail structurally, and a reviewer broke the earlier tree through exactly that gap (Section 11.5). The tree is deliberately *not* `OpenZeppelin MerkleProof`-compatible, since that library folds `keccak256` without a domain tag; the response says so and ships a five-line Solidity verifier for this tree instead of leaving the caller to infer the encoding.

5.18 Cross-venue portfolio exposure and correlated-crash stress

The services above price one position at a time; an agent's real risk lives in the whole book, and the book is often spread across venues. `portfolio-gate` aggregates a set of perpetual legs — on `Hyperliquid`, `dYdX`, or both — into four book-level numbers, reusing the `perp-gate` derivation of Section 5.10 for each leg so the aggregate inherits per-leg correctness. Write leg i with side $s_i \in \{+1, -1\}$, size q_i , and mark P_i , so its signed notional is $s_i q_i P_i$ and its gross notional $q_i P_i$. The *net* exposure to an underlying a sums the signed notionals of that underlying's legs,

$$E_a = \sum_{i \in a} s_i q_i P_i \quad (23)$$

so a long on one venue and a short of the same size on another net to zero real exposure rather than counting as two positions — a distinction a per-instrument tool cannot draw. Concentration is the Herfindahl index over gross notional by underlying, $H = \sum_a (G_a/G)^2$ with G_a the gross in underlying a and G the book total; its reciprocal $1/H$ is the effective number of independent bets, which collapses toward one when a nominally diversified book is really a single directional wager. The binding constraint is the *nearest* liquidation — the smallest adverse move to liquidation over all legs, each move computed by the Section 5.10 condition,

$$m_i = s_i(P_i - P_{liq,i}) / P_i \quad m^* = \min \{m_i : m_i \geq 0\}, \quad (24)$$

with s_i the leg's sign, so that m_i is the *adverse* move still available before leg i liquidates: m^* is the whole book's true distance to first blood. The restriction to non-negative m_i is not cosmetic. A leg whose mark has already passed its liquidation price has $m_i < 0$, and an earlier version took a minimum over absolute distances, which ranked such a leg as the book's nearest *future* liquidation when the event was in fact already past. Those legs are now reported separately, with an explicit below-maintenance status, as something to reconcile rather than to protect; Section 11.5 records how that was found. The catastrophic mode, though, is not one leg but many at once: in a crash, correlations go to one and independently-sized bets liquidate together — October 10, 2025 saw single-day liquidations reported between roughly \$9 billion and \$19 billion across trackers as exactly this unfolded. The correlated-crash stress moves the entire market by a uniform $\pm x\%$ and counts the legs crossing maintenance *simultaneously*: a down move with $x \geq m_i$ liquidates long leg i , an up move the shorts, and the fraction of the book breaching at each shock is the honest measure of how concentrated the tail truly is. The stress assumes co-movement, so a leg on a genuinely uncorrelated asset would not breach in reality; that assumption is disclosed with the result rather than buried.

Four exact identities guard the aggregation on every call: the per-asset net exposures sum to the reported book net, so a mis-bucketed leg fails loudly; every per-leg liquidation still satisfies its own maintenance invariant from Section 5.10; the reported nearest liquidation is genuinely the minimum distance across the legs that are still live; and the count of legs breaching under the correlated down-crash is monotone non-decreasing in the shock size. Isolated per-leg margin is assumed by default — a true cross-margin account shares equity across legs, and that is modelled by passing account context, never silently presumed.

The uniform-shock stress is deliberately an upper bound, and the service now brackets it from both sides. A *factor-beta* view scales each asset's move by an empirical crash beta — measured, not guessed, from the October 10 event itself as each coin's `Hyperliquid` 4h peak-to-trough drawdown against `BTC`'s -17.7% anchor (`ETH` 1.5, `SOL` 2.2, `XRP` 3.3, the meme cohort near 5; a first-pass prior table had understated alt betas two-to-threefold and was corrected against the recorded prices, which is the

point of measuring). These are severe crash-regime betas that embed liquidation-cascade feedback, and the response says so. The same construction was validated at population scale: replaying it over every open perp position on the venue (79,386 accounts, \$14.6 billion notional, real margins and the venue's own liquidation prices, from the public Reservoir archive) at T-24h before the October 10 cascade, the accounts it flagged were subsequently wiped at $3.9\times$ the rate of those it cleared, and the flagged band sitting 10–17.7% out — safe under routine volatility, reachable only by a correlated crash — was wiped at $7.4\times$ the rate of everyone further out. The betas themselves then survived a *pre-registered* cross-event test — hypotheses and pass thresholds fixed in published files before any number was computed — the pass bars and thresholds appear as literal constants in the query files, and the calibration set contains only 2025 episodes — with a hard temporal cutoff: betas calibrated on 2025 episodes predicted 2026 episodes' per-asset risk ranking (median Spearman 0.657 across five events), and on the two never-before-seen 2026 crashes the same flag, computed a day early with calibration-only betas against censoring-free victim sets from the full fill stream, carried a relative risk of $14.3\times$ (June, 16,492 liquidated addresses in the fill stream, of which 12,115 belong to accounts present in the T-24h snapshot and therefore enter the tables; the remaining $\sim 4,400$ opened and were liquidated inside the window and cannot be scored by a flag computed a day earlier) and $13.3\times$ (February, the archive's deepest event at -24.1% peak-to-trough in BTC, 19,541 addresses in the fill stream, of which 13,383 were in the snapshot and scored, leaving 6,158 — 31.5% of that crash's victims — outside anything the flag could have said) — both with perfectly monotonic dose-responses, and the consistency across independent events is itself the robustness evidence. The service exposes these cross-validated betas as severity tiers (`betaTier`), each response carrying the validation record; the worst-case single-event table remains the labeled default. A *cross-margin* view then computes the account-level liquidation for a shared-equity book — the market move at which pooled equity falls to total maintenance — under which a hedged book survives far beyond its scariest isolated leg, the real diversification an isolated view cannot see. The three views ship together, each labelled with its assumption.

The book itself no longer has to be typed in. Passing a single Hyperliquid address pulls the account's full live book through the keyless `clearinghouseState` API — positions, entry prices, margins and margin modes, account equity, and the venue's own per-leg liquidation prices — and runs everything above on it. Because that answer embeds a live fetch, it ships the observation envelope of Section 5 — an adversarial review pass caught the first build wearing the deterministic proof envelope here, an overclaim by this paper's own two-envelope doctrine, and it was corrected the same day — with the frozen book echoed in the envelope's inputs and the useful property stated precisely: the risk *mathematics* re-runs deterministically from that snapshot, while the snapshot itself is a committed observation. The venue's liquidation prices serve a second purpose: they are the one verifier the engine does not control. For isolated legs the venue's number and the engine's model the same quantity, so the response cross-checks them and the self-check fails beyond a small tolerance; for cross-margined legs the venue's price embeds the shared pool, the comparison is labelled rather than forced, and the per-leg deviations remain visible. On a real five-leg, \$2.2 million account observed during development, the isolated view read "nearest liquidation 3% away" while the venue's cross-margin prices sat 240% to 62,000% away — the gap between the two models, shown side by side with each one's meaning, is precisely the information a margin desk wants.

5.19 The proof envelope: correctness you re-derive

Every risk computation ships a *proof envelope*, and because the engines are deterministic the envelope is a reconstruction, not a promise. It carries the exact inputs, the code identity, a content hash $H(\text{engine}, \text{code}, \text{inputs}, \text{result})$, and the self-check results, so a caller re-runs the open-source engine on the echoed inputs and reproduces the number bit-for-bit — correctness re-derived, not trusted. When a signing key is configured the content hash is additionally signed with `secp256k1` over its EIP-191 digest, so a proof recovers to a known address — Quiver's published attestation signer `0x946324E0E5d7D77206731E35Ef4044a383e2a8C2` — and a tampered result no longer does. Signing, though, is only table stakes — a policy gate can sign its policy. The deeper guarantee is the one underneath: each engine tests its own output against an invariant — the liquidation condition, the Kelly first-order condition, the constant-product k , the martingale $E[S_T]=F$, the arbitrage-free density — so an answer that does not satisfy the condition it was solved from fails its own check *before* it is ever signed. That the checks bite was tested rather than asserted: a reviewer with live access copied the engines aside, corrupted one formula in each, and every corruption was caught, six for six, at tolerances tight enough that a one-percent error moves the residual by hundreds of units.

The same reviewer then named the limit of that guarantee, and this document carries it here rather than the flattering summary it used to. **These checks establish that the closed form matches the condition as coded. They do not validate the inputs, and they do not validate the choice of model.** If the maintenance-margin rate is wrong for the venue, perp-gate computes a wrong liquidation price and the invariant still passes at a residual near machine epsilon — because both sides of the comparison use the same wrong rate. They catch implementation error; they cannot catch specification error. An earlier draft called all five “ground-truth invariants” and concluded that Quiver “proves it is right rather than merely proving it spoke”; the first phrase oversells the distinction and the second is not earned by a check that shares its assumptions with the

thing it checks. Two of the five do reach further, and are worth naming as the exceptions: the options-risk greeks are checked against finite differences of an *independently repriced* book, and the recovered density's martingale residual tests a numerical integration against a quantity derived by a completely different route. Each of those two has caught a real defect. The external check that is not ours at all — the venue's own published liquidation price, compared against our computed one — is described in Section 5.18, and it is the only one in this system whose disagreement we cannot explain away.

VERIFYING A QUIVER ATTESTATION — RECOVER THE SIGNER OF ANY T1 PROOF

```
// Every T1 proof signs its content hash with secp256k1 over the EIP-191 digest,
// so anyone can recover the signer from a paid response and check it is Quiver's.
import { verifyMessage } from "ethers";

const QUIVER_SIGNER = "0x946324E0E5d7D77206731E35Ef4044a383e2a8C2";
const { contentHash, signature } = response.proof; // signature = { signer, signature, ... }
const recovered = verifyMessage(contentHash, signature.signature);

recovered === QUIVER_SIGNER; // true => the number came from Quiver, untampered
// The deeper check needs no key at all: re-run the open-source engine on
// proof.inputs and confirm sha256(canonical{engine, codeHash, inputs, result})
// reproduces contentHash — correctness re-derived, not trusted.
```

Live-market services cannot make the re-run promise — the market moves between calls — and pretending otherwise by signing everything and implying reproducibility is exactly the costume this design refuses. They ship the honest sibling instead: an *observation envelope*, which commits to what was observed and served at a stated instant. It carries the same content hash and the same secp256k1 signature, plus the observation timestamp, and its semantics field states outright that the answer is *not* re-runnable and why — the re-run claim is reserved for the deterministic services. One serialisation detail matters for anyone implementing verification: the hash is taken over the response's *wire form* (one JSON round-trip), because JSON serialisation drops undefined-valued keys and stringifies dates, so a hash over the raw in-memory object would false-fail for the consumer recomputing from the response they actually received. The SDK's `verify()` accepts both envelope kinds; its `reproduce()` refuses observation envelopes with the reason stated. Every paid response that delivers an answer therefore carries a verifiable, signed, on-chain-anchorable commitment, and the two claims are never conflated.

6. Verification and Testing

Two mechanisms keep the services honest: an automated test suite of model-free invariants, and a ground-truthing protocol that runs each service against live data and checks the result against an independent reference.

6.1 The invariant test suite

The financial mathematics is locked by 386 automated tests that run under the built-in Node test runner ^[34] with no external dependency — 381 of them run on every build, and five are live-archive integration tests exercised behind an RPC flag. They assert identities that must hold for any inputs, so a regression in the math breaks the build rather than shipping a wrong number; defects caught against live data during development (a mis-scaled reconciliation tolerance, a skipped enrichment path, a microstructure-contaminated front slice) are each locked by a test that fails on the pre-fix code. The load-bearing tests are the following.

Property asserted	Why it must hold
Put-call parity, $C - P = e^{-rT}(F - K)$	Model-free; a violation means the pricer is wrong.
Delta parity, $\Delta_c - \Delta_p = e^{-rT}$	Follows from parity; guards the greeks.
Vanna, volga equal analytic and finite-difference values	Certifies the closed forms of equation (3).
Corrected probability (8) matches a numerical Breeden–Litzenberger derivative	Certifies the smile-slope correction.
Risk-neutral density is non-negative and integrates to one	A proper density; negativity is butterfly arbitrage.
Distribution mean equals the forward (martingale)	The risk-neutral consistency condition, equation (9).
Total-variance interpolation is non-decreasing in maturity	Calendar no-arbitrage, equation (6).
Every indicator is causal (no look-ahead)	A value at bar i must not change when future bars are added.
Microstructure estimators recover known-answer synthetic tapes; dust does not explode Amihud	Guards the block estimators of Section 5.5.
An internal Merkle node presented as a member leaf does <i>not</i> verify, and the served root reproduces under an independent byte-level implementation	Soundness and on-chain verifiability of <i>risk-attest</i> . The first of these can fail structurally; the earlier non-member check could not, and a reviewer walked through the gap (Section 11.5).
Expected divergence lies in $(-100\%, 0]$, on the inputs actually served	V2 divergence loss is bounded; a figure outside the range is not a large loss but a wrong answer. Checked on the served value, not on a fixed reference.
Re-running <i>1p-risk</i> at its own reported break-even volatility nets zero	Forces the break-even and the headline expectation to be solutions to the same equation rather than two approximations that disagree.
A position whose mark is past its liquidation price is labelled below-maintenance and excluded from the book's nearest- <i>future</i> ranking	Separates an event that has happened from one that has not; the arithmetic was already consistent when the semantics were wrong.

Table 2 — representative invariants from the 386-test suite. The 381 that run in the default environment all pass; five need an archive node.

The no-look-ahead test deserves emphasis because it is the single most important property of anything computed on a time series and the easiest to get subtly wrong. It recomputes each indicator on a truncated prefix of the data and asserts that a settled past value is unchanged. An indicator that fails it is peeking into the future, and any backtest built on it is invalid.

6.2 Ground-truthing protocol

Passing synthetic tests is necessary but not sufficient, because a synthetic test only checks what its author thought to check. Every service was therefore run against live venues and its output checked against an independent method. This is where the real defects surfaced, and the value of the protocol is best shown by what it caught. Table 3 lists the corrections that ground-truthing forced; each was invisible to the synthetic tests and obvious the moment the service met live data.

Symptom on live data	Root cause	Fix
Probability off by up to 5 points versus a numerical density on a BTC smile	$N(d_2)$ omits the volatility-slope term of equation (8)	Replaced with the smile-corrected probability; correction size reported per level
Amihud illiquidity of order 10^9 on a memecoin tape	Sub-cent dust trades drive the per-trade denominator to near zero; hot-token feed is sampled	Estimate on equal-count volume blocks; Amihud falls to 0.303
USD Coin reported as not a proxy	USDC predates EIP-1967 and stores its implementation in the legacy OpenZeppelin slot	Added the legacy slot to the proxy check; USDC now flagged upgradeable
A variance premium that looked real	Overlapping windows inflate the sample and manufacture significance	Test at the effective independent sample; report $p = 0.254$, not significant
Near-expiry volatility slices fitting badly	Sub-day smiles are jump-dominated; SVI is diffusive	Exclude those slices by design and disclose, rather than force-fit

Table 3 — defects caught by ground-truthing against live venues, and the fix each forced.

The pattern in every row is the same: the code was internally consistent and wrong, and only contact with reality exposed it. The lesson is the protocol itself. Build, then point the thing at the market and believe the market over the code.

6.3 Validation against the real market — population-scale, pre-registered, out-of-sample

Tests and ground-truthing establish that the mathematics is implemented correctly. A harder question is whether the number the risk engine reports *means* anything — whether the accounts it would have flagged are, in fact, the accounts that die. That question was answered on the historical record, at population scale, using the venue's own data: daily snapshots of *every* open perpetual position on Hyperliquid (position, entry, leverage, margin mode, and the venue's own liquidation price), joined against the addresses actually liquidated in each crash, reconstructed from the raw fill stream so that every liquidation path is counted.

The protocol matters as much as the result, because a replay can always be tuned until it flatters. Here it could not be: the hypotheses, thresholds, data definitions, and pass/fail bars were **fixed in published files before any validation number was computed**, with a hard temporal cutoff at the end of 2025 — betas measured only on 2025 stress episodes, tested only on 2026 events the calibration had never seen. Deviations from the registered plan, where they occurred, are disclosed in the study; none was in the model's favour.

Event	Role	Accounts	Flagged & wiped	Cleared & wiped	Relative risk
Oct 10, 2025 (−17.7%, largest liquidation day on record)	replay (calibration event)	79,386	7.07%	1.83%	3.9×
Feb 2026 (deepest event in the archive)	out-of-sample	66,148	43.97%	3.30%	13.3×
Jun 2026 (−17.2%)	out-of-sample	100,034	25.37%	1.78%	14.3×

Table 4 — the risk engine's flag versus reality. "Flagged" = the account's beta-scaled distance to liquidation, computed from the snapshot taken roughly a day earlier, was within the registered threshold. Victim sets for the 2026 events are censoring-free (full fill stream). **The October row's levels are not comparable to the 2026 rows.** Its victim set was assembled by tagging fills to a liquidator label and six known liquidator addresses — 4,426 distinct addresses from 10,120 tagged fills over the 24 hours around the trough — so any liquidation routed through a path we did not recognise is missing, and both the 7.07% and the 1.83% are biased downward by an unknown amount. The 2026 rows instead read the archive's own liquidation flag over the entire fill stream and need no such recognition. Undercounting removes victims from the flagged and cleared cohorts alike, so the ratio survives the bias far better than the levels do, but not perfectly: were liquidator coverage systematically better for accounts near liquidation than for accounts far from it, the ratio would be inflated too. October is also the calibration event and therefore carries no out-of-sample weight in the first place; it is shown for the dose-response shape and the censoring lesson, not as a third result.

Three properties of these numbers deserve emphasis. First, the two out-of-sample relative risks — 14.3× and 13.3× on independent events — are consistent with each other, and cross-event consistency is precisely what tuning-to-a-backtest cannot produce. Second, the dose-response is monotonic on both out-of-sample events: on the June crash, accounts within 5% of liquidation were wiped at 40.9%, those 5–10% out at 11.6%, those 10–13.6% out at 3.2%, and those beyond at 0.8% (the bands are cut on raw distance-to-liquidation and cover only accounts carrying a measurable down-risk leg, which is why the widest band reads 0.8% while the flag's cleared cohort in Tables 4 and 6, which also contains accounts with no down-risk leg at all, reads 1.78%) — the flag is not a binary lucky guess but a graded distance that reality respects. Third, the October replay also identified *which* cohort a correlated-stress warning uniquely protects: accounts sitting 10–17.7% from liquidation — safe under routine volatility, reachable only by a correlated crash — were wiped at 7.4× the rate of everyone further out. The per-asset crash betas met their pre-registered transportability criterion on the median while missing it on one episode (the registered bar was a median Spearman above 0.60; rank-correlation of 2025-calibrated betas against five 2026 episodes: 0.530, 0.650, 0.657, 0.670, 0.775, median 0.657, minimum 0.530 against a pre-registered floor of 0.60 — one of the five episodes fell below that floor), and ship in the service as severity tiers with this validation record attached to every response. What the betas do *not* do is carry the prediction, which the next paragraph establishes rather than assumes. The full study, its honesty ledgers, and the scripts that reproduce every number from public data (about \$1 of requester-pays S3 access) are in the public repository, github.com/Tristan-tech-ai/Quiver, under `research/`.

6.4 The same result at its least flattering

Relative risk is the most flattering true framing of a classifier, because it divides by a cleared cohort that can be made arbitrarily clean. It is not the number an operator needs. Someone deciding whether to act on this flag is choosing what to do with the accounts it fires on, so what matters is how often firing is right and how much better than the base rate that is — the figures below. Every one is computed from the cells of Table 4 and Table 6 and can be recomputed by a reader from those tables alone.

Measure	Jun 2026	Feb 2026
Accounts	100,034	66,148
Flagged share of population	43.79%	41.63%
Base rate (all accounts liquidated)	12.11%	20.23%
Precision (flagged and liquidated)	25.37%	43.97%
Recall among accounts the snapshot could see	91.7%	90.5%
Recall against <i>every</i> liquidation in the fill stream	67.4%	62.0%
Lift over base rate	2.09×	2.17×
Relative risk (flagged vs cleared)	14.25×	13.32×

Table 5 — the June and February out-of-sample events scored the way that is hardest on the flag. Every rate is computed over the accounts present in the pre-crash snapshot, which is the only population a flag computed a day early can speak about. The two recall rows exist because an earlier draft published the first one under the label “recall of all liquidations”, which it is not: a quarter to a third of the addresses liquidated in each crash had opened their position after the snapshot was taken, so no flag could have scored them. The unconditional figure is the one to quote against a claim of catching liquidations; the conditional one is the one to quote about the classifier.

The two framings describe the same data and carry very different weight. **Lift over the base rate is 2.1×**, not 14×, and lift is the number an operator should use when deciding whether to act on the flag. The 14× is real arithmetic and it is also the largest available ratio, because the cleared cohort it divides by is unusually clean; a reader who sees only that number is being shown the classifier at its most impressive angle. What the flag buys, stated plainly: it catches about nine in ten of the accounts that will be liquidated, at the cost of flagging four in ten of all accounts, and an account it flags is about twice as likely to be liquidated as an account picked at random. That is a useful triage instrument and it is not a precision instrument, and no confidence intervals are reported here because the events are two, not a sample.

6.5 An ablation the result did not survive intact

A relative risk is only as meaningful as the baseline it is measured against, and the baseline here is not one. Inside a crash, accounts closer to liquidation get liquidated; a flag built on distance-to-liquidation will separate the population even if every refinement layered on top of it is worthless. The specific question this raises is whether the beta scaling — the part of the flag that required a separate calibration study and is presented above as the sophisticated component — contributes anything at all, or whether raw distance would do the same work. We had not tested that. An adversarial review of an earlier draft of this document pointed it out, and the test is cheap, so we ran it.

The pipeline was run twice on each out-of-sample event, over the same snapshot and the same victim set, once with the pre-registered betas and once with every beta forced to 1. To keep the comparison fair, the raw threshold was set to the quantile that flags the *same number of accounts* as the beta-scaled threshold, so neither arm can win by flagging more or fewer.

Event	Arm	Flagged	Flagged wiped	Cleared wiped	Relative risk
Jun 2026 (100,034 accounts)	beta-scaled	43,807	25.37%	1.78%	14.25×
	raw distance	43,807	25.32%	1.83%	13.85×
Feb 2026 (66,148 accounts)	beta-scaled	27,539	43.97%	3.30%	13.32×
	raw distance	27,539	43.99%	3.29%	13.36×

Table 6 — beta-scaled flag against raw distance-to-liquidation, at matched flagged-population size.

Beta scaling adds nothing measurable. The unrounded ratios are 14.25× against 13.85× in June and 13.32× against 13.36× in February; the headline 14.3× and 13.3× elsewhere in this document are these same figures rounded. Beta scaling is marginally ahead on the June event and marginally behind on the February one, which is what noise looks like. Nor is the threshold more transportable after scaling: the February-to-June threshold ratio is 1.61 in beta units against 1.46 in raw units, so a single fixed cutoff travels slightly *worse* in the scaled measure. The honest reading is that the calibration study established the betas are stable enough to rank assets, and the ablation establishes that this stability is not what produces the 14.25×

What survives is worth stating precisely, because it is less than the earlier draft implied and more than nothing. The flag is not tautological: 74.6% of flagged accounts were *not* liquidated in June, while cleared accounts were wiped at 1.8%, so the separation is real rather than a restatement of the definition. What the crash study validates is that distance-to-liquidation predicts liquidation in a crash, and that this held on two events the thresholds had never seen. It is worth being exact about *whose* distance, because an earlier version of this passage was not. The flag reads the `liquidation_price` column of the venue's own position snapshot — literally `liquidation_price AS liq` in `h2.sql`, in `h2b.sql`, and in both arms of the ablation — and divides the re-

sulting distance by a crash beta. It never calls `perp-gate`. No number this engine computed appears anywhere in the result. So the study establishes that *the quantity* predicts liquidation, and not that *our computation of it* does. The beta layer remains in the product as a severity tier, now shipping with a null result attached to it rather than an implication. The ablation script and its output are in the repository at `research/reservoir-data/ablation.sql` and `ablation-feb.sql`, with the recorded output in `ablation-result.json`, and reproduce in under a minute against the same public snapshots.

Two things follow from that, and the first is a criticism this document should have made of itself before a reviewer did. **The naive arm a sceptic would ask for is already in the study, unlabelled.** An agent that can read the venue's position endpoint gets that liquidation price for free, so the “raw distance” arm of Table 6 is not a stripped-down version of our method — it is the method an agent could run without us, and it scores 13.85× against 14.25×. Presenting the beta scaling as the sophisticated component obscured that the unsophisticated component was already a baseline, and the ablation that retired the betas was therefore doing more work than it was credited with. **And the sentence this passage replaces claimed the validated thing was distance “computed correctly, across venues and margin regimes”,** which is exactly the part the study did not test, because the venue did the computing.

The engine's own correctness therefore rests where it always did, on evidence of a different kind rather than on this study: the liquidation invariant asserted on every single call, which substitutes the solved price back into the venue's stated condition and fails loudly if it does not hold (Section 5.10); and `portfolio-gate`'s cross-check against the venue's *published* price on isolated legs, which fails beyond 2.5 points and is the one verifier in this system we do not control. Those establish that we compute the same number the venue does, wherever the venue publishes one. The case for computing it at all is the case where the venue does *not* — a second venue with a different margin regime, a cross-margin account whose legs offset, a position the agent is considering but has not opened — and none of those appear in this study either, because a historical position snapshot only contains positions that existed. A reader who wants the crash result and the product's value proposition to be a single argument is entitled to notice that they are two arguments, and that only the first one is measured here.

Finally, the payment surface itself was field-tested the way a buyer would: every one of the twenty-two services was purchased end-to-end with real money on *both* rails — USDC on Base through the CDP facilitator, and USD~~F~~o on X Layer through the OKX facilitator, the latter signed by an OKX agentic wallet's TEE exactly as a marketplace buyer would sign — with every 402 challenge, on-chain settlement, and response envelope independently verified, and the ledger reconciling to the cent. Those test purchases are quality assurance, not traction, and are never counted as sales.

6.6 Independent buyer verification, and a defect it found in the money path

Everything above shares one weakness: the party that writes the code also grades it. To narrow that gap, a separate autonomous buyer agent was commissioned to operate as a paying customer for four days (21–25 July 2026) from its own wallet (`0xba3a...1f9b` on X Layer), on a different machine and network egress, with no access to the server, its logs, or any internal credential. Its mandate was to buy services inside genuine decision contexts, verify every envelope independently against a clone of the public repository, and report defects; it was told explicitly that finding nothing would count as a failure of the exercise. One disclosure belongs at the front: that buyer was commissioned by the author, so this is an *arm's-length audit*, not a *third-party endorsement*. What makes it worth reading is that every figure it reports is reproducible from public data using the commands it published — and that it retracted its own headline finding when the finding turned out to be wrong.

Across 1,089 paid calls the buyer recorded **983 independent envelope verifications, the gap to 1,089 being calls whose answers were refusals or unavailable-data responses and so carried no envelope to verify, with zero failures**: content hashes recomputed, self-checks re-evaluated, signers recovered, code hashes compared against a fresh clone, and deterministic engines re-run for byte-identical output. That chain held across a mid-audit deploy, when a `git pull` on the clone immediately reproduced the new code hash. No response was ever found to disagree with its own proof.

The buyer wrote that report on 25 July, and its desk kept running for another day — stopping at 02:00 UTC on 26 July because its own spending envelope ran out, not because it had finished. The figures above are the ones the buyer published and stood behind. The figures that follow are *our* recomputation from the buyer's own ledger, carried to the point the desk stopped, and they are labelled that way because the buyer never certified them. The recomputation does not ask to be taken on trust either. The ledger is published at `research/BUYER_LEDGER.csv` and the script that reads it at `research/buyer-ledger-recount.mjs`, so `node research/buyer-ledger-recount.mjs` reproduces every number in this paragraph and in Table 7 from a clone, with no network access and nothing to configure. That script starts by re-deriving the three figures the buyer *did* publish — 18, 43 and 27 settled rows with no transaction hash on 21, 22 and 23 July — and exits with an error instead of printing anything else if they fail to come out, so the method is pinned to numbers somebody else checked before it is pointed at numbers nobody has. Extended to the full record it finds **1,785 ledger rows, 1,721 of them settled, and 1,750 envelope verifications with zero failures**; the desk log contains exactly one distinct verification outcome, `verify=OK`, and no line recording any other, which is the check that the zero is a measurement rather than an artefact of what we searched for. That count

should be read for what it is rather than for how large it sounds. The 1,785 ledger rows touch **fourteen of the twenty-two services**, and perp-gate alone accounts for 1,102 of them — 62%. So the figure establishes two things and not a third: that the envelope is stable under heavy repetition, and that no mismatch was ever observed on the services the desk exercised. It does *not* establish breadth. Eight services never appeared in the desk's traffic at all, and a thousand calls to one endpoint is one fact with a large exponent, not a thousand independent ones. What this mostly adds is post-fix volume, which is precisely where the earlier account was thinnest: **719 settled calls after the acceptance fix, of which zero lack a settlement transaction hash**, against the 87 the buyer had in hand when it wrote. What it does not add is coverage of the build this document publishes. The desk's last call was at 01:45 UTC on 26 July, against the build this document has since superseded, q1-bce7e7bccb16ea1b, so no part of this audit — neither the buyer's portion nor the extension — touched q1-e1fa99d08887d6cc. That gap is now partly closed, by us and labelled as such. Every one of the nine deterministic engines was called over the free MCP path on this build, and each response's content hash was recomputed independently from the response body — implementing the published recipe rather than reading the flag the service sets. All nine answered, all nine hashes reproduced, and every self-check passed. The sweep is worth reporting mainly because it *failed first*: on its first run `treasury-risk` returned `allSelfChecksPass: false` on a perfectly valid two-asset book, because its concentration check compared a full-precision recomputation against a figure published rounded to four decimals, under a tolerance four orders of magnitude tighter than that rounding. The identity held; the check could not pass. It is fixed, the check now compares at the precision it publishes, and every check in that engine states the tolerance it was judged against. A verification pass that never finds anything is not evidence that there was nothing to find.

The audit's substantive finding was in the money path, not the mathematics. The buyer's ledger claimed more settled value than its wallet had actually paid out: a fraction of calls returned a delivered answer while the facilitator reported `success:true` with `status:"timeout"` and *no transaction hash*, and those settlements never landed on chain. Measured two independent ways — by ledger rows lacking a hash, and by a balance identity that touches neither the ledger's claims nor any history API — the shortfall converged on **8.78% by call count over the 1,002 calls that predate the fix, 8.08% over all 1,089 calls, and 6.84% by value** (balance identity: 6.51%). The 8.78% figure is the right one for the defect, since the 87 post-fix calls in the wider denominator could not carry it; the buyer report quoted 8.08%, and an adversarial reader was right to notice that a diluted denominator sits awkwardly beside a claim to have adopted the least favourable reading. The defect was in Quiver's acceptance rule, which treated the success flag rather than the presence of a transaction as proof of settlement. The fix gates settlement on the transaction hash, retries once idempotently (the EIP-3009 nonce makes a duplicate settle harmless), and otherwise returns a 402 that states the caller was *not* charged. Note the direction of the error: it cost the operator revenue and cost callers nothing, which is the failure mode a paid service should prefer, but it was a defect either way and it was invisible from the server's own logs until an outside ledger exposed it.

Period	Settled rows	Rows with no transaction hash	Share
2026-07-21	250	18	7.20%
2026-07-22	503	43	8.55%
2026-07-23	249	27	10.84%
2026-07-24	<i>no rows: the buyer desk was halted for the day by an outage on the payment-rail provider's side, unrelated to this service, which is also the day carrying the eighteen-minute availability gap reported in Section 8</i>		
2026-07-25 (after fix)	661	0	0.00%
2026-07-26 (after fix, to 01:45 UTC)	58	0	0.00%

Table 7 — settlement shortfall before and after the acceptance fix, measured by the buyer from its own wallet. The three pre-fix rows are the buyer's published figures. The two post-fix rows are recomputed from the same ledger after the desk had run to a stop, and are larger than the buyer's own report because it was written mid-afternoon on 25 July with 87 calls in hand; the desk made 574 more that day and 58 the next before its spending envelope ran out. The recomputation reproduces the buyer's three pre-fix numbers exactly, which is why it is shown beside them rather than in place of them.

The strongest post-fix evidence is a closed window rather than a percentage. Over twenty-five minutes in which no trading fill contaminated the balance (the wallet's OKB holding was identical at both ends), sixteen paid calls claimed 0.195000 USD $\mathbb{F}_0$ and the on-chain balance fell by exactly 0.195000 USD $\mathbb{F}_0$ — **a gap of 0.000000**. That test depends on neither a history API nor any field in a response, which is what makes it hard to fabricate. Twelve transaction hashes drawn at random from the post-fix day were additionally queried one at a time and all twelve confirmed as successful on chain. In the same pass the buyer verified, by wallet balance before and after rather than by trusting a response field, that an empty loop-digest read, three wrong-endpoint calls, and a transient upstream failure were all served for **exactly zero**.

The episode that most deserves recording, however, is the buyer's correction of itself. Its first report put the shortfall near 11%, using the wallet's paginated aggregate history endpoint as on-chain truth. Challenged to establish that number independently, it queried eight of the supposedly missing transaction hashes individually — and all eight existed on chain, successful. The aggregate history endpoint was incomplete; a large part of what it had called a leak was a hole in an API. It withdrew the 11% figure, re-measured by two methods that avoid that endpoint entirely, published the arithmetic that separates a per-count from a per-value rate (a distinction its first draft and the operator's own first estimate had both blurred), adopted the reading least favourable to Quiver as its headline, and wrote a methodological warning for whoever measures next: never treat a paginated aggregate history as ground truth. A measurement discipline that corrects its own published number is the only kind whose surviving numbers mean anything, which is the same argument this document makes for proof-carrying answers.

Concurrency, which this document previously had nothing to say about. The buyer's own limitation list, repeated below, records that its planned ten-identity test was cancelled and that consequently no figure described behaviour under simultaneous load. That gap is now partly closed. Eighty paid calls were fired at the live service in bursts of 2, 3, 6 and 12 *simultaneous* requests, on a deterministic service with fully explicit inputs so that nothing depended on an upstream venue having a good minute. The result separates cleanly into two findings that must not be merged. **The settlement accounting held exactly at every level tested.** Forty-nine calls were delivered and charged; the wallet's on-chain balance fell by exactly what the responses claimed, to the last decimal, in every burst — 0.490000 claimed against 0.490000 moved — with forty-nine distinct transaction hashes, no duplicate, and every one of the forty-nine confirmed individually on chain. Nothing was charged twice and nothing was charged without an answer. **Delivery, separately, degrades under simultaneity.** At two and three concurrent calls every request was served; at six, two independent runs delivered 8 and 9 of 12; at twelve, 24 of 48. The thirty-one calls that were not served were re-challenged with a 402 and cost **exactly zero** — the documented behaviour when settlement does not complete, and the direction of failure a paid service should prefer.

Three qualifications belong with those numbers rather than after them. First, **the cause is not isolated.** Payment authorisations are signed by the wallet provider's own command-line client, which generates the authorisation nonce inside its backend rather than in our code, so the degradation could sit in that client under twelve simultaneous invocations, in the facilitator, or in this service — and this document is not entitled to say which. Reading it as "the service handles three concurrent callers" would be a claim the measurement does not support; what it supports is that the *end-to-end path*, including a payment rail this document elsewhere records as the fragile part, delivers less than it accepts as simultaneity rises. Second, **one payer identity:** the ten-identity design existed to test isolation *between* payers, and that remains untested — two wallets paying at once could still interfere in a way a single-wallet burst cannot see. Third, this is our own money moving to our own operator wallet, which makes it quality assurance and not revenue, and it is counted as neither a sale nor traction, consistent with every other figure in this section.

The buyer's own limitations are stated in its report and are worth repeating here, because they bound what the section above establishes. The post-fix sample is 87 calls across roughly one day against 1,002 before the fix, so a zero-gap day is not yet a zero-gap guarantee at volume — the one item on this list that the extended ledger above partly retires, taking that sample to 719 calls across two days with the gap still at zero, while leaving it one desk, one payer identity, and no concurrency. A planned ten-identity concurrency test was cancelled by an infrastructure outage on the payment-rail provider's side, so *no* figure in the buyer's own audit describes behaviour under high concurrency — the gap the burst test above was run to close, and which it closes only for a single payer identity, leaving the between-payer isolation the ten-identity design existed to probe still untested. From outside the server, a settlement that was never attempted cannot be distinguished from one that failed silently. And the claim that all thirty-two probed protocol dossiers return complete — measured by the operator — could not be confirmed by the buyer, because one of its three spot checks met an upstream timeout; the service answered honestly and free of charge, but completeness depends on a third party's uptime at the moment of the call. The buyer's full report, including the exact commands to reproduce every figure, is in the public repository at `research/BUYER_VERIFICATION_REPORT.md`, with its defect journal beside it at `research/BUYER_BUG_REPORT.md`.

7. Worked Walkthrough

This section is the case-study companion to the specification: it runs services against live data and annotates what a caller actually receives and why each line can be trusted. The outputs below are real responses captured during validation, lightly trimmed for space; Appendix B gives the exact request that reproduces each. Two larger case studies live elsewhere in this document and are worth reading together with these: the population-scale crash validation of Section 6 (what the risk flag would have said the day before three real crashes, and what then happened), and the agent's-loop scenario at the end of Section 10 (how the services compose inside one trading decision).

7.1 Options: the risk-neutral distribution of Bitcoin

A single call to options-desk for BTC returns, among its layers, the full risk-neutral distribution of the terminal price for the front expiry. The block below is the live output for an expiry roughly two days out.

RESPONSE FRAGMENT — OPTIONS-DESK, BTC, DISTRIBUTION BLOCK

```
"distribution": {
  "quantilesUsd": { "p05": 61423, "p25": 63116, "p50": 64091, "p75": 65057, "p95": 66650 },
  "expectedShortfallUsd": { "left5": 60186, "right5": 67755 },
  "rnVolPct": 2.6, "rnSkew": -0.204, "rnExcessKurtosis": 3.303,
  "selfCheck": { "meanVsForwardPct": 0.001, "densityMass": 1.0 }
}
```

The quantiles are monotone and centred near spot, as a two-day distribution should be. The negative skew of -0.20 is the downside-protection demand characteristic of a crypto smile, and the excess kurtosis of 3.3 is the fat tail. The line that matters most is `selfCheck.meanVsForwardPct: 0.001`: the mean of the recovered density reproduces the forward to a thousandth of a percent, so the distribution is not a plausible-looking artefact but a properly normalised risk-neutral density whose mean reproduces the forward, which is necessary for correctness without being sufficient for it. Every figure downstream, the probability ladder, the expected shortfall, the cross-market divergence, inherits that correctness.

7.2 Safety: an unlimited approval to a wallet

A call to `calldata-x` with an approve of unlimited USDC to an externally owned address returns a danger verdict, because an unlimited allowance to a personal wallet is the signature of an approval drainer.

RESPONSE FRAGMENT — CALLDATA-X, UNLIMITED APPROVE TO AN EOA

```
"verdict": "REVIEW_HIGH_RISK",
"alert": { "level": "DANGER" },
"spenderReputation": {
  "tier": "eoa",
  "basis": "eth_getCode -> EIP-7702 delegated wallet",
  "activity": { "outboundTxCount": 5932 }
}
```

The same call structure with the spender set to a recognised router returns a routine note, not a danger, which is the false positive a naive rule would raise on every legitimate swap approval. Pointed at USD Coin, the target-proxy check now reports `is-Proxy: true` under the legacy OpenZeppelin standard, the correction that ground-truthing produced.

7.3 Microstructure: price impact on a live tape

A call to `tape-pulse` on a liquid token returns the three estimators of Section 5.5. On a dense tape such as PEPE, Kyle's lambda is estimated with a meaningful fit; on a sampled tape it honestly reports low confidence rather than a fabricated coefficient.

RESPONSE FRAGMENT — TAPE-PULSE, PEPE MICROSTRUCTURE

```
"microstructure": {
  "kyleLambda": { "priceImpactBpsPer10kUsd": 13.9, "rSquared": 0.175, "confidence": "medium" },
  "amihud": { "pctMovePer1kUsd": 0.089 },
  "vpin": { "value": 0.66 },
  "tape": { "medianGapSeconds": 48, "spanHours": 22.2 }
}
```

A net ten thousand dollars of one-sided flow is associated with about fourteen basis points of move, with a fit R -squared of 0.175, a moderate fit for a volume-block impact regression, which is why the service labels the confidence rather than presenting the coefficient alone. VPIN of 0.66 marks moderately one-sided flow. The tape-density line shows a forty-eight-second median gap, dense enough that the impact read stands on firm ground.

7.4 Execution: the cost of size on a prediction market

A call to `poly-fill` on a live Bitcoin-threshold market walks the book and fits the impact curve.

```

"marketImpact": {
  "model": "cost ~ k * sqrt(notional) (square-root impact law) FITTED to the live book",
  "coeffPctPerSqrtUsd": 0.176, "fitR2": 0.552, "visibleDepthUsd": 94167
},
"bookDepth": { "totalBidUsd": 18845, "totalAskUsd": 94167, "imbalance": 0.167 }

```

The fit R-squared of 0.55 is the honest signature of a discrete prediction-market book: it follows the square-root law moderately, not perfectly, and the service measures that rather than asserting a clean coefficient. The depth imbalance of 0.167 shows a book heavily weighted to the ask side.

7.5 Charting: a rendered figure with numbers baked in

A call to `chart-press` returns a PNG an agent can attach to a message, together with a facts block that states, field by field, whether each number came from the candle series that is drawn or from the live token quote. The figure below is a live render for wrapped Bitcoin on the four-hour interval with an exponential moving average, Bollinger bands, a relative-strength panel, and a volume panel, produced by the fast browserless tier.

FIGURE 6 — LIVE CHART-PRESS RENDER (WBTC/USD, 4H, 72 BARS, EMA20 WITH BOLLINGER BANDS PLUS RSI AND VOLUME PANELS), CAPTURED 2026-07-26 13:55 UTC. THE FACTS BLOCK RETURNED WITH IT IS PRINTED BELOW SO THE PICTURE AND THE NUMBERS CAN BE CHECKED AGAINST EACH OTHER RATHER THAN TAKEN ON TRUST.



THE FACTS BLOCK RETURNED WITH THAT IMAGE — ABRIDGED TO THE FIELDS THE PICTURE CAN BE CHECKED AGAINST

```

"facts": {
  "symbol": "WBTC", "interval": "4H", "bars": 72,
  "priceUsd": 64480.6,
  "priceSource": "token-price-feed (live quote)",
  "lastDrawnCandleClose": 64480.5865119,
  "change24hPct": 0.57, "change24hSource": "token-price-feed (live quote)",
  "high": 68096.80909367,
  "low": 61231.93735519,
  "indicators": ["EMA20", "BOLL", "RSI", "VOL"], "width": 1200, "height": 675, "timezone": "UTC"
}

```

Three of those are readable straight off the image: the right-edge label prints 64480.59, and the annotated extremes print 68096.81 and 61231.94, matching `lastDrawnCandleClose`, `high` and `low` exactly. `priceUsd` is a different measurement — the live token quote — which is why the response names its source. Here the two agree, because the final bar is still open and its close tracks the quote; in a capture forty-five minutes earlier they differed by 65.70, or ten basis points.

That small gap is worth the paragraph, because until this document was revised the response asserted it could not exist. The provenance block claimed the facts “cannot drift from the picture”, and named `price` and 24-hour change among the fields computed from the drawn series — which they were not. A reader following the response's own reconciliation instruction would have found a mismatch the response said was impossible, and would have been right to distrust everything around it. The fields `priceSource`, `change24hSource` and `lastDrawnCandleClose` exist so the comparison can be made rather than assumed, and the guarantee is now stated only over the two fields that earn it. Section 11.5 records this alongside the defects a reviewer found, because it is the same kind of error and it was ours.

The property worth having here is narrower than “the numbers and the picture cannot diverge”, which is what this document claimed until the paragraphs above were written. The candle series is drawn once and summarised once, so an agent that quotes `high`, `low` or `lastDrawnCandleClose` back to a user cannot contradict the image it attaches. Where a second source is genuinely better — a live quote is fresher than a closed bar — the service uses it and says so, rather than degrading the number to preserve a tidier guarantee. The anti-hallucination property is that every field names its own origin, not that every field has the same one.

8. Limitations and Honest Disclosures

A service is only as trustworthy as its account of what it cannot do. The following limitations are real, and each is disclosed in the relevant output rather than buried here. They are of two different kinds, and conflating them would be its own small dishonesty, so each is labelled: **[structural]** marks a limit imposed by available data, market microstructure, or statistics, which no amount of engineering on our side removes; **[scheduled]** marks an approximation that is shipping today because the better treatment is not built yet, with the commitment and its definition of done in Section 11.4. Nothing is moved out of this section for being merely unbuilt — a reader who discovers an undisclosed weakness is right to distrust everything else — but neither is unfinished work allowed to pose as a law of nature.

1. **[structural] The envelope is signed by our own server, so for live data a signature establishes that we spoke, not that we were honest about what we fetched.** This is the largest trust limitation in the document and it belongs first. It does *not* touch the deterministic services: there the caller clones the repository, checks the code identity, re-runs the engine on the echoed inputs and re-derives the number, so our honesty is not in the loop at all. It applies to every answer that reads a live venue, where the caller must take our word that the bytes we fed the engine are the bytes the venue returned. What a signature adds there is accountability, not correctness, which Section 3.1 already says; what this entry adds is that the gap is not uniform across the catalogue, and pretending it is would understate the good cases and hide the real one.

Measured across 1,785 responses the buyer desk actually received, the live surface separates into three kinds. **Pinned on-chain state** needs no trust today: `calldata-x` publishes the block number and block hash its simulation ran against, so any reader re-queries any node at that block and gets the same state or catches us. **Immutable public history** — a settled candle series, a past trade tape, a replayed swap sequence — is re-fetchable after the fact and therefore checkable with a delay rather than in the moment; the weakness there is ours, not the data's, because `1p-desk` replays real on-chain swaps and now names the block range it walked — `firstBlock` and `lastBlock` beside the day count and swap count — so a reader re-fetches the identical window rather than an approximately similar one. The block numbers were already on every log the replay iterated; not publishing them was the defect, and the fix was two fields. **Ephemeral live state** is the irreducible case: an order book, a mid, a live volatility surface at an instant that will never recur. Nothing in the response can make that re-derivable, which is why those services ship an observation envelope that says so rather than a proof.

The honest size of the remaining problem is therefore the third kind, not the whole live surface. The remedies are real and named rather than gestured at: a zkTLS proof (TLSNotary and its successors) attests that a particular TLS session with a particular host returned particular bytes, which is exactly the missing link, since the computation on those bytes is already re-derivable; a hardware enclave attests the execution instead. Neither is trustless in the way the word is usually sold — zk-TLS moves the trust to a notary that can still collude with a prover, and an enclave moves it to a hardware vendor's root of trust — but both move it to a named party with published assumptions, which is strictly better than moving it nowhere. That work is not done here for two reasons, and the second is the more useful one to a reader. Running an enclave means

different hosting, and the endpoint is a registry entry (item 10 below), so it is not a change to make days before a deadline. And on the day this was written we went to check the best-known public notary and found it down at the origin — not blocked by our network, since the same host failed from a second, unrelated network as well. A remedy whose public infrastructure is unavailable on the morning you reach for it is worth reporting as such.

Two limits on the paragraphs above. The tier map was measured over the fourteen services the buyer desk exercised; the remaining eight were classified from their source, which is a weaker basis and is why `calldata-x`, one of the strongest cases, appears here on the strength of its code rather than a captured response. And an absent marker in that sweep means the desk never called the service, not that the service lacks the field — the distinction that separates a measurement from its own blind spot.

2. **[scheduled] The per-leg liquidation model is isolated margin, and on a cross-margined account that is the wrong model by orders of magnitude.** This belongs near the top because it is the largest realistic-harm surface in the catalogue and an earlier draft presented it as an insight rather than a limitation. Every per-leg liquidation price this service computes solves the isolated condition; a cross-margined account pools equity across legs, so each leg survives far past its isolated price. Section 5.18 reports the size of that gap on a real five-leg book: the isolated view read *3% away* while the venue's own cross-margin prices sat between 240% and 62,000% away. Two consequences were understated. The venue cross-check — the only verifier in this system we do not control — is enforced for isolated legs and, correctly but inconveniently, cannot be enforced for cross legs, because the venue's number and ours model different things; so the check is off exactly where the model is furthest from the truth. And an autonomous caller polling this endpoint takes the default, which is what an agent does. What has changed is the disclosure, not the model: the nearest-liquidation headline now names the margin model it solved under, says outright that its number is *not* the book's distance when any leg is declared cross, points at the account-level figure that is comparable, and treats silence about margin mode as an assumption rather than as isolated. *Done means*: a per-leg liquidation under the venue's published cross-margin formula, cross-checked against the venue's own price with the same 2.5-point gate the isolated path already uses.
3. **[structural] Dealer positioning in GEX is an assumption, not a measurement.** The gamma-exposure sign convention assumes dealers are long call gamma and short put gamma. An earlier version of this item, and of the service's own output, said the public feed “carries no block tag”. It does — `block_trade_id` and `block_trade_leg_count`, plus `block_rfq_id` on ETH — so that justification was simply false. The obstacle is attribution rather than tagging: a trade's reported direction names the side it was booked from, not whether a dealer was the buyer or the seller, and nothing in the feed says whether a block maker is a dealer at all. Because block-tagged trades are a large share of volume — 48.9% of BTC and 30.2% of ETH option contract volume in a 200-trade window sampled 27 July 2026 — a flow-based sign would be invertible on exactly the largest trades. GEX is a positioning map under a stated convention, not measured dealer inventory.
4. **[structural] The variance risk premium is not statistically significant on current data.** At the honest effective sample the p-value is 0.254. The real-world-calibrated probabilities that depend on it are marked indicative, and the raw risk-neutral figures, which do not depend on it, are the ones to trust.
5. **[scheduled] One-touch barrier prices use a single volatility.** A path-dependent barrier is not smile-consistent under any single volatility. What ships today is the honest width of that approximation: the service reports the model-uncertainty span between the barrier and at-the-money volatilities, so the caller sees how much the answer could move. An earlier version of this document argued that the rigorous treatments — a local-volatility model or a vanna-volga overhedge correction ^[9] — could not run per call. That is true of local volatility and *not* true of vanna-volga, which is a closed-form correction from three vanilla quotes the service already has, since it fits the whole smile to price these barriers in the first place. The barrier is therefore our unbuilt work rather than a property of the market, and it is committed in Section 11.4 on those terms.
6. **[scheduled] Transaction simulation is single-transaction and single-block.** It prices one call at one block. Two of the three things this item used to bundle together are not blocked at all: the adapter already speaks `eth_simulateV1`, which is a bundle simulator, and the array it passes simply has one element. Measured against the first public RPC in the service's own list, with no key: a three-call bundle carried state between calls (an allowance moving 0 → 1,000,000 across an approve in the middle), `stateOverrides` were accepted, and a two-block simulation ran. So multi-transaction bundles and custom state overrides are unbuilt work, not a missing capability, and they are committed as such. What remains genuinely out of reach for a stateless HTTP service is the MEV half: sandwich exposure needs a view of the pending mempool, which this service does not have and will not fabricate. Spender reputation uses reachable on-chain signals; a verified-source check and an approval-graph would need an indexer that is not wired.
7. **[structural] The exchange tape can be a sampled feed.** For a hot token the DEX trade endpoint returns a large-trade subset rather than the full tape, which weakens tick-level impact estimates. The service surfaces the tape-density diagnostic so this is visible, and reports Kyle's lambda with a confidence label.

8. **[scheduled] The macro calendar is curated and will age.** Events are a hardcoded table transcribed from published Federal Reserve, BLS, and BEA release schedules, which makes the service deterministic and free of an upstream dependency that can fail mid-call — a deliberate trade, but a trade. Its coverage ends with the last transcribed 2026 release. Past that date the service does not report a clear window: it returns `CALENDAR_EXHAUSTED`, a third verdict distinct from both `CLEAR` and `EVENTS_AHEAD`, alongside `certified: false` and a note saying that absence of events beyond the horizon is unknown rather than clear. A caller can therefore tell “nothing is scheduled” from “I cannot see that far” — the distinction that matters to anything trading around a release. A live primary-source fetch, with the curated table retained as the fallback when that fetch fails, is committed in Section 11.4.
9. **[structural] None of the services is financial advice.** They report what markets price and what data shows. Every output that touches a decision carries a not-advice disclosure, and the short-window up/down service refuses to output a directional edge at all.
10. **[scheduled] Hosting is single-region with no redundancy, and the record is published rather than promised.** Quiver runs as one container in one region on a trial-tier host. An externally hosted prober, on infrastructure separate from the service but our own rather than a third party's, has called the two free endpoints every two minutes. Over the window 21 July 13:49 UTC to 26 July 11:07 UTC — 4.89 days, in which the schedule fired 3,521 times against 3,519 expected at that cadence, so it did not miss a beat — 13 cycles saw at least one endpoint fail, giving **99.63% of cycles clean (3,508 of 3,521)** with a specific shape: four isolated single-cycle blips that cleared on the next cycle, and nine consecutive failures spanning one genuine outage of about eighteen minutes on 24 July 2026, during which the platform edge accepted TLS but returned no bytes — the container was gone and the edge held connections open instead of failing fast. Two consequences are worth stating plainly. First, during such an outage Quiver cannot explain itself: the component that would apologise is the one that is unreachable, so a caller sees a timeout with no reason. That is why the availability record and current status are served from separate infrastructure, at `cg9npwmm0.execute-api.us-east-1.amazonaws.com` (machine-readable at `?format=json`), which stays reachable when the service is not. Second, a watchdog on that same external infrastructure now probes twice a minute and, on two consecutive failures, restarts the deployment to force a fresh container; the restart was measured to be graceful, showing no observable downtime across eighty seconds of one-second polling, and its heal path was deliberately exercised end to end rather than assumed. This shortens the failure mode we have actually seen, but it cannot help when the platform's edge or region is itself broken. Redundancy across regions requires a custom domain, and the endpoint currently registered on chain is the platform subdomain; changing it is a registry update, so that work is sequenced in the roadmap rather than rushed. A caller that times out should retry after a short delay: in the measured record, every outage cleared without intervention.

9. Related Work and Positioning

Quiver sits at the intersection of three bodies of work: the agent-economy infrastructure that makes priced machine-to-machine services possible, the quantitative-finance methods the services implement, and the security tooling for on-chain transactions.

On infrastructure, `x402` ^[27] supplies the payment rail, `ERC-8004` ^[26] the identity and reputation layer, and the `A2A` and `MCP` conventions ^{[28],[29]} the calling and tool-exposure patterns. These standards define how agents pay and discover; they are deliberately silent on what is worth buying, which is the space Quiver occupies. Existing agent-registry services skew toward general assistants and simple data relays; a survey of the registry at build time found no service offering arbitrage-free options analytics, and the closest options-adjacent listings were not agent-callable.

On methods, the options stack rests on Black-76 ^[1], the Breeden–Litzenberger density ^[4], and the SVI surface of Gatheral with the no-arbitrage conditions of Gatheral and Jacquier ^{[5],[6]} and the wing bound of Lee ^[8]; the risk-neutral density literature ^[36] ^{[37][38]} and its practitioner treatments inform the distribution layer. The microstructure estimators are Kyle ^[10], Amihud ^[11], and the VPIN of Easley, López de Prado, and O'Hara ^[12], with the square-root impact law ^[15] of Almgren and collaborators ^[13], ^[15] behind the fill service. The variance risk premium follows Bakshi and Kapadia and Carr and Wu ^{[16],[17]}. None of this is novel mathematics; the contribution is implementing it correctly, arbitrage-free, tested, and behind a priced agent interface, which is precisely what has not existed for agents before.

On security, the transaction path is a conventional simulation-and-decode pattern; the distinguishing element is the EIP-712 signature analysis ^{[20],[21],[25]}, which addresses a vector that transaction-only tools cannot observe and that accounts for a large share of modern wallet losses ^[35].

10. The Build: Story, Process, and a User Scenario

This section steps back from the specification to tell how Quiver was built, what was learned along the way, and how the pieces are meant to be used together. The engineering record is part of the argument, because the decisions and the corrections are what a specification cannot show.

10.1 Starting from a gap

The project began with one observation. Agents had just gained the ability to pay for a service in-band over HTTP, and almost nothing worth paying for existed on the other side of that payment. We looked at what an autonomous trading or wallet agent needs at the moment it acts, and the list was concrete: the real probability that a price level is reached, the true cost of a size against real depth, a verdict on a signature before it is signed. None of it was available in a form an agent could call and trust. That gap set the direction. The decision that followed was to build computations a desk would recognise rather than another data relay, since a relay adds nothing an agent could not fetch for itself.

10.2 Choosing the set

Not every candidate survived. We held ideas to a floor: a real recurring need, an identifiable caller, and durability past the moment of a demo. Anything that depended on a fad, or that dressed a single API call as a product, was dropped. What remained clustered into the twenty-two services of Section 4. Options analytics was chosen as the widest moat, because the mathematics is unforgiving and the registry held no options service; a wrong volatility surface is worse than none, so getting it right is itself a barrier. Transaction safety was chosen for the opposite reason, that it is the highest-frequency and highest-stakes decision an agent makes, and the signature-drainer vector was covered by nobody. The rest fill out the surfaces an agent touches inside a loop: the tape, the book, the protocol, and the calendar. The most recent additions came from taking that floor seriously rather than from adding surface — the concentrated-liquidity desk earns its place by *refusing* to sell the optimiser the floor cannot justify, and reports the measurement instead; the authenticity scans exist because a number an agent trades on should be auditable back to the wallets and trades that produced it.

10.3 Holding a standard, and what it caught

The standard was set higher than a demo needs, on the principle that a financial service is judged by its worst number rather than its best. Early on, that principle paid for itself. The textbook probability formula, $N(d_2)$, was the first thing we reached for and the first thing we caught: on a skewed smile it is biased, and the bias was measured against a numerical density before the formula was replaced. The volatility surface took two attempts. A single global surface across all maturities was tried first and rejected by its own error gate, which was the right outcome; the literature prescribes fitting each expiry, and the second attempt did that and passed. The variance risk premium looked real until it was tested at the honest sample size, at which point it was not, and the service was made to say so.

The sharpest lessons came from pointing finished code at live markets. An illiquidity estimator that passed every synthetic test returned a value of order 10^9 on a real memecoin, because a live tape carries sub-cent dust the tests never imagined. A proxy detector that correctly handled the modern standard reported USD Coin, one of the most famous proxies in existence, as not a proxy, because USDC predates the standard and hides its implementation in an older slot. Each defect was invisible until the code met reality, and each one changed the implementation. The habit hardened into a rule that governs the whole project: build the thing, then point it at the market and believe the market over the code. Table 3 in Section 6 is the ledger of what that rule caught.

10.4 The record, dated

The project was built inside the OKX.AI Genesis window (July 2–28, 2026), and the dates below are taken from the on-chain record and the project's append-only engineering log, not from memory.

July 12. Agent #5152 is registered on X Layer under the name *Veritape* — at that point an authenticity-forensics service: wash-trade detection on token tapes, statistical audits of wallet track records, market-microstructure reads. The instinct was right (agents need numbers they can audit) but the frame was narrow.

July 17. The schedule had submissions closing that night, and the second version is shipped against that clock in one push: fourteen services, and four live defects in the volatility-surface fitter found and fixed the same day. That evening, OKX extends the Genesis window itself — the official schedule now runs to July 28. Eleven unexpected days changed the plan from "ship what exists under the gun" to "make the one existing entry worth judging": a single submission, levelled upward day after day, rather than a second one.

July 18. The day of self-audit. Before asking the platform to re-review anything, every service is audited for the failure class we had learned to fear most — the verifier that cannot fail — and the audit finds one more (an arbitrage check that scanned 14% of a smile and reported success). The catalogue is also curated: an off-theme service is withdrawn from the listing, and the original forensics services are kept because they fit the thesis that was forming.

July 19. The pivot, committed on-chain. The listing's description is rewritten to *"the verifiable risk brain for autonomous agents"* and nine deterministic risk engines are added — perp-gate, size-gate, exec-verify, options-risk, portfolio-gate, lp-risk, treasury-risk, event-vol, risk-attest — taking the catalogue from thirteen to twenty-two, with the cross-venue portfolio engine following as the twenty-second the same day. The reasons were two ground truths, not taste. First, the data-relay end of the agent-services market was commoditizing in real time: wrapper feeds were multiplying and a large share of observable payment volume was self-dealing, so "another API" had no durable value. Second, the durable gap was the one October 10, 2025 had exposed — agents and the risk products around them enforce *limits*, but nothing *computes* the numbers those limits are checked against, correctly and provably. Veritape's forensics did not die in the pivot; they became the authenticity services inside the risk brain. The same day, the first real settlement is proven on the second payment rail (a signed USDC payment on Base, verified on-chain).

July 20. The verifiability claim is made literal: an attestation schema is registered on Base EAS and the first batch of proof hashes is anchored under it. An adversarial review is run against our own live surface, simulating the platform's reviewer; it passes with notes, and all seven findings are fixed the same day. The service is published on the official MCP registry. The platform's human reviewer replies with a single request — the avatar should be a square — and the fix is on-chain within hours.

July 21. The listing goes live: all twenty-two services approved. What follows is the deepest day of the project: a systematic audit of every service against live data (about fourteen defects found and fixed, each locked by a test that fails on the pre-fix code), signed observation envelopes added to every live-data answer, the account-mode portfolio engine with the venue's own liquidation prices as an external cross-check, and — the part this paper leans on — the crash census, the population-scale replay from the venue's position archive, the pre-registered out-of-sample validation of Section 6, and a field test in which every service was purchased end-to-end with real money on both rails. A second adversarial review pass that day caught the project wearing the wrong envelope on one new feature (a deterministic "proof" over a live fetch); it was corrected, and the correction is owned in Section 5 rather than hidden.

The pattern across those ten days is the one this section keeps returning to: every date pairs a thing built with a thing caught — often caught in our own work, by our own checks, on the same day. That is not an embarrassment to disclose; it is the product working on itself.

10.5 A day in an agent's loop

The services are designed to be used together, and the clearest way to see the product is to follow one agent through a single decision. Consider a trading agent that has resolved to take a directional position and is about to size it and sign for it.

1. It calls `options-desk` and reads the risk-neutral distribution, checking whether the market already prices the move it expects and at what probability, so it is not paying for a move the options market considers likely.
2. It calls `poly-fill`, or reads the options expected move, to learn what its intended size will actually cost against live depth rather than against a midpoint that a thin book makes fictional.
3. It checks `macro-sentry` so it does not size into a scheduled print, and reads `tape-pulse` to see whether current flow is one-sided against it.
4. Having sized the trade, it constructs the approval and, before signing, passes the unsigned transaction and any permit signature to `calldata-x`. An unlimited approval to an unrecognised wallet returns a danger verdict and stops the loop; a routine approval to a known router passes.
5. On the next iteration it calls `loop-digest` with its cursor and reads only what changed, instead of re-fetching and re-paying for the entire state.

Each call costs half a cent to five cents, cheap enough to sit inside the loop, and each returns a number the agent can act on because the number can be checked. A market read, an execution cost, and a safety gate, all callable and all verifiable in one loop, is what Quiver is for.

11. Roadmap: Operating Quiver After the Hackathon

A hackathon submission that ends at the deadline is a demo. This section states what happens next, what would falsify the plan, and what is committed regardless of any competition result — because a service an agent is asked to depend on has to say

how long it intends to exist.

11.1 Standing commitments

Three things continue independent of any outcome. The endpoint stays live at its registered address, because agents discover Quiver through an on-chain registry entry and breaking that entry breaks every integration built against it. The availability record stays public and externally measured, including the outages: a reliability claim a reader cannot check is not a claim. And the repository stays open under its current licence with the research scripts intact, so every number in this document remains reproducible after the event that prompted it.

11.2 The single metric that governs the plan

Quiver's next milestone is deliberately not revenue, usage counts, or service breadth. It is **recurrence by callers who are not us**: distinct external agents calling more than once, because a second call is the only signal that an answer was worth its price in a real loop. Usage without recurrence measures curiosity; revenue without recurrence measures novelty. The desk currently instruments this directly — distinct payers and repeat rate per payer — and reports it honestly, including the fact that the great majority of calls to date are the operator's own disclosed quality-assurance traffic and are never counted as sales. Until recurrence is real, additional building is the wrong response, and the plan says so in advance so the temptation is on the record. Since the metric governs the plan, it is reported rather than described. As of this writing, and measured *on chain* rather than from our own counter, six payer addresses that are not ours have paid the service. Over the eight days to 27 July 2026 they sent **44 payments** totalling **0.575 USD** to the `payTo` advertised in every 402 challenge, at per-call amounts between 0.0096 and 0.016 — inside the published price band. **Four of the six paid more than once**, which is the metric as this section defines it. One returned across 2.55 days (25 to 27 July); another made twelve calls on 20 July and came back a week later. Half a dollar is not a business, and it is reported here because the metric governs the plan and the plan is worth nothing if the number is flattered.

An earlier draft of this section said *three* wallets, one or two calls each, and external recurrence *zero*. That was wrong twice over. The conclusion tested for a *third* call while the definition two paragraphs above says *more than once*; and the count itself came from an in-memory instrument that resets on every deploy — this service deployed eight times on 27 July alone. The chain does not forget and our own instrumentation did, in a document whose entire argument is that a number you cannot re-derive is a number you should not trust. The addresses are `0x1b010a9c...` (22 payments), `0xbc59eb75...` (12), `0xc385e2df...` (5), `0xcab2b9e3...` (3), `0x8d295ff5...` (1) and `0x86f10e00...` (1); a reader recomputes the whole figure from the USD transfer log on X Layer over blocks 65,711,861 to 66,403,061, which needs no cooperation from us. One of roughly seventy scan windows failed and was not retried, so treat 44 as a floor. That is against 568 calls from our own audit desk over the same instrumented window. One qualification belongs with that, and it is ours: those calls predate the acceptance-rule fix described in Section 11.5, and we have not verified on chain that their settlements landed. A facilitator response carrying a transaction hash does land — sampled 5/5 with receipt status `0x1` — but one without a hash does not, and on 27 July seven such responses produced zero transfers to the `payTo` address. So what is reported here is *calls*, not *revenue*: an external caller returned, and whether the money followed is something we can no longer assert from the facilitator's word alone. That is the number, and by the standard set two sentences ago it is the only number in this document that would justify building anything further.

11.3 Sequence

Near term (to roughly September 2026) — distribution over construction. The engine is ahead of its adoption, so the work is meeting agent builders where they already are rather than adding services. Quiver is already an MCP server carrying the risk-brain tools with a free fair-use tier, listed on the official MCP registry, and shipped with framework snippets for the common agent stacks; the near-term work is getting it in front of builders and publishing the correctness story in the terms that matter to them. The reliability item sequenced here is the custom domain and a second region, which together make redundancy possible and retire the disclosure in Section 8 — a registry update, planned deliberately rather than executed under deadline pressure.

Then (roughly September to December 2026) — prove the number is worth a real price. Sub-cent per-call pricing is right for discovery and wrong as a revenue engine: the dollar volume in agent payments sits at transactions above a dollar, not at a hundredth of one. The step is bundle and subscription pricing for high-loop callers, alongside an embeddable form of the engine so that a risk platform or treasury product can call these numbers inside its own product rather than sending its users elsewhere. The portfolio layer — cross-venue true exposure and nearest liquidation, whose self-check is reconciliation against chain truth — is the service most asked for and the one whose proof is strongest.

Then (into 2027) — attestation where it carries liability. The natural end state of a proof-carrying answer is an attestation a third party can consume in an audit: typed, signed, revocable, anchored on chain. That is a compliance artefact rather

than a mathematical advance, and its verifier is external — an attestation actually relied upon in a real decision or audit. Deeper venue coverage and any trustless-execution work (zero-knowledge or enclave-based) are gated behind a counterparty that genuinely requires them, not built speculatively.

11.4 Known gaps that are engineering work, not limits

Section 8 separates what cannot be fixed from what is merely unfinished. This is the unfinished list, stated as commitments with a definition of done, so that a reader can hold the plan to it rather than take a promise on faith. Item 4 has since shipped and is left in place, rewritten as what it became, because a commitments list that quietly deletes the commitments it met teaches a reader nothing about whether to believe the rest. Everything else here is still unfinished, and each is disclosed in the relevant service output today.

1. **Vanna-volga correction for one-touch barriers.** The service currently prices barriers under a single volatility and publishes the model-uncertainty span. The correction is a closed-form overhedge from three vanilla quotes already present in the fitted smile, so this is arithmetic we have not yet written rather than a modelling barrier. *Done means:* barrier prices carry a vanna-volga-adjusted figure alongside the single-volatility one, the difference is reported, and a self-check asserts the correction vanishes as the smile flattens.
2. **Live macro calendar with the curated table as fallback.** *Done means:* releases are fetched from primary sources, the response states which source answered and how old it is, and a failed fetch falls back to the transcribed table and says so rather than reporting a clear window it cannot certify.
3. **Deeper tape coverage where the venue allows it.** The sampled-feed limitation in Section 8 is imposed by the exchange endpoint, but the current implementation does not exhaust the pagination the endpoint does offer. *Done means:* the tape is walked as deep as the venue permits, and the density diagnostic reports coverage achieved against coverage available, so a caller can see the remaining gap instead of inferring it.
4. **A succinct proof a verifier that cannot run Node can check — shipped on 28 July 2026, and the three limits this entry used to list are closed rather than restated.** Re-execution already makes these answers verifiable: clone, check the code identity, re-run, compare. A proof system does not strengthen that and this document does not claim it does. What re-execution needs is a *runtime*, and a smart contract has none — so a contract consuming a Quiver number could previously check only a signature, which means trusting the signer rather than the arithmetic. That is now closed for the flagship computation, end to end. A caller adds "snark": true to a perp-gate request; the answer returns unchanged, carrying a retrieval URL, and a free GET /proof/<contentHash> returns a Plonk proof that the served liquidation price satisfies the liquidation identity for that exact position. QuiverProofRegistry at 0xd50A91E36673443749Ee22031cb2Ff09d4Bb8D60 on X Layer hands that proof to the verifier at 0x59F6Aa860eE0d26Db873f7c7015CE869170b3b25 and records what survived: one transaction accepting a proof **bought from the live endpoint** (0x50397d71...f368a, block 66,412,787, 468,459 gas, ProofAccepted) and one rejecting the same proof with the certified price moved a single grid step — one part in 10^9 (0x97502c78...4aac, block 66,412,794, 333,155 gas, ProofRejected). Read back from the chain by a script that did not deploy it, the registry holds 58329.113924051 against the 58329.11 the service sold. The liquidation identity $M + s \cdot q \cdot (P_{liq} - P_0) = q \cdot P_{liq} \cdot mnr$ compiles to a **1,301-constraint Plonk circuit** over BN254 with a residual bound derived from the position rather than fixed: $2|R| \leq \hat{q}(\text{SCALE} + \hat{mnr})$. The circuit constrains every input's range, forces side to exactly ± 1 , forces the maintenance rate below 1 and size non-zero, so the contract re-checks none of it — redundant require statements over facts a SNARK already enforces cost gas and buy the appearance of rigour.

The word for it is *succinct*, not *zero-knowledge*: the circuit has **zero private inputs**, every value it consumes is one the service already publishes, and it hides nothing. Reviewers who know the field check that count first, so this document states it rather than letting the phrase do work the circuit does not.

The three limits this entry carried are closed, and each is worth stating with what closed it. **First, the proof certified a fixed-point restatement rather than the engine's floating-point output** — over 1,600 scenarios the two agreed exactly 70.8% of the time and diverged by up to 1.9×10^{-4} , because the encoder truncated the maintenance rate onto the $1e-9$ grid while the engine kept the full double, so the two were computing about *different positions*. The service now snaps its inputs onto that grid before computing, which is why the worst observed divergence over 3,000 sampled positions is 5.53×10^{-10} with none above $1e-9$. Leverage is snapped too, because the engine derives margin from it and the derived value lands off-grid even when its ingredients do not. The snap uses `toFixed(9)` and not `Math.round(x·109)/109`, which is wrong above roughly 9×10^6 where the product passes 2^{53} — a price in the tens of thousands scaled by a billion sits exactly there. None of this moved a published content hash, because every value in the worked proof of Appendix C was already grid-exact.

Second, the proof bound six numbers to each other and not to anybody's account. Every input is public and unconstrained, so an adversary picks a liquidation price and solves the identity for the margin that makes it true — measured, and the circuit accepts a claimed price of 1 with a residual of exactly zero, because the identity genuinely holds for that pair. This is not a circuit defect; it is what the statement says. The remedy did not need new cryptography — verifying a signature *inside* the circuit costs millions of constraints — but it did need more care than this document originally proposed. Pairing the envelope signature with the proof leaves a gap you could drive a position through: a valid proof of one position beside a valid signature over *another*, each checking out alone, because the content hash is a SHA-256 over canonical JSON and nothing on chain can recompute it. So the service signs the **public signals themselves**, `keccak256(abi.encodePacked(uint256[8]))`, which is the same eight words the contract hashes from calldata; the contract recovers the signer and reports it. An unattested proof is still accepted and recorded as unattested — the arithmetic stands on its own or the exercise is pointless — and a signature over the right digest by the wrong key does not set the flag.

Third, and the one that would have been disqualifying: the circuit-specific phase of the Groth16 setup had a single participant, and it was our machine. Whoever holds that secret can forge a proof of a false liquidation price and it will verify on chain. Phase 1 was never the problem — it is byte-for-byte the public Hermez ceremony file. The remedy was to remove the per-circuit ceremony rather than to organise one, and the deployed verifier is therefore **Plonk over that same public reference string**, which costs 13% more gas and 22× the proving time than the Groth16 artifact we did not deploy. Measured: Groth16 proves in 32 ms warm and Plonk in 703 ms, median. That is the price of not asking anyone to trust a ceremony we ran alone, and it is paid in a place where it does not reach the caller.

Proving never touches the request path. It runs in a **separate process** — not merely off the synchronous path, because Node has one thread and 700 ms of unbroken WASM arithmetic froze the event loop for 506 ms, which production showed as a p95 of one full second for callers who had asked for no proof at all. Measured against the live service afterwards: p95 403 ms with a proof requested, 384 ms for ordinary calls while five proofs are building, against a 273 ms network floor from a residential connection. A caller who wants an on-chain proof spends it in a transaction that takes seconds to confirm, so 703 ms is invisible next to block time. What is *not* yet done, stated here rather than left to be discovered: **one of the twenty-two services has a circuit**, and proofs are held in memory, so a redeploy clears them and they are rebuilt on the next request. *Done means*: the catalogue's other deterministic identities carry circuits of their own, and a proof outlives the process that built it.

5. **A block range on the concentrated-liquidity replay.** 1p-desk replays real on-chain swaps, which is the most reproducible input any service here reads, and then reports the window as a span of days and a swap count rather than as the block range it actually walked. That makes an exactly-reproducible measurement into an approximately-reproducible one for no reason other than that the field was never published. *Done means*: the response names the first and last block of the replay and the node it read them from, so a reader re-runs the same walk and gets the same swaps, and the limitation entry for immutable public history in Section 8 moves from checkable-with-a-delay to pinned.
6. **Cross-region redundancy behind a custom domain.** The availability record and its single-region caveat are in Section 8. *Done means*: the registered endpoint resolves through a domain we control, a second region can serve it, and the published availability record shows the improvement rather than asserting it.
7. **Reproducible builds.** Two different hashes sit behind this heading, and only one of them has a runtime question; an earlier draft of this item conflated them and understated the result. The *code* hash is sha256 over the engine's source bytes and performs no arithmetic at all, so it is runtime-independent by construction rather than by promise — and that is checkable rather than asserted: feeding the same file list through `coreutils sha256sum`, outside JavaScript entirely, returns `q1-e1fa99d08887d6cc`, the identical string the service serves. `/build` publishes the selection and ordering rule alongside the hash precisely so that recomputation can be done in any language. The residual risk is the *content* hash, which covers computed floating-point results: basic IEEE-754 arithmetic is bit-identical across platforms, while transcendentals (`exp`, `log`, `pow`, `erf`) are stable within a V8 version. That too was measured rather than assumed — *size-gate*'s content hash is byte-identical on Windows and Linux on the same Node major. *Done means*: a locked toolchain and a published OS×runtime matrix, so the transcendental case is a table rather than a mechanism argument.

Two of these — the barrier correction and the calendar fetch — were within reach during the hackathon and were deliberately not shipped in its final days, because both live in the deterministic engine and would have changed the build hash that this document, its demonstration video (in the launch thread, linked from the repository README), and the buyer's independent audit all quote. The scope of that constraint is worth stating plainly, because it is narrower than the freeze makes it sound: the build hash is taken over `src/engine/**` and nothing else, so the adapters, the routing layer, the payment code and the served documentation can all change without moving it. Most of the roadmap above is therefore hash-free, and the two items named here are among the few that are not. That reasoning survived contact with a reviewer who then found four outright defects in

the same engine, and the build hash was changed to fix those (Section 11.5). The distinction we are drawing, and a reader is entitled to test it, is that a wrong answer is worth breaking a freeze for and a better answer is not: the six that remain unfinished above would each make the service more useful without making any current output less true. Item 4 is the case that tested the distinction rather than merely stating it: shipping the succinct proof required grid-snapping the service's inputs, which sits in the routing layer and not the engine, so the whole of it shipped without moving the hash at all.

11.5 Defects a live adversarial test found after the freeze, and how they were closed

The freeze described above held against improvements. It did not survive the defects. A reviewer was given live access to the service and asked to break it, and found four things that are worse than unfinished work — two of them in the one service whose entire purpose is proving something. At that point the argument for the freeze collapsed: an artefact that reproduces exactly is worth little if what it reproduces is wrong. All four were fixed on 26 July 2026; the published build moved from `q1-bce7e7bccb16ea1b` to `q1-6593c32ce84319b8`, and the worked proof in Appendix C was regenerated on the new build through a real paid call and re-verified through all four of its checks. It has moved again since, as further rounds of findings were closed — the identity this document cites throughout is the current one; each move regenerated the appendix rather than editing its numbers, because the code hash sits inside the content hash's preimage and a find-and-replace there would manufacture a proof that refutes itself. This section records what was wrong, because a reader assessing the engineering should see the defects and not only the patched result.

- 1. Merkle inclusion proofs were not verifiable by any on-chain verifier, and the response claimed they were.** `risk-attest` folded hashes as ASCII hex *strings* rather than as packed 32-byte words, so a Solidity verifier computes a different root and rejects a valid proof. The arithmetic was internally consistent and useless to the only consumer that matters. The published recipe was also imprecise enough that the reviewer brute-forced four conventions before proofs checked out; everywhere else in this document a verification recipe reproduced first try. *Closed:* the tree folds packed bytes, the response states plainly that it is *not* OpenZeppelin `MerkleProof`-compatible and why, and it ships the Solidity loop that does verify it. A test recomputes the served root through an independent byte-level implementation and pins that the old hex-text scheme produces a different one.
- 2. The Merkle tree did not domain-separate leaves from internal nodes.** Any internal node could be presented as a member leaf with a one-element proof and it verified against the root — a soundness break in the service whose entire purpose is proving membership. The engine's own soundness self-check could not catch it, because it only tried a random non-member, which fails for arithmetic reasons rather than structural ones. This was the sharpest instance in the whole document of the failure mode Section 3 warns about: a self-check that cannot fail in the direction that matters. *Closed:* leaves are tagged `0x00` and nodes `0x01`, and a second soundness check now presents a real internal node as a member leaf on every call — a check that can fail structurally, which the original could not.
- 3. `lp-risk` emitted an impossible number.** Its expected-divergence headline was a small-variance expansion, $-\sigma^2T/8$, which is unbounded; pushed to $\sigma^2T = 10.8$ it reported an expected impermanent loss of -135% , when V2 divergence loss is bounded in $(-100\%, 0]$. A verdict was then built on that figure, and the accompanying note said the approximation understates the loss where in fact it overstates it. The self-check validated the expansion only near $\sigma^2T = 0.01$, so it passed while the answer was nonsense. *Closed, and further than the fix required:* rather than refusing outside the expansion's range, the headline is now the exact lognormal expectation of $2\sqrt{r}/(1+r) - 1$, which is bounded by construction — the same input now returns -74.08% . The expansion is reported beside it with the gap between them stated, a boundedness check runs on the served value rather than on a fixed reference, and the fee breakeven is now solved against the exact expectation too, since leaving it on the expansion had the two numbers in one object disagreeing by 1.3 percentage points at 20% APR over thirty days.
- 4. An already-liquidated position was reported as a future event.** When the mark is beyond the liquidation price, `perp-gate` returned a negative distance-to-liquidation with no status flag, and `portfolio-gate` then ranked that leg as the book's nearest future liquidation and narrated it as the distance to first blood. The arithmetic was consistent; the semantics were wrong, and a caller acting on it would be de-risking a position that was already gone. *Closed:* `perp-gate` returns an explicit `positionStatus` of `BELOW_MAINTENANCE` with a note saying the threshold has been crossed rather than approached, and `portfolio-gate` ranks only live legs while reporting breached ones separately under `breachedLegs`, so they are disclosed rather than silently dropped from the ranking.

A fifth defect belongs here, found not by the reviewer but by us, while regenerating Figure 6 to check a claim its caption made. `chart-press` returned a provenance block asserting that the whole facts block was “computed from the SAME candle series drawn on the image” and therefore “cannot drift from the picture”. Two of the fields it named — the price and the 24-hour change — came from the token price feed instead, and in the capture that exposed it the served price and the price labelled on the image differed by ten basis points. The claim was checkable, invited the reader to check it, and was false; that combination

is worse than an unproven claim, because it spends credibility a reader extends to everything else in the response. Closed by naming the source of each field (`priceSource`, `change24hSource`), publishing the number the image actually labels (`lastDrawnCandleClose`) so the comparison is possible, and narrowing the guarantee to the two fields that earn it. Section 7 now prints the facts block beside the figure rather than asserting they agree.

Each fix ships with a test that fails on the pre-fix code, which was verified by reverting each defect in turn and confirming the corresponding tests go red — a test that passes both before and after a fix is evidence of nothing. The suite is 294 tests, 289 passing and five skipped because they require a live archive node, none failing. One of the five new tests passes both before and after its fix by design — it asserts that `high` and `low` still come from the drawn series — and is a guard against the fix breaking what was already right, not evidence for it. The cost of the change is stated plainly: the earlier build hash appears in this document’s demonstration video and in the buyer’s independent audit, and those now quote a superseded build. Both remain reproducible — the repository history contains the exact sources that hash to `q1-bce7e7bccb16ea1b` — but a reader checking `/build` against the video will see a difference, and the reason is this section. The Merkle root over the crash-study manifest moved twice, and the second time is the more instructive. First it moved because the defective tree had built it, with all fourteen per-file hashes unchanged. Then, checking that the manifest could actually be verified the way it instructs a reader to verify it — clone the repository, recompute each hash — one of the fourteen did not match. A query file sat in our working copy with a single stray carriage return while the repository stored it with a line feed, so the published hash was over 2,013 bytes and any reader following the instructions would compute it over 2,012 and get a different answer. `git status` cannot show this, because with `text=auto` it compares files after normalising line endings, so the divergence hides in precisely the place it does harm. For thirteen files the manifest verified; for the fourteenth it did not, which in a document about verifiability is a failure rather than a footnote. The file is normalised, `.gitattributes` now pins that extension explicitly instead of relying on content sniffing, and `research/RESEARCH_ANCHOR.md` records all three roots with the reason for each move.

11.6 A second review, and the reason its findings are worse than the first’s

The section above was written, and the fixes in it shipped, before a second reviewer was given the same brief plus live access to the service and the public repository. They confirmed a great deal by independent computation — the build hash from a fresh clone, the worked proof’s content hash and signature, five on-chain settlements at the block heights claimed, the EAS schema, all fourteen research-manifest hashes and the Merkle root through an implementation they wrote themselves, and the option mathematics to machine precision. They also found that **three of the five fixes above were fixes only on the code path the first reviewer had walked**. That finding is worth more than any individual defect, so it is recorded here in the same terms.

1. **The showcase soundness check still could not fail.** The check described above — the cure for “a self-check that cannot fail in the direction that matters” — presented an internal node with a *one-element* proof. A single sibling reaches the root only when the tree is exactly three layers deep, so for a batch of two, or of five or more, it folded halfway, compared that against the root, got a mismatch for arithmetic reasons, and reported success. Worse, it returned the same answer on a tree with no domain separation at all: it could not detect the defect it was written to detect. It now folds the full path and publishes how many siblings it used against how many the tree requires, so a reader can audit the check’s reach instead of trusting its verdict. Deleting the domain tags now turns it red at every batch size from three to sixteen.
2. **The boundedness check had an escape hatch written into it.** The `lp-risk` check that this document holds up as running “on the inputs actually served” read `pass: conc > 1 ? true : ...` — disabled precisely in the regime where the number breaks. At a concentration factor of 5 and a nine-fold move it served an impermanent loss of -200% , and a dollar loss of $-\$200,000$ against $\$100,000$ of capital, with the check green and the response asserting two lines above that such a loss can never exceed -100% . The hatch is gone; past -100% the linear amplification has not found a larger loss, it has stopped describing anything, so the headline reverts to the exact full-range figure and the raw linearisation is reported beside it.
3. **The already-liquidated leg was still narrated as live.** The status added above was set on the branch that solves a liquidation price, and not on the earlier return for a position already liquidatable at entry. `portfolio-gate` therefore saw no status, ranked that leg as the book’s nearest *future* liquidation at a 0% move, and described it as the distance to first blood — the exact sentence the fix was supposed to prevent, reached one branch over. Both branches now carry the status, and the aggregate invariant counts only legs that had an invariant rather than counting an unchecked leg as checked.
4. **Two defects the first review did not reach.** `risk-attest` accepted leaves that were not 32-byte hex; because the decoder truncates at the first invalid character, two different malformed inputs became the same leaf, a non-member verified against a real root using another leaf’s proof, and the duplicate counter reported none. And `size-gate` never validated the drawdown levels it was handed, so the ruin formula was evaluated outside its domain and returned 128 and 2187 as probabilities. Both are refused now.

The most serious finding was not in this list at all. `perp-gate` in symbol mode — the advertised path that resolves a ticker against a venue — attached its live-provenance block to the result *before* the content hash was taken. The hash therefore covered a field that re-running the engine on `proof.inputs` can never reproduce, so a caller who followed the instruction this document is built around got a mismatch, from an envelope whose own text says a mismatch means the response was altered. The verification instruction accused an honest answer, on the flagship service, in its live mode. It also shipped `deterministic: true` over a venue read with no timestamp, which is the very distinction Section 3.1 claims is enforced in code. Symbol mode now ships an observation envelope carrying `observedAtUtc`, which is what `portfolio-gate` had been doing correctly all along; calls that supply every input explicitly are unchanged and still reproduce their content hash exactly, as the worked proof in Appendix C does.

Fifteen tests cover these, and each was confirmed to fail against the pre-fix code by reverting the defect and watching it go red. One of them did not, at first: the test written for the Merkle soundness fix asserted that the check reported a pass, which the vacuous version also did, so the test certified nothing either — the same disease one level up, caught only because the revert was actually run rather than assumed. It now asserts the path length. The suite is 303 tests, 298 passing, five skipped for want of an archive node, none failing.

The same review produced a second, quieter list: places where the service made a claim it did not deliver, rather than a number that was wrong. They are the same defect class one layer out, and they are recorded because the first list would look better without them. `exec-verify`'s constant-product self-check carried an *absolute* tolerance proportional to the product of the pool reserves, which is quadratic in pool size while the output it certifies is linear: on a pool of 10^6 and 2×10^9 it permitted a residual worth roughly a million basis points of the honest output, against the five basis points at which the same engine calls a fill adverse. A check looser than the effect it is paired with certifies nothing; it is now a relative error on the invariant plus a reconstruction of the benchmark measured in output units, both at 10^{-12} . In the mode where the caller supplies the reference price there is no pool and no invariant, and the response used to report that non-check as a pass; it now reports it as not-run. `options-risk` took its scenario margin over the conventional seven price points, which finds the worst corner for a monotone book but can miss the interior worst case of a butterfly or a condor — so the number a caller holds capital against could sit below the true worst inside the service's own scan box. The requirement is now swept at 122 price points across the same box, the seven-point figure is still reported, and the gap between them is stated. And `chart-press`'s three reconciliation fields — the ones added earlier this same day so a reader could check the facts against the picture — were rounded to eight decimal places, which returns zero for any price below 5×10^{-9} . For a sub-nano token the comparison the response invites was arithmetically impossible. They now carry significant figures.

One generalisation in that review does not hold, and correcting it matters as much as accepting the rest. It reported that no service refuses to serve on a failed self-check. Two do: the martingale-transport bounds return `refused: true` when their exact identities fail, and the recovered density is withheld when its martingale residual or its integral does not check out — the behaviour Sections 5.3 and 5.8 describe. What was true is narrower and still worth saying plainly: *outside* those cases a failed check is disclosed in the envelope rather than converted into a refusal, so a caller who ignores `allSelfChecksPass` can act on an answer the service already knows is suspect. That is a deliberate choice — a hard gate on every check would refuse answers that are merely at the edge of a numerical tolerance, which is its own kind of wrong — but it is a weaker guarantee than “wrong answers do not ship”, and this document should describe it as the disclosure it is.

The honest summary of two rounds is this. The first review found defects; the fixes were real but stopped at the branch where each defect had been demonstrated, and two of the replacement verifiers could not fail. A reader deciding how much to trust the engineering here should weigh that pattern more heavily than the defect count, because it is the part that predicts what is still undiscovered. What can be said in the other direction is that both rounds are published, including the parts where the second reviewer caught the first round's repairs, and that everything an outsider can check independently — hashes, signatures, settlements, the manifest, the mathematics — was checked by someone with no stake in the answer and held.

One further note belongs here because a reviewer found it independently and read it, reasonably, as a hole in the revenue story: **the nine deterministic risk engines are also reachable free over the MCP endpoint**, under a fair-use daily quota. That is a deliberate distribution choice — the cheapest way to put the engines in front of the agent frameworks where builders already work — and not an oversight, but this document should have said so where it discusses pricing rather than leaving it to be discovered.

11.7 A third round, and the pattern that predicts what is still there

Two more reviews were run on 27 July — one with the document alone, one with live access to the service and the repository — alongside a sweep of our own. Their findings are recorded here on the same terms as the two rounds above, and the build moved again to close them.

The money path was charging for answers the engine had already flagged. Under overflow inputs two engines returned a success envelope whose own self-check reported failure, and billing keyed on the success flag alone, so a caller paid for a result the engine did not stand behind. What is right here and was preserved: the engines sanitise non-finite arithmetic to null rather than emitting it, and the self-checks correctly detected the violation. Only the billing rule was wrong, and it is now gated on the checks — *gated on billing, not on delivery*. The answer is still served with its failed check disclosed, because converting every failed check into a refusal would withhold answers that are merely at the edge of a numerical tolerance. The asymmetry is what makes that safe in one direction only: not charging for a borderline answer costs a fraction of a cent, and charging for one we refused to stand behind takes a buyer's money for it.

The build hash did not cover the build. Every envelope carries the words “one hash over ALL engine sources”. The hash was computed with a non-recursive directory read, so `src/engine/chart/` — the rendering path and the indicator library, 42,442 bytes that `chart-press` imports directly — was never hashed. That code could change while the published build identity stood still, in the one field a caller uses to decide which code produced their answer. The walk is recursive now and the hash covers 38 files rather than 35. The defect was the claim, not the arithmetic, which is the more uncomfortable of the two to have shipped.

A fix held only on the branch a reviewer had walked, for the fourth time. Section 11.6 records live provenance being sealed inside a deterministic proof envelope, and records it as closed. It was closed in the paid path and in one of the two handlers next to it — and left open on the free endpoint, which is the one a builder is most likely to try first. A reviewer ran the verification recipe printed inside the envelope and got a mismatch, from an envelope whose own text says a mismatch means the response was altered. Repairing call sites one at a time had now failed four times, so the rule was moved into the envelope constructor: a result carrying live provenance is not a proof and is routed to the envelope that can carry it honestly. **A reader deciding how much to trust this engineering should weigh that recurrence more heavily than any individual defect**, because it is the part that predicts what is still undiscovered.

Smaller, and each one a claim the service did not deliver. The aggregate `allSelfChecksPass` read an explicitly skipped check as a passing one, so a response where nothing was checked published `true` — while the engine underneath it refused, in its own comment, to report an un-run check as a pass. The nearest-liquidation headline closed with “that is the whole book's real distance to first blood” regardless of margin mode, which is the one reading a caller must not take on a cross-margined account, and it is now conditional on what the caller actually declared. A perpetual call naming an unsupported venue was served with an error string welded into the signed result instead of being refused. The 400 note on an unparseable body was generic while Table 9 described it as self-teaching, so the hint was added to that path rather than the claim narrowed to fit. Impermanent loss was labelled as loss-versus-rebalancing throughout, and Section 5.14 now separates them. Equation (12) omitted the reporting scale that makes its own figures reproducible.

Every fix in this section ships with a test that fails on the pre-fix code, and that was established by running a scripted revert for each rather than by reasoning about it — a discipline this document has recorded failing twice before. It caught something again here: one revert produced a file that would not parse, so the test went red for the wrong reason and the verdict had to be withdrawn and the revert rewritten. At the close of that round the suite stood at 338 tests, 333 passing, five skipped for want of an archive node, none failing; later rounds have taken it to 386 and 381, the last of them adding the succinct-proof binding tests and a guard on event-loop lag.

11.8 Research programme

The crash study in Section 6 is the template, not a one-off. Its discipline — hypotheses and thresholds fixed, in published files, before the data is touched, a temporal cutoff, out-of-sample events the model has never seen, and publication of the runs that failed — carries forward to the questions that come next: whether the correlated-stress betas remain transportable as market structure changes, whether the flag's dose-response holds on venues outside the calibration set, and whether the variance premium becomes significant on a longer sample or should be retired. Each will be pre-registered the same way, and each will be published whether or not it supports the product. A validation record that only ever confirms the thing being sold is not a validation record.

11.9 What the proof changes, and what it does not — the year after

The registry described in 11.4 covers **one computation of twenty-two**. Saying so first is not modesty; it is the only way the rest of this section can be read as a plan rather than a boast. The ladder a buyer actually climbs has four rungs, and it is worth naming which services stand on which.

Rung	What the buyer must trust	Services today
To — re-runnable	that they can run Node	all 22 deterministic answers
T1 — signed	our key, for provenance only	all 22
T2 — proven on chain	nothing; a contract checks the arithmetic	1 — perp-gate liquidation
T3 — attested input	that the venue reported what we say it did	0

Table 11 — the four rungs, and how many services stand on each today.

The distance between T2's one and T1's twenty-two is engineering with a known shape. Six engines rest on a closed-form identity a circuit can state: *size-gate* (the Kelly first-order condition, smaller than the liquidation circuit and therefore the one to do next, precisely because doing it easily is what turns *we built a circuit* into *we build circuits*), *portfolio-gate* (harder, because the first leg to liquidate is a minimum over legs and proving a minimum means proving no other leg qualifies), *1p-risk* and *treasury-risk* (closed form, and the treasury residual bound is already published), *exec-verify* (trivial arithmetic whose honest difficulty is that *fair price* is an input, so the proof says less than it appears to until the input is itself attested), and *options-risk* last, because Black-76 greeks need *exp* and *erf* in-circuit, which is where this stops being arithmetic and becomes a research project. A circuit whose constraint count pushes proving past a few seconds, or whose identity needs a value the service does not publish, should be abandoned and said so publicly, leaving that service at T1 — a circuit that proves *nearly* the served answer is worse than none, which is the lesson 11.4 already paid for once.

Two structural pieces follow. Proofs are held in memory today, so a redeploy clears them and the next request rebuilds; content-addressed storage makes a proof outlive the process that built it, which anything on chain will eventually depend on. And every proof presently costs its own transaction, which no agent polling risk in a loop will accept: folding *n* proofs into one aggregate replaces *risk-attest*'s Merkle root of *claims* with a single proof of *arithmetic*, and it is worth being exact that a Merkle root establishes inclusion and nothing else — this document has never said otherwise, and aggregation closes precisely that gap.

The harder problem is not more circuits. Everything above proves arithmetic *over inputs*. It says nothing about whether the inputs are true, and a proof that a liquidation price follows from a mark of 64,000 is worthless if the mark was 61,000. Where the caller supplies the inputs this is correct and sufficient — they chose them. For live-market answers it is the whole question, and it is why those ship as observations rather than proofs (Section 5.19). Three approaches exist and the cheapest may be enough: carrying a venue's own signature through where one exists, which is partial and honest; a TEE attestation binding *this binary to this response at this time*, which converts trust-in-Quiver into trust-in-a-silicon-vendor — a genuine improvement, not a proof, and the history of enclave side-channel breaks is not short; and zkTLS, which is the honest end state and the least mature. A real possible outcome, stated now so it cannot be quietly redefined later, is that for a public API "*go and fetch it yourself*" dominates every cryptographic answer. If three months of work does not beat that, the finding is the deliverable.

One speed note belongs here rather than in a footnote. Groth16 proves in 32 ms against Plonk's 703 ms and verifies for 13% less gas; with native proving it would be fast enough to put back on the request path and the proof would become free from the caller's perspective. The blocker is social, not technical — a per-circuit ceremony needs participants who do not know each other, and that cost multiplies with every circuit added above. If seven independent participants cannot be found, Plonk stays permanently and so does the 703 ms, which is a better outcome than a ceremony staffed by our friends.

11.10 What would falsify this plan

Stating the kill signal is part of the honesty this document argues for. If, after distribution work is genuinely done rather than merely intended, no external agents return for a second call, the correct conclusion is that this capability is not yet worth paying for in a loop — and the response is to stop building services and either re-target the buyer or stop. Two further signals would force a change rather than more construction: a measured leak reappearing after the settlement fix at volume, which would mean the acceptance rule is still wrong in a way one day of data could not reveal; and an availability record that stops improving despite the roadmap's redundancy work, which would mean the hosting choice, not the code, is the product's limit. Each is checkable by a reader from public artefacts, which is the point.

12. Conclusion

Quiver is a live, paid Agentic Service Provider that gives autonomous agents twenty-two financial and security computations they can call over x402 and discover under ERC-8004. The services implement methods a desk would recognise, and they are held to a standard that this document has tried to make legible rather than merely assert: correctness proven by 386 model-

free tests, claims grounded against live venues, a risk flag validated pre-registered and out-of-sample against real crashes it had never seen (relative risks of $14.3\times$ and $13.3\times$ on a flag that fires on 41.6% and 43.8% of accounts, and which our own ablation reduces to raw distance-to-liquidation — the venue's published number rather than one we computed, so the study validates the quantity and not our arithmetic on it, Section 6), and every service purchased end-to-end with real money on both payment rails before this document claimed it worked. Underneath all of it sits a stated refusal to emit a number the data does not support. That refusal is not a limitation of the product; it is the product. In a market where an agent must trust a computation it cannot supervise, a service that corrects its own $N(d_2)$, that catches its own Amihud explosion, and that reports a variance premium as insignificant when it is, is worth more than one that returns a confident number and hopes. The mathematics here is public and old. The discipline of applying it honestly, at the point where an agent is about to move money, is the part that was missing.

Appendix A • API Reference

All services are reached at <https://quiver-production-c3a8.up.railway.app>. Each paid route answers an unauthenticated request with the `402` challenge; a request carrying a valid payment authorisation receives the JSON result. Inputs are passed as query parameters on a GET or as a JSON body on a POST.

Endpoint	Principal inputs	Headline output fields
POST /api/options-desk	currency (BTC/ETH/SOL), focus	impliedView, distribution, gex, vrpModel, crossMarket
POST /api/calldata-x	data, to, from, chain, or typedData	alert, verdict, simulation, spenderReputation, targetContract
POST /api/chart-press	chain, address, interval, chartType, indicators, drawings	hostedUrl, imageBase64, facts
POST /api/tape-pulse	chain, address	read, microstructure, dataWindow
POST /api/poly-fill	market, side, usd, maxSlippagePct	fill, marketImpact, bookDepth, bookWalk
POST /api/poly-desk	wallet	positions, unrealizedPnl, movers
POST /api/updown-pulse	coin (BTC/ETH)	marketImplied, window, remainingRisk, edgeS-tance
POST /api/protocol-pulse	protocol	riskLevel, riskFlags, tvl, incidents
POST /api/macro-sentry	hours (+ optional spot, atmIvPct for the implied move)	events, nextHighImpact
POST /api/loop-digest	wallet, chain, cursor	newFills, pnlDrift, cursor
POST /api/perp-gate	symbol (or entryPrice), side, size or notional, margin or leverage, optional venue	liquidationPrice, moveToLiquidationPct, marginTier, funding, proof
POST /api/portfolio-gate	positions (legs) or account, optional betaTier, shockScenariosPct	netExposure, firstLegToLiquidate, correlatedStress, proof
POST /api/size-gate	winProb + winLossRatio (or expectedReturn + volatility), bankroll, kellyFraction	recommendedSize, fullKellyFraction, expectedLogGrowth, riskOfRuin, proof
POST /api/exec-verify	amountIn, amountOutRealized, reserveIn/reserveOut or fairPrice, feeTier	slippageBps, priceImpactBps, verdict, proof
POST /api/options-risk	positions (legs), forward, optional scanRangePct, volShiftVolPts	netGreeks, scenarioGrid, worstCase, proof
POST /api/lp-risk	priceRatio or volatility + horizonPeriods, feeAprPct, concentrationFactor, capitalUsd	impermanentLossPct, breakEvenFeeApr, netVsHodl, proof
POST /api/treasury-risk	positions (holdings), concentrationLimitPct, depegScenarios	concentration, depegImpact, flags, proof
POST /api/event-vol	spot, atmIvPct, daysToEvent, optional thresholdsPct, ivBeforePct	impliedMove, thresholdProbabilities, ivCrush, proof
POST /api/risk-attest	items (envelopes) or contentHashes	merkleRoot, inclusionProofs, anchorCalldata, proof
POST /api/token-scan	chain, address	manipulationRisk, evidence, verdict
POST /api/wallet-audit	chain, address	trackRecord, significance, washCrossCheck, grade
POST /api/lp-desk	pool, chain, days, widthPct, capital	feesEarned, impermanentLoss, netVsHodl, verdict

Table 8 — endpoints and principal inputs. Outputs are structured JSON; the key fields are named in Section 4.

The challenge an unpaid request receives

A caller does not need this document to learn how to call a service, because the 402 carries that information itself. Every unpaid request returns both payment rails and, inside each, a JSON Schema for the request body — including the *alternative* ways to satisfy it, which is the part a flat parameter list cannot express. The abridged reply below is what POST /api/perp-gate answers with today.

```

{
  "x402Version": 2,
  "accepts": [{
    "scheme": "exact", "network": "eip155:8453",
    "maxAmountRequired": "10000", // 0.01 USDC, 6 decimals
    "asset": "0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913",
    "payTo": "0x65bb932d9987f1d1a98b8942a3fa98cb28ec073b",
    "maxTimeoutSeconds": 300,
    "resource": { "url": ".../api/perp-gate", "description": "...", "mimeType": "application/json" },
    "extra": { "name": "USD Coin", "version": "2" }, // EIP-712 domain for the authorisation
    "outputSchema": { "input": { "type": "http", "method": "POST", "bodyType": "json", "body": {
      "type": "object",
      "properties": { "symbol": {...}, "side": {...}, "entryPrice": {...}, "size": {...}, ... },
      "allof": [
        { "anyOf": [{ "required": ["entryPrice"] }, { "required": ["symbol"] } ],
          "description": "price: entryPrice OR symbol (defaults to live mark)" },
        { "anyOf": [{ "required": ["size"] }, { "required": ["notional"] } ],
          "description": "size: size (base units) OR notional" },
        { "anyOf": [{ "required": ["margin"] }, { "required": ["leverage"] } ],
          "description": "collateral: margin OR leverage" },
        { "anyOf": [{ "required": ["maintMarginRate"] }, { "required": ["maxLeverage"] }, { "required": ["symbol"] } ],
          "description": "maintenance: maintMarginRate OR maxLeverage OR symbol" }
      ] } } }
    }, { "network": "eip155:196", "asset": "0x779Ded...3736", "extra": { "name": "USDT", "version": "2" }, ... } ],
  "error": "Payment required"
}

```

The `allof` of any `of` groups is the load-bearing detail. `perp-gate` needs a price, a size, collateral, and a maintenance rate, and each of those can arrive in more than one way — a symbol resolves three of them from live venue data at once. An agent reading the schema can construct a valid call without a human in the loop and without guessing, which is the difference between a machine-readable price tag and a machine-readable *interface*. The `extra` block is the EIP-712 domain the payment authorisation must be signed under, so the payer needs nothing from us beyond this response.

Failure modes, and which of them cost money

Only one rule matters here and it is worth stating before the table: **a request the engine refuses is never settled**. When a result carries `ok: false` the payment authorisation is not submitted, and the reply carries a `PAYMENT-RESPONSE` receipt reading `{ success: false, status: "not_charged", reason: "input rejected by engine — no settlement" }`. A caller can therefore probe a service, get a refusal explaining what was missing, and pay nothing for it. This was not always true — Section 6.4 records the leak that made it true and the buyer who measured it.

Status	Body	Cause	Charged?
200	the result, with its proof or observation envelope	Valid payment, valid input.	Yes — receipt carries the settlement transaction hash
200	ok: false with a reason and, where relevant, a coverage note	The data does not support an answer: a venue returned nothing, a window held too few observations, a smile could not be fitted. The service refuses rather than interpolating.	No — receipt reads not_charged
400	{ error: "bad_input", note: ... }	Input fails the schema, or the body is not parseable JSON at all. An unparseable body used to fall through to the 500 handler and echo the raw parser message, reporting a caller error as a fault inside the service; it is a 400 now. The note is self-teaching on <i>both</i> paths: it names what was missing, or that the body would not parse, and then the service, its purpose, and every alternative required-field group, so the caller can fix the request from the error alone. An earlier version of this row claimed that of both paths when it was true of only one — the parse case returned a generic sentence about invalid JSON and stopped there. The route is known even when the body is not, so the hint was added to that path rather than the claim narrowed to fit. The parse case also returns parserDetail, the runtime's own message naming the character position that failed, which is the one thing a caller cannot work out from its own request; it discloses which JSON parser is in use and nothing about this service's data or state.	No
402	the challenge above	No payment header — or a malformed, expired, or forged one. An invalid authorisation is re-challenged rather than answered, so a forged header cannot buy a computation.	No
404	{ error: "not_found", note: "no route ...", index: "/", docs: "/paper" }	Unknown service. The reply names the route it did not find and points at the index and this document.	No
429	{ error: "rate_limited", note: "max 60 requests/min per IP" }	Sixty requests per minute per IP, applied before the payment surface, so it costs an abusive caller nothing and costs us nothing either.	No
413	{ error: "bad_input", note: ... }	Body over the 16 kb limit, on a route with no payment gate in front of it. On a <i>paid</i> route an oversized body is answered with the 402 challenge instead, because the payment gate is evaluated before the body is read — verified live at 40 kb. An earlier version of this row claimed the 413 for paid routes too, which was written from the handler rather than from a test.	No
500	{ error: "engine_error", detail: ... }	An unhandled fault inside a service. Distinguished from a refusal on purpose: a refusal is a correct answer about missing data, this is a bug.	No
502	{ error: "facilitator_unreachable", detail: ... }	The payment facilitator did not answer verification or settlement. Reported as an upstream failure rather than folded into a generic 500, because the caller's next action differs: retry, or switch rails.	No

Table 9 — every status the API returns, what causes it, and whether it costs the caller anything. Captured against the live service.

The free MCP endpoint at `POST /mcp meters tool calls` only — `initialize` and `tools/list` are unmetered so a client can always discover the surface — at three hundred calls per day per IP. Exceeding it returns a JSON-RPC error `-32000` whose message names the quota and points at the paid `x402` route with both rails, rather than failing silently or degrading the answer.

Appendix B • Reproducibility

The figures in this document divide into three kinds, and only one of them involves a venue: Figures 2 to 5 are deterministic mathematics that reproduce from the equations alone, the crash study reproduces from a public requester-pays archive, and the captured service responses reproduce against live venues only in the sense that the method repeats — a live number cannot be reproduced once the market has moved. Appendix C carries the artifacts that reproduce exactly. Because reproduction of a live number requires the live market to be in a comparable state, the figures should be read as representative of the method rather than as constants.

- **Options distribution and martingale check (Section 7).** Call `options-desk` with `currency=BTC` and read `impliedView[0].distribution`. The `selfCheck.meanVsForwardPct` field is the martingale residual of equation (9) and should be near zero on any well-formed chain.
- **Probability correction (Section 5.3).** The `probabilityLadder` reports both the corrected probability and the field `skew-CorrectionPts`, the size of the smile-slope term of equation (8) at each level. Independently, fetch the Deribit ticker greeks for the same instruments and confirm the delta matches to three or four decimals.
- **Signature and proxy safety (Section 7).** Call `calldata-x` with an `approve(spender, MAX_UINT)` `calldata`, to a token, and from any funded address; vary the spender between a known router and a wallet to see the verdict flip. Re-run `eth_simulateV1` with the same inputs on any node to confirm the emitted Transfer and Approval logs.
- **Tape microstructure (Section 5.5).** Call `tape-pulse` on a liquid token; the `tape.medianGapSeconds` field indicates whether the read rests on a dense or a sampled feed, and Kyle's lambda carries its own R-squared and confidence.
- **Market impact (Section 7).** Call `poly-fill` on a live market; `marketImpact.fitR2` is the measured quality of the square-root fit, and `bookDepth.imbalance` is computed from the same book that the walk consumes.

Everything referenced in this appendix lives in the public repository: <https://github.com/Tristan-tech-ai/Quiver> — the full engine source, the test suite, and the research artifacts under `research/` (the crash study, the validation scripts, and the field-test harnesses with their raw results). The invariant test suite is self-contained and deterministic; it runs with the project's standard test command and requires no network access, so 381 of the 386 model-free properties of Section 6 can be verified offline, and five are live-archive integration tests skipped unless an archive RPC is configured. The market-validation numbers of Section 6 reproduce from public data with the scripts in `research/reservoir-data/` (episode detection, beta measurement, and the two pre-registered event tests; about \$1 of requester-pays S3 access — the folder's README gives the exact fetch commands).

Appendix C • Checkable Artifacts

A reviewer of an earlier draft of this document made an observation that was both correct and embarrassing: a paper whose thesis is *do not trust the number, re-derive it* shipped almost nothing a reader could re-derive. Every figure in the preceding sections is an assertion by us. This appendix exists to fix that. Each row below is an artifact that lives outside this document, on a public chain or a public endpoint, together with the command that checks it. None of them requires our cooperation, and none can be edited by us after the fact. If any of them fails to resolve as stated, treat the corresponding claim in this document as unproven.

What	Artifact	Where it lives
ERC-8004 agent listing registered as <i>Veritape</i> and renamed to Quiver on 19 July, so the on-chain name may differ from this title page	0x25326e5504e5213b1a8f9dce81818157a3995ea49b17e8f1d0987fa0f1b78e7d	X Layer (eip155:196), block 65,692,173 (0x3ea620d) www.oklink.com/xlayer/tx/0x25326e55...
x402 settlement, X Layer rail (USDf0)	0x68444c5462bddecd1a587b762d3f7b8f2f4bce2724fa3bc04dcacdadd7cba1af	X Layer (eip155:196), block 65,848,752 (0x3ecc5b0) www.oklink.com/xlayer/tx/0x68444c54...
x402 settlement, Base rail (USDC)	0x429a1efe31a82078bb61bc781435c09038adb3dd07d0d7641a2496a5fbb42483	Base (eip155:8453), block 48,956,716 (0x2eb052c) basescan.org/tx/0x429a1efe...
Second Base settlement, different service	0x88a85f49b4d51e28eab713a436fd6112252ee0ee465e73bb01e23791d8c6a3a6	Base (eip155:8453), block 48,993,708 (0x2eb95ac) basescan.org/tx/0x88a85f49...
A real paid settlement for this exact service and these exact inputs. The exhibit below was regenerated later, on a newer build, so this transaction evidences the payment rail rather than that particular envelope	0xa07957667cf53eb52814c4c4488027da2596f109c90f8d68f323eb60eec7e4b6	X Layer (eip155:196), block 66,383,878 (0x3f4f006), 27 July 2026 www.oklink.com/xlayer/tx/0xa0795766... The proof below was regenerated on the current build, so the settlement that paid for it moved with it. The earlier capture was paid on the Base rail (0xedfc0322..., still on chain); this one is on X Layer, which is the rail the audit desk uses. Both are real; only the second one pays for the artifact printed here.
EAS attestation schema for risk-attest	0x59a8587b287d3f13776dccbe49e19d2e887f90b5e16650464b07e613d89287e0 schema string: bytes32 merkleRoot, uint256 itemCount, string engineVersion	Base, EAS SchemaRegistry 0x4200...0020
Merkle root anchored on-chain under that schema	0x01ffd2f9934a2c7f7df119e1e2043231fef59be21011cb3184c956ea479a1b1 attestation 0x69d42632..., Base, 20 July 2026, itemCount 2	committed under the schema above by the wallet that owns the agent identity. It batches two computations, not a day's worth — an earlier draft of this row described it as “a day's attestation”, which overstated a two-item proof of the mechanism. It was also produced by the pre-fix tree, so it records what was anchored on that date and is not a root the current engine reproduces; re-batching the same items today gives a different one, for the reason in Section 11.5

Research artifacts behind the crash study, hashed file by file	merkleRoot 0xd376b71f94d54967325fbddc60b5d35d478884f1a70e63e0844532c24642c784 over 14 files	manifest published at research/RESEARCH_ANCHOR.md; recompute each file's sha256 from a clone and re-derive the root through risk-attest. This root supersedes two earlier ones: 0x5f9da998..., produced by the defective tree of Section 11.5, and 0x79965fac..., which was correct as a tree but was built over a working copy of one query file carrying a stray carriage return, so it did not reproduce from a clone — found by running the manifest's own instructions against a fresh checkout. Both are recorded in research/RESEARCH_ANCHOR.md. The per-file hashes of the other thirteen files never changed and are what actually pin the research. An on-chain attestation under the schema above is prepared but not yet submitted, so this establishes that the bytes are unchanged relative to a git-times-tamped manifest and nothing stronger.
Proof registry that verifies a Quiver answer's arithmetic on chain, without trusting the seller	0xd50A91E36673443749Ee22031cb2Ff09d4Bb8D60 verifier 0x59F6Aa860eE0d26Db873f7c7015CE869170b3b25	X Layer (eip155:196), deployed 28 July 2026 okx.com/web3/explorer/xlayer/... the registry holds the verifier and attester addresses as immutables; read them back with verifier() and quiverAttestor() rather than taking this row's word for it
A proof bought from the live endpoint , accepted on chain	0x50397d713b96414800fef2dc6c2b4b8a48bd89d7f793683df9deddfbe73f368a	X Layer (eip155:196), block 66,412,787, 468,459 gas, event ProofAccepted okx.com/web3/explorer/xlayer/... the registry stores 58329.113924051 for this position against the 58329.11 the service sold; liquidationPrice(0x25669da5...) returns it
The same proof with the certified price moved one grid step, rejected on chain	0x97502c78e61958a9a1013a257bf281c665684e214d6474c41444eb0294cb4aac	X Layer (eip155:196), block 66,412,794, 333,155 gas, event ProofRejected okx.com/web3/explorer/xlayer/... one part in 10 ⁹ is enough to invalidate it. A verifier that cannot refuse is not a verifier, so the refusal is published beside the acceptance
Build identity of the engine that produced every proof in this document	q1-e1fa99d08887d6cc	served live at /build; rebuildable from the public repository. Replaced q1-bce7e7bccb16ea1b on 26 July 2026 to close four defects (Section 11.5); the earlier build's sources remain in the repository history and still hash to it
Independent availability record	status page, plus the same data as JSON	https://cgn9npwmm0.execute-api.us-east-1.amazonaws.com/.../?format=json — hosted off this service, on AWS

Table 10 — artifacts a reader can verify without contacting the authors. Verified as resolving at the time of writing; a reader should expect them to resolve identically. Each transaction hash, its success status, and the schema were verified on 26 July 2026 by direct RPC call to each chain and, for the schema, by getSchema against the Base SchemaRegistry. Block heights are given in hex as returned by eth_getTransactionReceipt alongside the decimal, be-

cause an earlier draft of this table converted them by hand and got all four wrong — an error found by a reviewer who ran the checks this appendix invites, which is the appendix working as intended and the authors not.

A worked proof, end to end, from inputs a reader chooses

The following is a complete deterministic answer, captured on 27 July 2026 from the live service on build q1-e1fa99d08887d6cc and, separately, regenerated offline from the repository alone — the two agree byte for byte, content hash included, across a Linux container and a Windows laptop. That agreement is the exhibit rather than a footnote to it: the capture shows what the service served, and the offline regeneration shows the number is right without the service's cooperation. All four checks below were re-verified before it was printed here. It uses explicit inputs rather than a live market price, so it is re-runnable indefinitely rather than only at the moment of capture. Every field a verifier needs is *named* here; the content-hash check additionally needs the full result object, which the live response returns and this page abridges for readability.

REQUEST — POST /API/PERP-GATE

```
{ "side": "long", "entryPrice": 64000, "size": 1, "leverage": 10, "maintMarginRate": 0.0125 }
```

ANSWER, AND THE PROOF IT CARRIES

```
liquidationPrice      58329.11
moveToLiquidationPct  8.861
positionStatus        ABOVE_MAINTENANCE

proof.engine          perp-gate
proof.codeHash        q1-e1fa99d08887d6cc
proof.deterministic    true
proof.inputs          {"side":"long","entryPrice":64000,"size":1,"leverage":10,"maintMarginRate":0.0125}
proof.selfChecks[0]   account_value(P_liq) == maintenance_margin(P_liq)
                      residual 2.05e-12, tolerance 0.064, pass true
proof.contentHash     8575ce5ae5bfae9cdfdfc604250f8032e4ba85fb33560386586b7538d0ab0960
proof.signature.signer 0x946324E0E5d7D77206731E35Ef4044a383e2a8C2
proof.signature       0xcabfb1954b39b306c041d546fb151582cc7d31dce403fbd0a056d7be824188fb274d2f9af2a1aa01469c2d96998c871f3a36b4ac8a6762c
                      36f0b9d2e45c824111b
```

Four independent checks follow from that block, and they can be run in any order. One caveat first, because the exhibit was chosen for reproducibility rather than for difficulty: the self-check shown here substitutes the solved liquidation price back into the condition it was solved from, so it proves the algebra and the code path, and proves nothing about whether the maintenance-margin tier, the account-value definition, or the venue's real margin regime are the right ones. The harder self-checks in the catalogue — six finite-difference portfolio greeks in options-risk, the martingale residual on the recovered density, martingale-transport feasibility as an independent calendar-arbitrage certificate — carry more information and less convenience, since none of them can be recomputed by a reader with a calculator. Section 5 gives each of them in full.

1. **Does the build identity match?** `curl -s https://quiver-production-c3a8.up.railway.app/build` must report q1-e1fa99d08887d6cc. Cloning the public repository and hashing the engine sources must give the same string. If it does not, the code that answered is not the code that is published.
2. **Does the arithmetic hold?** The liquidation price for a long at entry 64,000 with 10× leverage and a 1.25% maintenance rate follows from equation (18): $(P_0 - M/q) / (1 - mmr)$ with margin $M = 6,400$, giving $(64,000 - 6,400) / 0.9875 = 58329.11$. A reader with a calculator can confirm the served number without running anything.
3. **Does the content hash reproduce?** sha256 over the key-sorted canonical JSON of {engine, codeHash, inputs, result}, where result is the response with its proof key removed, must equal 8575ce5ae5bfae9cdfdfc604250f8032e4ba85fb33560386586b7538d0ab0960. The response carries its own recipe for this in proof.verifyContentHash, and the SDK exposes it as verify().
4. **Does the signature recover?** ethers.verifyMessage(contentHash, signature), where contentHash is passed as the 64-character lowercase hex string exactly as printed above (no 0x prefix, hashed under EIP-191 as UTF-8 text), must return 0x946324E0E5d7D77206731E35Ef4044a383e2a8C2, the same signer published in the agent card at /.well-known/agent-card.json. This is the step that ties the number to an identity rather than merely to a hash.

An unpaid request to POST /api/perp-gate answers with the x402 402 challenge, so the shape of the payment surface costs nothing to inspect. All four checks then run **offline, from the repository and this page alone** — no live call and no pay-

ment. The reason is that `proof.inputs` is printed above in full and the engine is open source, so the result object is not merely *received*: it can be *regenerated*. Re-running the published perp-gate engine on those five inputs and sealing it exactly as the service does reproduces this exhibit in every digit — liquidation price 58329.11, self-check residual 2.05e-12, and the content hash above byte-for-byte. That reproduction is itself locked by a test in the suite, so an engine change that stales this appendix fails the build rather than quietly misleading a reader. A paid call proves that the service *served* it; the repository proves that it is *right*. Neither requires the other, and at this service's price the paid confirmation is one cent.

One claim deserves singling out, because it is the one a sceptical reader should press hardest. The crash study's out-of-sample status rests on thresholds having been fixed before the 2026 events were examined. The evidence for that is narrower than an anchored log would be: the thresholds are literal constants in the published query files, the calibration set contains only 2025 episodes, and the repository's commit history shows the order the files appeared in. All three of those are under our control. We did not anchor a hash of them before running the tests, which is what would settle it, and an anchor made now cannot back-date itself. The manifest above freezes them from this point forward; it does not vouch for what came before it. A reader who discounts the out-of-sample claim on that basis is reasoning correctly.

What this appendix does not do is make the unverifiable parts verifiable. The population-scale crash replay, the test suite, the availability record and the buyer audit are still assertions backed by scripts in the repository rather than by anything a reader can check in a browser; the honest way to read them is as claims from a source that has, in the artifacts above, given the reader the means to catch it lying about the parts that can be checked.

References

Every entry below is cited in the text and carries part of the argument. An earlier draft listed a further twenty-nine background sources that nothing in the document depended on, under a note telling the reader to ignore them — a bibliography a third of which its own author disclaims is decoration, so it was removed rather than defended. Three entries also carried citation markers inside their own titles, an artefact of an earlier edit; a reference cannot cite itself, and those are gone too.

1. F. Black, "The pricing of commodity contracts," *Journal of Financial Economics*, vol. 3, no. 1–2, pp. 167–179, 1976.
2. F. Black and M. Scholes, "The pricing of options and corporate liabilities," *Journal of Political Economy*, vol. 81, no. 3, pp. 637–654, 1973.
3. R. C. Merton, "Theory of rational option pricing," *The Bell Journal of Economics and Management Science*, vol. 4, no. 1, pp. 141–183, 1973.
4. D. T. Breeden and R. H. Litzenberger, "Prices of state-contingent claims implicit in option prices," *The Journal of Business*, vol. 51, no. 4, pp. 621–651, 1978.
5. J. Gatheral, "A parsimonious arbitrage-free implied volatility parameterization with application to the valuation of volatility derivatives," presentation, Global Derivatives & Risk Management, Madrid, 2004.
6. J. Gatheral and A. Jacquier, "Arbitrage-free SVI volatility surfaces," *Quantitative Finance*, vol. 14, no. 1, pp. 59–71, 2014. arXiv:1204.0646.
7. J. Gatheral, *The Volatility Surface: A Practitioner's Guide*. Hoboken, NJ: Wiley, 2006.
8. R. W. Lee, "The moment formula for implied volatility at extreme strikes," *Mathematical Finance*, vol. 14, no. 3, pp. 469–480, 2004.
9. A. Castagna and F. Mercurio, "The vanna-volga method for implied volatilities," *Risk*, vol. 20, no. 1, pp. 106–111, 2007.
10. A. S. Kyle, "Continuous auctions and insider trading," *Econometrica*, vol. 53, no. 6, pp. 1315–1335, 1985.
11. Y. Amihud, "Illiquidity and stock returns: cross-section and time-series effects," *Journal of Financial Markets*, vol. 5, no. 1, pp. 31–56, 2002.
12. D. Easley, M. M. López de Prado, and M. O'Hara, "Flow toxicity and liquidity in a high-frequency world," *The Review of Financial Studies*, vol. 25, no. 5, pp. 1457–1493, 2012.
13. R. Almgren and N. Chriss, "Optimal execution of portfolio transactions," *Journal of Risk*, vol. 3, no. 2, pp. 5–39, 2000.
14. C. M. C. Lee and M. J. Ready, "Inferring trade direction from intraday data," *The Journal of Finance*, vol. 46, no. 2, pp. 733–746, 1991.
15. R. Almgren, C. Thum, E. Hauptmann, and H. Li, "Direct estimation of equity market impact," *Risk*, vol. 18, no. 7, pp. 58–62, 2005.
16. G. Bakshi and N. Kapadia, "Delta-hedged gains and the negative market volatility risk premium," *The Review of Financial Studies*, vol. 16, no. 2, pp. 527–566, 2003.
17. P. Carr and L. Wu, "Variance risk premiums," *The Review of Financial Studies*, vol. 22, no. 3, pp. 1311–1341, 2009.
18. T. Bollerslev, G. Tauchen, and H. Zhou, "Expected stock returns and variance risk premia," *The Review of Financial Studies*, vol. 22, no. 11, pp. 4463–4492, 2009.
19. C. Acerbi and D. Tasche, "On the coherence of expected shortfall," *Journal of Banking & Finance*, vol. 26, no. 7, pp. 1487–1503, 2002.
20. "EIP-712: Typed structured data hashing and signing," Ethereum Improvement Proposals, no. 712, 2017. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-712>
21. "EIP-2612: Permit — 712-signed approvals," Ethereum Improvement Proposals, no. 2612, 2020. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-2612>
22. "EIP-1967: Proxy Storage Slots," Ethereum Improvement Proposals, no. 1967, 2019. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-1967>
23. "EIP-7702: Set EOA account code," Ethereum Improvement Proposals, no. 7702, 2024. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-7702>
24. "EIP-3009: Transfer With Authorization," Ethereum Improvement Proposals, no. 3009, 2020. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-3009>
25. Uniswap Labs, "Permit2," 2022. [Online]. Available: <https://github.com/Uniswap/permit2>

26. M. De Rossi, D. Crapis, J. Ellis, and E. Reppel, "ERC-8004: Trustless Agents," Ethereum Improvement Proposals, no. 8004, 2025. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-8004>
27. Coinbase, "x402: A payments protocol for the internet — Specification v2," 2025. [Online]. Available: <https://github.com/coinbase/x402>
28. Google, "Agent2Agent (A2A) Protocol Specification," 2025. [Online]. Available: <https://a2a-protocol.org>
29. Anthropic, "Model Context Protocol (MCP) Specification," 2024. [Online]. Available: <https://modelcontextprotocol.io>
30. J. A. Nelder and R. Mead, "A simplex method for function minimization," *The Computer Journal*, vol. 7, no. 4, pp. 308–313, 1965.
31. M. Abramowitz and I. A. Stegun, *Handbook of Mathematical Functions*, formula 7.1.26. Washington, DC: National Bureau of Standards, 1964.
32. Apache Software Foundation, "Apache ECharts." [Online]. Available: <https://echarts.apache.org>
33. "technicalindicators (npm package), v3.1.0." [Online]. Available: <https://www.npmjs.com/package/technicalindicators>
34. OpenJS Foundation, "node:test — the built-in Node.js test runner." [Online]. Available: <https://nodejs.org/api/test.html>
35. ScamSniffer, "Web3 Scam Report: Wallet Drainers," 2024. [Online]. Available: <https://scamsniffer.io>
36. S. Figlewski, "Estimating the implied risk-neutral density for the U.S. market portfolio," in *Volatility and Time Series Econometrics*, T. Bollerslev, J. Russell, and M. Watson, Eds. Oxford: Oxford University Press, 2010.
37. A. M. Malz, "Estimating the probability distribution of the future exchange rate from option prices," *The Journal of Derivatives*, vol. 5, no. 2, pp. 18–36, 1997.
38. G. J. Jiang and Y. S. Tian, "The model-free implied volatility and its information content," *The Review of Financial Studies*, vol. 18, no. 4, pp. 1305–1342, 2005.
39. M. Beiglböck, P. Henry-Labordère and F. Penkner, "Model-independent bounds for option prices — a mass transport approach," *Finance and Stochastics*, vol. 17, no. 3, pp. 477–501, 2013.
40. Coinbase, x402 adoption reporting, April 2026: 69,000 active agents and 165M transactions.
41. x402 first-year totals: 169M+ payments across 590,000 buyers and 100,000 sellers, reported at the protocol's one-year mark, 2026.
42. M. Zhang et al., "FAITH: a framework for assessing intrinsic tabular hallucinations in finance," arXiv:2508.05201, 2025.
43. Nof1, "Alpha Arena" season 1, 18 October – 3 November 2025: six frontier models each trading \$10,000 of perpetuals autonomously on Hyperliquid. Final standings: Qwen3-Max +22.3%, DeepSeek V3.1 +4.9%, Claude Sonnet 4.5 and Gemini 2.5 Pro each down more than 40%, Grok 4 about –58%, GPT-5 last at –62.66%. Four of six ended in the red; one passed –60%.
44. J. Milionis, C. C. Moallemi, T. Roughgarden and A. L. Zhang, "Automated market making and loss-versus-rebalancing," arXiv:2208.06046, 2022.

Quiver technical documentation, version 1.0, July 2026. Prepared for the OKX AI Genesis Hackathon. All quantitative figures were captured from live venues during validation and are representative of the methods described; live values vary with market state. This document is technical documentation and not financial advice.